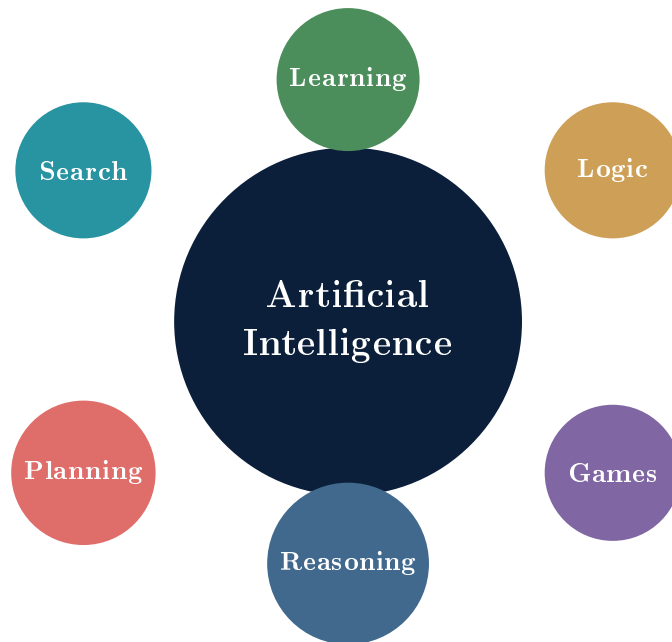


Artificial Intelligence

Search Methods for Problem Solving

A Complete Master E-Book of Study Notes

Weeks 1–12 • Concepts • Algorithms • Worked Examples
Proofs • Quizzes • PYQs • Exam Strategy



Made by Anmol Kansal with AI

Reference: NPTEL “AI: Search Methods for Problem Solving”
IIT Madras | Prof. Deepak Khemani

This e-book is a comprehensive, beginner-friendly companion that *replaces* the original lecture slides, transcripts, and class notes for the course.

Preface and How to Use This Book

How this book is organised

Part I — Complete Notes (Weeks 1–12): every concept from the syllabus, expanded from slides and lecture transcripts into a true textbook. Each week ends with a Quick Revision sheet, important exam points, and practice questions.

Part II — PYQ Solutions: thoroughly worked solutions to past-year questions, with thought process, traps, and shortcuts.

Part III — Concepts Found Only in PYQs: anything that appears in PYQs but isn't in the main syllabus.

Teaching philosophy. I assume you have *never* studied AI before. Every algorithm is first explained in plain English (*intuition first, math later*), then formally, then with a worked example, and finally with edge cases and exam tips. Special coloured boxes highlight definitions, intuition, common mistakes, exam traps, and real-world applications.

Notation conventions.

- S, s denote *states*; S_0 is the start state; G (or S_g) is a goal state.
- b = branching factor (avg. children per node); d = depth of shallowest goal; m = max depth of search tree.
- $g(n)$ = cost from start to node n ; $h(n)$ = heuristic estimate from n to a goal; $f(n) = g(n) + h(n)$.
- “OPEN” / “CLOSED” are the standard search lists (frontier / explored set).

Contents

Preface	1
Part I — Complete Notes (Weeks 1–12)	3
1 Week 1: Introduction, Philosophy, and the Landscape of AI	3
2 Week 2: State Space Search, DFS, BFS, and Iterative Deepening	21
3 Week 3: Heuristic Search, Local Search, and Escaping Local Optima	38
4 Week 4: Population-Based Methods, Genetic Algorithms, and Ant Colony Optimization	54
5 Week 5: Finding Optimal Paths – Branch & Bound, Dijkstra, and Algorithm A*	65
6 Week 6: The Monotone Condition, Space-Saving A*, and Sequence Alignment	76
7 Week 7: Game Playing – Minimax, Alpha-Beta, and SSS*	90
8 Week 8: Automated Domain-Independent Planning	105
9 Week 9: Problem Decomposition and Algorithm AO*	120
10 Week 10: Pattern-Directed Inference Systems and the Rete Algorithm	131
11 Week 11: Constraint Satisfaction Problems	143
12 Week 12: Course Revision and the Big Picture of AI	156

Part I — Complete Notes (Weeks 1–12)

1 Week 1: Introduction, Philosophy, and the Landscape of AI

“The fundamental goal of Artificial Intelligence research is not merely to mimic intelligence or produce some clever fake. AI wants the genuine article: machines with minds, in the full and literal sense.”

— John Haugeland, *AI: The Very Idea* (1985)

Goal of Week 1. Before we touch a single algorithm, we must answer: *What problem is AI trying to solve? Where does search sit inside the much bigger picture of AI?* This week is part philosophy, part history, and part road-map. You will leave knowing (a) what an *intelligent agent* is, (b) what *symbolic reasoning* means, (c) the Turing Test and its modern successor (the Winograd Schema Challenge), and (d) why *search* is the workhorse of classical AI.

1.1 What is Artificial Intelligence?

There is no single, universally accepted definition of AI. Rather than pick one, look at how leading thinkers have phrased the goal:

Four classical definitions of AI

- **Herbert Simon:** “We call programs intelligent if they exhibit *behaviors* that would be regarded as intelligent if they were exhibited by human beings.”
- **Barr & Feigenbaum:** “Physicists ask what kind of place this universe is. Biologists ask what it means for a system to be living. *We in AI wonder what kind of information-processing system can ask such questions.*”
- **Elaine Rich:** “AI is the study of techniques for solving *exponentially hard problems in polynomial time* by exploiting knowledge about the problem domain.”
- **Charniak & McDermott:** “AI is the study of mental faculties through the use of computational models.”

Notice two different camps already:

- One camp (**Simon, Rich**) wants *useful programs* that do intelligent-looking things.
- The other camp (**Barr–Feigenbaum, Charniak–McDermott, Haugeland**) wants to *understand thinking itself* as a computational process.

Intuition

You can think of AI as having two faces: a **practical face** (build self-driving cars, expert systems, chatbots, planners) and a **cognitive face** (figure out what intelligence *is*). The course we are starting leans more towards the practical face — but the philosophy is what keeps us honest about what we are really doing.

Philosophical Note

Elaine Rich's definition is subtle and important. She says AI solves exponentially hard problems in polynomial time. Strictly, no algorithm can do this (unless $P=NP$). What Rich means is: by using *domain knowledge*, we sacrifice the guarantee of an optimal answer and get a *good enough* answer fast. This trade-off is the soul of heuristic search, which is the dominant theme of this course.

1.2 The Intelligent Agent

The whole course is framed around one entity: the **intelligent agent**. Understand this picture once, and everything else slots into place.

Definition

Intelligent Agent. An autonomous program (or robot) that:

- **Perceives** its environment through sensors,
- **Deliberates** using an internal *model* of the world,
- **Acts** on the environment through effectors, in order to pursue *goals*.

1.2.1 The Four Characteristics of an Agent

Khemani gives four properties that an agent must satisfy:

1. **Persistent.** It is “there all the time” — running continuously, not a one-shot script. A persistent agent maintains state between perceptions.
2. **Autonomous.** It decides what to do on its own — there is no human reaching in at every step.
3. **Proactive.** It can *generate* its own subgoals. (A self-driving car doesn't just react; it plans a route, schedules charging stops, etc.)
4. **Goal-directed.** Given a goal — by a user, or by itself — the agent works towards achieving it.

Memory Trick

Mnemonic: “**P-A-P-G**” = **P**ersistent, **A**utonomous, **P**roactive, **G**oal-directed. Read as “Papa-G”. Every intelligent agent is your “Papa-G”!

1.2.2 The Agent Carries a Model of the World

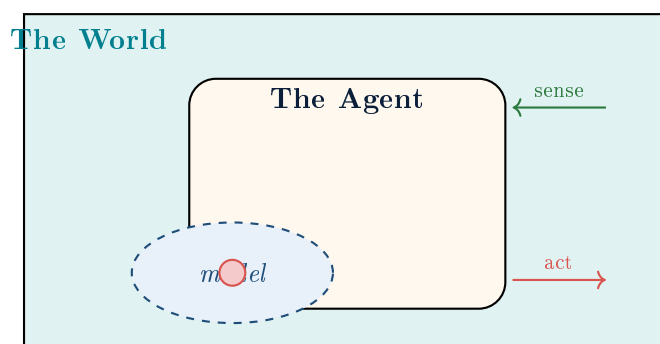
Core idea

An intelligent agent in a world carries a **model of the world** in its “head.”

The model may be an *abstraction*.

A *self-aware* agent would model itself *inside* that world model.

Why a model at all? Because the agent cannot act intelligently by reflex alone. It must *imagine* the consequences of actions before performing them. The model is the agent's private theory of how the world works. Critically, this model is always an **abstraction** — you cannot store every atom of the real world inside your head, so you keep what matters and throw away the rest.



Intuition

A chess agent doesn't carry the 32 *physical* chess pieces in its head; it carries an abstract 8×8 board where each cell holds a piece-type. The model is *enough* to reason about chess — that's all it needs.

1.3 The Three Pillars of Intelligence

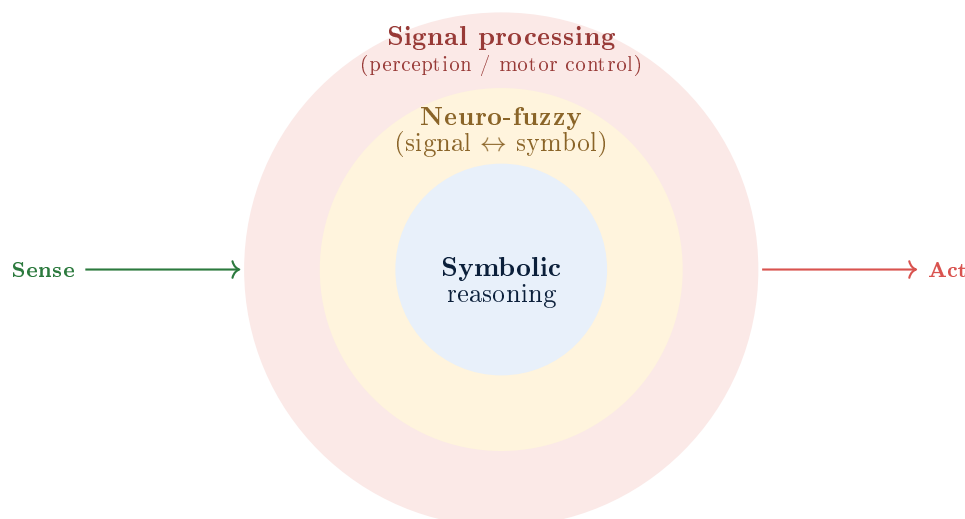
Khemani frames intelligence in three time-oriented capacities:

Remember the past (Learn from it)	Case-based reasoning, Machine learning, Pattern recognition via deep neural networks.
Understand the present (Be aware of the world)	Knowledge representation, Logic and reasoning (inference).
Imagine the future (Work towards goals)	Heuristic search , Automated planning — <i>this is the course</i> .

This course focuses overwhelmingly on the third row: *imagining the future via search*. We touch the second row at the end (constraint reasoning, rules, planning).

1.4 The Information-Processing View: Three Layers of an Autonomous Agent

Khemani draws the autonomous agent as a set of *concentric layers*, like the cross-section of an onion.



- **Outer layer — Signal processing.** The agent interacts with the world via raw signals: photons hitting a camera, sound waves at a microphone, currents driving motors. This is the layer of computer vision, speech recognition, robot control.
- **Middle layer — Neuro-fuzzy.** Signals are converted into *symbols*. This is the realm of neural networks, deep learning, classifiers. A picture of a cat becomes the symbol `cat`.
- **Inner layer — Symbolic reasoning.** Once we have symbols, we can reason *logically* about them. This is what Haugeland calls *GOF AI* — *Good Old-Fashioned AI*, and this is where **search** lives.

Remember This

The course operates in the **innermost (symbolic) layer**. Everything we will discuss — DFS, A*, Minimax, Planning, CSPs — manipulates symbols, not signals. We *assume* that perception has already done its job and handed us a symbolic description of the world.

1.5 Building Blocks of AI (And What This Course Covers)

AI is not one algorithm. It is a society — Marvin Minsky’s word — of components. A short tour:

Perception	Computer vision, speech recognition, tactile sensing.
Learning	Machine learning, deep learning, reinforcement learning.
Memory	Case-based reasoning, knowledge bases.
Knowledge representation	Ontologies, logic, semantics, qualitative reasoning.
Search and problem solving	The focus of this course.
Planning	Choosing sequences of actions to reach goals.
Adversarial reasoning	Game playing — when the world contains opponents.
Constraints	The “holy grail” that subsumes search and reasoning.
Action	Natural-language generation, speech synthesis, motor control.

Exam Tip

In a definition-style question, you may be asked: *Which of the following are within the scope of classical AI?* The course-relevant answers are bolded above: **search, planning, adversarial reasoning, constraints, logical deduction**.

1.6 A Decade of Machine Learning (Why ML $\neq$ AI)

Many newcomers equate AI with machine learning. This is mostly wrong, and Week 1 sets the record straight.

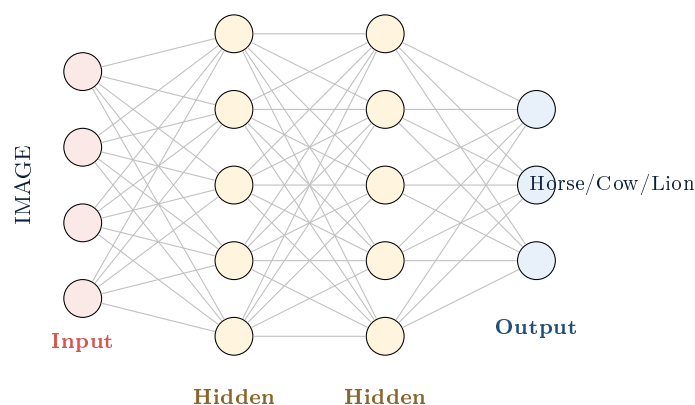
1.6.1 A Brief Story of Neural Networks

- **1943** — **McCulloch & Pitts** propose a mathematical model of a neuron.
- **1958** — **Rosenblatt’s Perceptron** is a single-layer network that classifies linearly-separable data.
- **1969** — **Minsky & Papert** prove the Perceptron *cannot* learn even the simple XOR function (because XOR is not linearly separable). This famously starts an “AI winter” for neural nets.
- **1986** — **Rumelhart, Hinton & Williams** re-popularize the multi-layer perceptron and backpropagation: with hidden layers, networks can learn *any* non-linear classifier.

- **2012** — **Hinton et al.** demonstrate that *deep* networks (many hidden layers) trained with GPUs and lots of data dramatically outperform classical methods at image recognition (ImageNet moment).
- **2018** — Hinton, LeCun, and Bengio win the Turing Award for deep learning.

1.6.2 What Deep Networks Are (and Aren't)

A **feed-forward deep neural network** maps an input image (or signal) through a series of *hidden layers* to an output classification:



Information flows left to right. The network is *trained* by adjusting weights (via back-propagation) so that for each labelled image the correct output neuron lights up.

1.6.3 The Performance vs. Competence Critique

Here is the punchline from Rodney Brooks (and Khemani highlights this):

Philosophical Note

A computer that labels an image “people playing Frisbee in a park” has *no idea* what a Frisbee is, whether you can eat one, how far one can be thrown, that parks are usually outdoors, or what weather is. It has **performance** (labelling) without **competence** (understanding).

This is why ML alone is not AI. To be intelligent, you must also *reason* about what you see — and that is where symbolic AI (and this course) comes in.

1.6.4 Suitcase Words

Minsky called words like “learning”, “intelligence”, and “understanding” **suitcase words**: words that pack many very different meanings into one syllable. When you hear “machine learning,” it is tempting to imagine the way *you* learn (curiously, generally, with little data). In reality, an ML system needs *huge* amounts of carefully labelled data and a custom architecture for each new problem.

Remember This

Animal-like abilities vs. Human-level intelligence (Judea Pearl, Adnan Darwiche): “Eagles and snakes have better vision than us; cats navigate better; parrots mimic speech.” These animals are not human-intelligent. Modern ML is closer to animal-like abilities than to genuine reasoning. The reasoning layers — the inner symbolic layer — are

what is missing, and what we will study.

1.7 Knowledge and Reasoning: The Core of Intelligence

Definition

The central question of knowledge representation.

What does the agent *know*, and what *else* does the agent know as a consequence of what it knows?

We care about **explicit declarative knowledge** (facts, rules, statements you can write down), not implicit procedural or tacit knowledge (like how to ride a bicycle — easy to do, very hard to put into symbols).

1.7.1 Representation: The Semiotics of AI

Semiotics is the study of signs. A **symbol** is a perceptible something that stands for something else.

- The character “7”, the Roman numeral “VII”, and the word “seven” are three different symbols for the same number.
- Road signs (a triangle with an exclamation = “danger ahead”) are symbols.
- Spoken and written language is one giant semiotic system.

Biosemitotics extends this to biology: pheromone trails left by ants are symbols by which simple agents communicate, producing complex emergent behaviour. (We’ll revisit this in Week 4 under Ant Colony Optimization.)

1.7.2 Reasoning: Symbol Manipulation

Definition

Reasoning. The *manipulation of symbols in a meaningful manner*.

Mathematics is full of reasoning algorithms you already use:

- Multi-digit addition and multiplication,
- Long division,
- Solving a system of linear equations,
- Fourier transforms, convolution, ...

All of these are *symbol manipulation rules*. The bold claim of symbolic AI — going back to Hobbes — is that *thinking is the same activity*.

1.8 A Whirlwind History of AI

The story of AI is older than computers. Here is the cast of characters that matters for Week 1.

1.8.1 Ancient and Medieval Imagination

Humanity has imagined “artificial people” for millennia:

- **Talos** — the bronze giant of Crete in Homer’s *Iliad*, created by Hephaestus.
- **Pandora** — also crafted by Hephaestus; opening her box releases evils on mankind.
- **Pygmalion’s Galatea** — an ivory statue brought to life by Aphrodite.
- **Daedalus’s wheezing statues** — described as scuttling about and impressing onlookers.
- **Golem** (Judah Loew ben Bezalel, ~1580 Prague) — an animated clay man made to defend the Jews of Prague.
- **Paracelsus’s homunculus** (~1530) — a “little man” grown alchemically.

1.8.2 The Zairja and Ars Magna

Arab astrologers in the medieval period built a device called the **Zairja**, designed “to generate ideas by mechanical means” using letter-categories and number values. The Catalan missionary **Ramon Lull** (1232–1315) was inspired to build a Christian version called **Ars Magna** — “to bring reason to bear on all subjects and arrive at truth without the trouble of thinking or fact finding.” These are the very first attempts at *mechanizing* reasoning.

1.8.3 The Mechanical Era

- **Pascal (1642)** — the Pascaline, a mechanical adder/subtractor.
- **Leibniz (~1673)** — the Stepped Reckoner, capable of all four arithmetic operations. *Leibniz dreamed of a **calculus ratiocinator**: “Let us calculate, without further ado, to see who is right.”* This is essentially the dream of symbolic AI.
- **Vaucanson’s mechanical duck (1739)** — could quack, splash, and “digest” (well, fake it).
- **Von Kempelen’s Mechanical Turk (1770)** — a chess-playing automaton that defeated Napoleon and Benjamin Franklin. . . later exposed as a fraud (a human master hid inside).
- **Babbage (1822, 1837)** — the Difference Engine and the Analytical Engine. The Analytical Engine had an arithmetic unit, control flow, conditional branching, and memory: it was Turing-complete *on paper, in 1837*.
- **Ada Lovelace (1843)** — wrote what is considered the first computer program (an algorithm to compute Bernoulli numbers). Crucially, she saw that the engine could “act on *any* things” whose relations could be expressed symbolically — even compose music. The first programmer was also the first prophet of AI.
- **ENIAC (1946)** — the first electronic general-purpose computer. 17,468 vacuum tubes, 27 tons, 167 m², 150 kW of power.

1.8.4 The Philosophical Pillars

- **Galileo (~1623)**: the universe is written in mathematics; all reality is mathematical.
- **Thomas Hobbes (1655)**: “*Reasoning is computation. . . to reason is the same as to add or subtract.*” Hobbes is sometimes called the **grandfather of AI**.
- **Descartes (~1640)**: animals are wonderful machines; humans are too, except they have a mind. Thoughts themselves are symbolic representations. He also introduces the *mind-body dualism*: how can a non-physical mind interact with a physical body?

Philosophical Note

Haugeland's "Paradox of Mechanical Reason":

If reasoning is the manipulation of meaningful symbols according to rational rules, who is doing the manipulating? Can a mechanical manipulator ever pay attention to meaning?
This is the question modern AI still wrestles with.

1.8.5 The Birth of the Field (1956)

The name **Artificial Intelligence** was coined by **John McCarthy** for the **Dartmouth Conference** of 1956. The proposal called for a “two-month, ten-man study” on the conjecture “that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.”

The four organizers:

- **John McCarthy** (Dartmouth) — would invent LISP, the language of classical AI.
- **Marvin Minsky** (Harvard) — would co-found the MIT AI Lab, write *Society of Mind*.
- **Nathaniel Rochester** (IBM) — designed the IBM 701 and wrote the first assembler.
- **Claude Shannon** (Bell Labs) — already famous for information theory.

The **show-stealers**: **Herbert Simon** and **Allen Newell** (Carnegie Tech / RAND) arrived as “two vaguely known persons” invited almost as an afterthought. They brought the **Logic Theorist (LT)** — the first deliberately engineered intelligent program, which went on to prove theorems from *Principia Mathematica*, sometimes more elegantly than Russell and Whitehead themselves!

1.9 The Physical Symbol System Hypothesis**Definition****Physical Symbol System Hypothesis (Newell & Simon).**

“A physical symbol system has the *necessary and sufficient* means for general intelligent action.”

Read this carefully:

- **Necessary** = any genuinely intelligent system must be (or contain) a physical symbol system.
- **Sufficient** = if you build a physical symbol system rich enough, you have built intelligence.

This is the ideological backbone of **Symbolic AI** / **Classical AI** / **GOF AI** — and it is the framework inside which search, planning, logic, and CSPs all live.

1.9.1 What is a “Physical Symbol System”?

Three increasingly specific definitions:

- **Symbol.** A perceptible token that stands for something else.
Examples: numerals, alphabet letters, road signs, musical notes.
- **Symbol system.** A structured collection of symbols — a pattern.
Examples: words, lists, arrays, a melody.
- **Physical symbol system.** A symbol system that obeys laws — a *formal* system.
Examples: long division, the abacus, an algorithm.

Remember This

In one sentence: The PSSH says intelligence is computation over symbols. Not signals, not numbers in the narrow sense — *symbols whose meaning is conferred by the system's behaviour.*

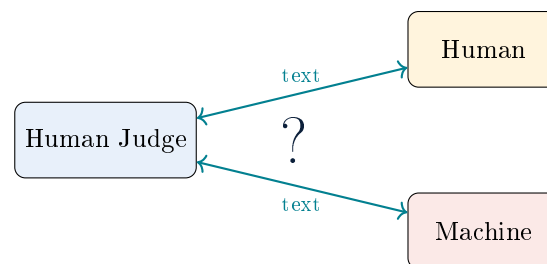
1.10 The Turing Test (Alan Turing, 1950)**1.10.1 Why “Can Machines Think?” Is the Wrong Question**

In 1950 Alan Turing wrote *Computing Machinery and Intelligence*. He observed that the question “Can machines think?” is *too meaningless* to admit a scientific answer — it depends on what you mean by “machine” and “think.”

Turing’s move: **replace the question by an operational test.**

1.10.2 The Imitation Game**The Turing Test (Imitation Game)**

A human **judge** converses (by text, to remove voice/appearance cues) with two hidden entities: a **human** and a **machine**. The judge must determine which is which. If a sufficiently long sample of judges, after sufficiently long conversations, cannot reliably distinguish the machine from the human, then *by definition* the machine is said to be **intelligent**.

**1.10.3 Strengths and Criticisms**

- **Strength:** a *behavioural* test — neutral about brain-versus-silicon, mind-versus-mechanism. If it behaves like an intelligent being, it is one.
- **Criticism 1 (the multiplication trick):** Hand the unknown entity two 11-digit numbers and ask for the product. A human will (almost always) fail; a computer will instantly answer. The judge can use this asymmetry to detect the machine. So the machine has to *deliberately dumb itself down* — which feels like the wrong test for intelligence.
- **Criticism 2 (chatbots that fake it):** Modern programs use cheap tricks (rephrasing, distraction, canned responses) to fool judges briefly. The Loebner Prize, an annual Turing Test contest, has produced “winners” that no one would call intelligent.

1.10.4 ELIZA — The First Chatbot

Joseph Weizenbaum (MIT, 1966) wrote **ELIZA**, named after Eliza Doolittle from Shaw’s *Pygmalion*. A popular script called **DOCTOR** mimicked a Rogerian psychotherapist by simple pattern matching:

Visitor: I'm feeling a bit tired.

DOCTOR: Why do you think you're feeling a bit tired?

Visitor: Well, I've been traveling, away from home.

DOCTOR: Tell me about your family.

Weizenbaum's own secretaries asked him to leave the room so they could "talk to the program in private." He was so disturbed by how readily people projected intelligence onto ELIZA that he wrote a book criticising the very idea of intelligent machines: *Computer Power and Human Reason*.

Intuition

ELIZA shows that **passing the Turing Test is not the same as being intelligent**. Humans are wired to anthropomorphise — we'll see a face in a cloud and a soul in a chat window. We need a better test.

1.11 The Winograd Schema Challenge (Levesque, 2011)

Hector Levesque proposed a stronger, *Google-proof* alternative to the Turing Test, based on a kind of sentence first studied by Terry Winograd in 1972.

1.11.1 Anatomy of a Winograd Schema

A Winograd Schema Challenge (WSC) question has three parts:

1. **A sentence** (or short discourse) containing:
 - Two noun phrases of the same semantic class (e.g. both male humans, or both inanimate objects),
 - An *ambiguous pronoun* that could refer to either,
 - A **special word**, and an *alternate word* such that swapping them changes which noun phrase the pronoun naturally refers to.
2. **A question** asking which noun phrase the pronoun refers to.
3. **Two answer choices**.

Crucially the machine receives a *standardised form* with both answer options. It is forced into a binary choice. Statistical co-occurrence "which words show up near which other words on the internet" *cannot* solve these, because the only word that changes is grammatically neutral.

1.11.2 Four Classic Examples

Winograd Schema — Example 1 (Terry Winograd, 1972)

"The city councilmen refused the demonstrators a permit because **they** feared violence."
"The city councilmen refused the demonstrators a permit because **they** advocated violence."

Who does *they* refer to?

- Answer 0: The demonstrators
- Answer 1: The councilmen

Trick: only "feared" vs. "advocated" changes. The first → councilmen (they were the fearful ones), the second → demonstrators (they were the advocates of violence).

Example 2 (IJCAI 2016)

“John took the water bottle out of the backpack so that **it** would be *lighter*.”

“John took the water bottle out of the backpack so that **it** would be *handy*.”

First sentence: *it* = the backpack (removing weight makes the backpack lighter).

Second sentence: *it* = the bottle (the bottle is now handier to reach).

Example 3

“The trophy would not fit in the brown suitcase because **it** was too *small*.”

“The trophy would not fit in the brown suitcase because **it** was too *big*.”

Small: *it* = the suitcase. **Big:** *it* = the trophy.

Example 4

“The lawyer asked the witness a question, but **he** was reluctant to *repeat* it.”

“The lawyer asked the witness a question, but **he** was reluctant to *answer* it.”

Repeat: the lawyer was reluctant. **Answer:** the witness was reluctant.

Remember This

The Winograd Schema Challenge tests **world knowledge and common-sense reasoning**. To answer correctly, the machine must understand what kinds of things are big or small, what fitting means, what an answer is, etc. This is *competence*, not just performance.

Exam Tip**Exam-favourite distinction:**

- **Turing Test:** behavioural, conversational, can be passed by chatbots using superficial tricks.
- **Winograd Schema Challenge:** forced-choice on linguistic ambiguities, *Google-proof*, requires actual world knowledge.

A typical question: “*Why is the Winograd Schema Challenge considered Google-proof?*”
 Answer: because the only words that change between the two sentence variants are grammatically and statistically very similar (e.g. “feared” vs. “advocated”), so corpus-based co-occurrence statistics give roughly the same prediction for both. Only common-sense reasoning can disambiguate.

1.12 Minds and Machines: Reality, Perception, and Ontology

A short philosophical bridge before we get to algorithms.

1.12.1 Reality is Made of Particles — but Reasoning is Not

The physical world is composed of fundamental particles (quarks, electrons, etc.). The laws of physics in principle explain everything. But:

- A human adult is roughly 10^{27} atoms.
- These atoms interact with zillions more atoms in the environment continuously.
- Even *describing* a grain of rice at the atomic level is hopeless.

So we **abstract**. We pick a level of description — atoms, molecules, cells, organs, people, societies — that is right for our problem.

1.12.2 Ontology — Each Domain Has Its Own Vocabulary

Definition

An **ontology** is the formal vocabulary (*the objects, properties, and relations*) that a given domain of study uses for representation and reasoning.

A short “Powers of Ten” tour of domains:

Scale	Domain (ontology)
10^{-17} m	quarks, gluons — sub-atomic physics
10^{-15} m	protons — chemistry begins
10^{-9} m	DNA, molecules — cell biology
10^{-5} m	bacteria — biology
1 m	people, doors — Newtonian physics, psychology, physiology
10^3 – 10^5 m	towns, geography — sociology, economics
10^{11} m	Earth–Sun distance — astrophysics, astronomy
10^{26} m	Visible Universe — cosmology

Intuition

The lesson: when you build an AI for chess, your ontology is pieces and squares — not quarks. When you build a planner for a hospital, your ontology is patients, beds, doctors, and meals — not cells. Choose the smallest ontology that captures what your problem needs.

1.12.3 The World in Our Minds

The physical world operates by physics. But we, the thinking creatures, build our own worlds in our minds — and *only* those mental worlds are *meaningful* to us. (Movies like *The Matrix* and *Inception* play on this idea.) An AI agent’s “world” is similarly its internal model.

1.13 Problem Solving: The Setup for the Rest of the Course

We now arrive at the central frame of the course.

Definition

A **problem-solving agent** is an autonomous agent in some world that has

- a **goal** to achieve, and
- a **set of actions** to choose from,
- which it uses to *strive* for the goal.

1.13.1 The Football Vignette (Khemani’s Running Example)

Picture yourself as a striker on the blue team, running with the ball. The defenders close in. **What should you do?** An intelligent agent would *pass the ball to a free teammate who has a better chance to score*. To do this, you must:

- **Perceive** the positions of teammates and opponents,
- **Model** the situation internally,
- **Imagine** the consequence of various actions,
- **Choose** the best one.

In the real world, the situation is complicated by other agents acting simultaneously and by uncertainty. So we make *simplifying assumptions*.

1.13.2 The Six Simplifying Assumptions (“First You Walk, Then You Run”)

Standard simplifying assumptions for classical search

For most of this course we will assume that:

1. The world is **static** — nothing changes unless the agent acts.
2. The world is **completely known** — the agent has perfect information.
3. Only one agent changes the world — **no other actors** (relaxed in Week 7, Game Playing).
4. Actions **never fail** — they are deterministic.
5. The representation of the world is given — **you don’t have to invent it**.
6. (Implicitly) actions are **discrete** — finite in number, atomic in time.

Exam Tip

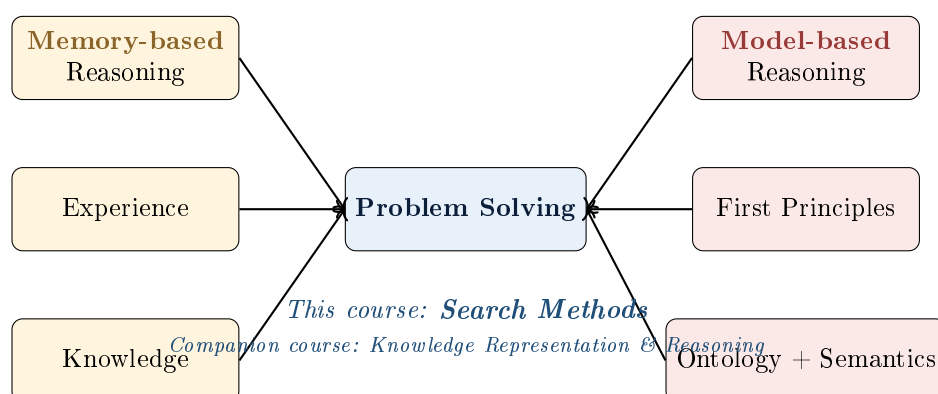
The above six assumptions are a classic exam question. “*State the assumptions made by classical search algorithms.*” Memorize them.

Memory Trick

Mnemonic: “SCOAR-D” — **S**taic, **C**ompletely known, **O**ne agent, **A**ctions never fail, **R**epresentation given, **D**iscrete actions.

1.14 Humankind: a Problem-Solving Species

Khemani draws a beautiful summary diagram of how humans (and AI agents) solve problems. There are two complementary routes:



- **Memory-based reasoning** (left branch) — solve by recalling. “*I have seen this before; here is what worked.*” This is the world of case-based reasoning, machine learning.

- **Model-based reasoning** (right branch) — solve from scratch using a model. “*From first principles, here is what should work.*” This is the world of search and planning — **our world**.

1.14.1 Rubik’s Cube: Search vs. Knowledge

Two ways to solve Rubik’s cube:

- **Knowledge-based:** Memorize the standard human algorithm. A child can solve any scrambled cube in minutes. But *somebody had to invent* that algorithm — Rubik himself took a month.
- **Search-based:** With no knowledge, try moves and see what happens. Deep reinforcement learning (DeepCubeA, 2018) discovered efficient solutions from scratch with no human guidance.

1.14.2 Sudoku: Search vs. Reasoning

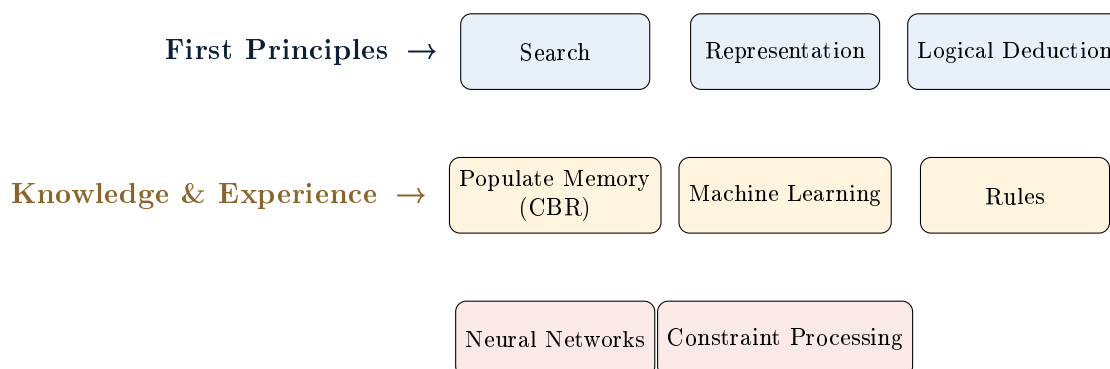
In a Sudoku puzzle, you can often *reason* (“5 is already in this row and column, so 5 must go in the unique empty cell”) without any search at all. When pure reasoning runs out, you *search* (try a value, backtrack if it fails).

Remember This

Both reasoning and search are subsumed by Constraint Satisfaction — the topic of Week 11. In a CSP framework, “reasoning” is constraint propagation; “search” is backtracking. They cooperate.

1.15 An Architecture for Problem Solving

A complete problem-solving agent uses multiple techniques in concert:



This course mostly visits the top row (search, representation, logical deduction) plus constraint processing.

1.16 The Map-Colouring Teaser (preview of Week 11)

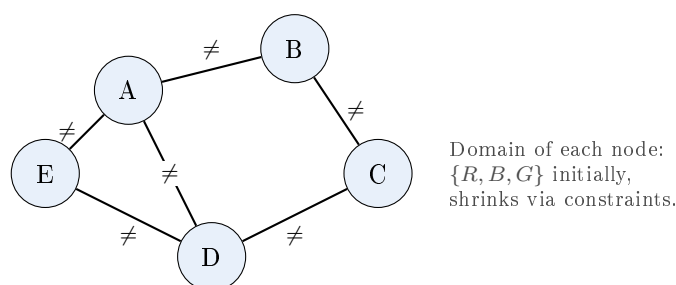
To close Week 1, Khemani shows a teaser of constraint satisfaction:

Problem. Colour each region of a map with one of $\{R, B, G\}$ such that no two adjacent regions get the same colour.

Encoding as a constraint graph.

- Each region $\rightarrow$ a node (variable).
- Each region has a *domain* of allowed colours.

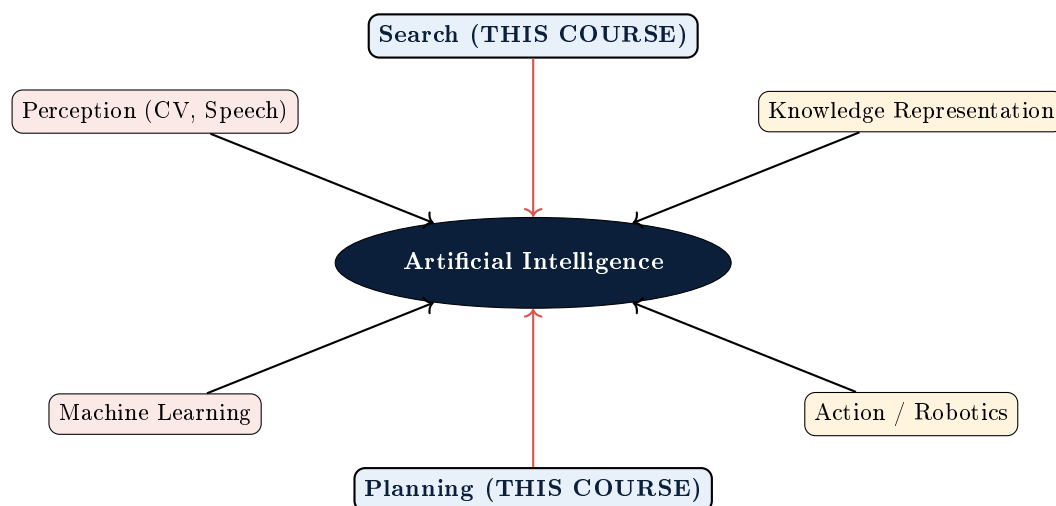
- Each pair of adjacent regions $\rightarrow$ an edge with constraint $\neq$ (must differ).



We will see in Week 11 how a tiny dose of *arc consistency* (a reasoning step) combined with *backtracking search* can crack such problems quickly. For now, just observe: *reasoning + search* is the combination that wins.

1.17 Where Does Search Fit in the AI Landscape?

To close the week, here is one big map:



Search is the engine of classical AI. Whenever you have to choose from a large number of options, or compose a sequence of decisions, you are searching. Theorem proving, planning, game playing, constraint satisfaction, even logical deduction — *all* are search problems in disguise. That is why we will spend the next eleven weeks on it.

1.18 Quick Revision — Week 1

Quick Revision

One-page summary

- **Intelligent agent:** persistent, autonomous, proactive, goal-directed. Carries a *model* of the world.
- **Three pillars of intelligence:** (i) remember the past (ML, CBR), (ii) understand the present (KR, logic), (iii) imagine the future (**search, planning**).
- **Information-processing view:** 3 layers — signal processing (outer), neuro-fuzzy (middle), symbolic reasoning (inner). This course = inner layer.
- **Physical Symbol System Hypothesis (Newell & Simon):** a physical symbol system

is *necessary and sufficient* for general intelligent action. The credo of GOFAI.

- **Turing Test (1950)**: behavioural test of intelligence; the machine must fool a human judge in conversation. Criticised because chatbots can fake it.
- **Winograd Schema Challenge (Levesque, 2011)**: forced-choice pronoun-resolution problems requiring common-sense world knowledge; *Google-proof*.
- **Simplifying assumptions (SCOAR-D)**: Static, Completely known, One agent, Actions never fail, Representation given, Discrete.
- **Memory-based vs. Model-based reasoning**: recall what worked vs. reason from first principles. This course is the model-based branch.
- **Famous names you must know**: Hobbes, Descartes, Pascal, Leibniz, Babbage, Lovelace, Turing, McCarthy, Minsky, Newell, Simon, Shannon, Winograd, Levesque, Hinton/LeCun/Bengio.

1.19 Most Important Quiz Points — Week 1

Exam Tip

Highest-yield facts

1. The four agent characteristics: **P**ersistent, **A**utonomous, **P**roactive, **G**oal-directed.
2. The three pillars of intelligence.
3. Definition of a **symbol**: “a perceptible something that stands for something else.”
4. Statement of the **Physical Symbol System Hypothesis**.
5. Difference between **Turing Test** (behavioural, conversational) and **Winograd Schema Challenge** (forced-choice, knowledge-based, Google-proof).
6. Why the WSC is Google-proof (only one grammatically/statistically similar word changes).
7. “GOFAI” stands for **Good Old-Fashioned Artificial Intelligence** (Haugeland).
8. Newell & Simon’s **Logic Theorist** (1956) was the first program designed to mimic human problem-solving.
9. The name “Artificial Intelligence” was coined at the **Dartmouth Conference (1956)** by **John McCarthy**.
10. Performance $\neq$ Competence (Brooks): labelling an image is not understanding it.

Quiz Trap

Traps to avoid:

- Confusing AI with machine learning. *ML is one route into AI; symbolic AI (search, logic) is another.*
- Saying “Turing Test = Winograd Schema”. They are different, with the WSC explicitly designed to fix Turing Test weaknesses.
- Saying “a model is the world itself”. A model is always an *abstraction*.

1.20 Practice Questions — Week 1

Conceptual questions.

1. State and briefly explain the four characteristics of an intelligent agent.
2. “A self-aware agent models itself inside its world model.” What does this mean, and why is it considered a hallmark of deeper intelligence?
3. Describe the three-layer information-processing view of an autonomous agent, and identify which layer is the focus of classical AI.
4. State the Physical Symbol System Hypothesis (Newell & Simon, 1976). Why is it controversial?
5. Compare the Turing Test and the Winograd Schema Challenge along three axes: (a) test format, (b) what they actually measure, (c) susceptibility to “cheap tricks”.
6. Why is the Winograd Schema Challenge described as *Google-proof*? Illustrate with one example.
7. List the six simplifying assumptions made by classical (un-relaxed) search algorithms.
8. Contrast *performance* and *competence* (Rodney Brooks). Give an example of each.
9. Explain Khemani’s Rubik’s-cube illustration of the distinction between search-based and knowledge-based problem solving.
10. “Both reasoning and search are subsumed by constraint satisfaction.” Explain in one paragraph.

MCQ-style.

11. Which of the following is NOT a characteristic of an intelligent agent?
(a) Persistent (b) Autonomous (c) Deterministic (d) Goal-directed.
Answer: (c). Determinism is an environmental assumption, not an agent property.
12. The Logic Theorist (1956) is associated with:
(a) Turing and Church (b) Newell, Simon and Shaw (c) Minsky and Papert (d) Hinton and LeCun.
Answer: (b).
13. The Physical Symbol System Hypothesis claims that a physical symbol system is:
(a) sufficient but not necessary (b) necessary but not sufficient (c) necessary and sufficient (d) neither necessary nor sufficient *for general intelligent action.*
Answer: (c).
14. “*The trophy didn’t fit in the brown suitcase because it was too small*” — to what does *it* most naturally refer?
(a) the trophy (b) the suitcase (c) ambiguous (d) neither.
Answer: (b). If something doesn’t fit because it’s too small, the container is too small for the contents.
15. Which of the following is the most accurate description of GOFAI?
(a) Deep learning (b) Symbolic / classical AI based on the PSSH (c) Reinforcement learning (d) Genetic algorithms.
Answer: (b).

Short descriptive questions (5–8 marks each).

16. Trace the historical development of computing machines from Pascal's Pascaline to ENIAC, naming at least five intermediate milestones.
17. Discuss the philosophical roots of AI in Hobbes, Descartes, and Leibniz. Why is Hobbes sometimes called "the grandfather of AI"?
18. Why was the Dartmouth Conference (1956) a pivotal event in the history of AI? Name the four organizers and one major contribution from each.
19. Critique the Turing Test as a definition of machine intelligence. Mention at least two specific weaknesses with examples (e.g. the multiplication trick, ELIZA-style chatbots).
20. Explain the difference between *performance* and *competence* using an example from modern image-recognition systems.

Discussion / interview-style.

21. If you had to design a 21st-century test of machine intelligence, what would you change about the Turing Test? Justify.
22. Suppose your robot has a working camera and microphone but no symbolic reasoning module. Which problems can it solve, and which can it not?
23. Argue for or against: "Deep learning is sufficient for AGI."

End of Week 1.

2 Week 2: State Space Search, DFS, BFS, and Iterative Deepening

“Instead of writing a custom program for every problem, we write general search algorithms into which individual problems can be plugged in.”

— The promise of general-purpose AI

Goal of Week 2. This is where the course gets technical. We will turn every problem into a graph of *states* and *moves*, then design *general-purpose* algorithms (independent of the problem domain) that explore this graph in search of a goal. By the end of the week you will know two foundational algorithms — **Depth-First Search (DFS)** and **Breadth-First Search (BFS)** — their complexity, their weaknesses, and a clever hybrid called **Depth-First Iterative Deepening (DFID)**.

2.1 Why “General-Purpose” Search?

In Week 1 we said an agent has a model of the world and chooses actions. If we hand-wrote a custom program for each problem (one for water jugs, one for the 8-puzzle, one for chess, ...) the AI dream of *general intelligence* would die a thousand deaths.

Intuition

Think of search algorithms as *engines*. The engine doesn't care whether you put petrol, diesel, or hydrogen in — it just consumes fuel. Similarly, a general search engine doesn't care whether you plug in a Rubik's cube, an 8-puzzle, or a maze — it just consumes *state representations* and *move rules*.

2.1.1 Three Approaches to Solving a Problem

For any problem we have three broad attacks:

1. **Brute force.** Try *all* combinations and check each one. (Map colouring: try every possible colour assignment for every region.)
2. **Informed search.** Use domain insight to avoid obvious dead ends (don't bother colouring a region in a way that already conflicts with a neighbour).
3. **General search.** Re-pose the problem in a domain-independent form and feed it to a generic search algorithm.

Remember This

This course is overwhelmingly about **general search**. We will build informed search on top of it (Week 3 onwards), but the chassis is generic.

2.1.2 Two Frameworks: State Space Search vs. Constraint Processing

Khemani identifies two related general frameworks:

State Space Search (Solution Space Search)	Describe states, devise an operator to choose an action in each state, navigate the state space to a goal state.
Constraint Processing	Define variables (each with a domain), describe constraints between subsets of variables, search for a complete assignment satisfying all constraints.

We will spend the next ten weeks on *state space search* and return to *constraint processing* in Week 11.

2.2 The State Space: an Implicit Graph

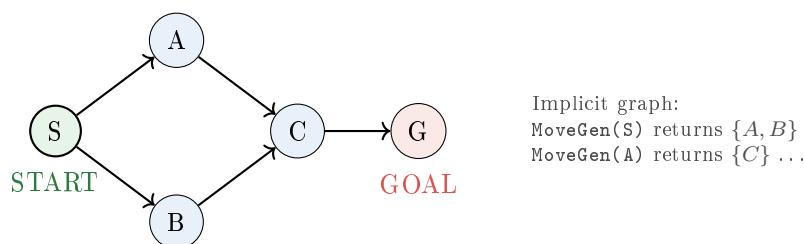
Definition

State space search as graph search.

- A **state** is a representation of a situation.
- A **MoveGen(N)** function returns the set of states (neighbours) reachable from N by one move.
- The **state space** is the (often huge, often infinite) graph implicitly defined by **MoveGen**.
- A **GoalTest(N)** function returns true iff N is a goal state.
- A **search algorithm** navigates the state space using these two functions.

Remember This

The state space is **implicit** — we never write down the whole graph in advance. We *generate* new states on demand by calling **MoveGen**. This is why search algorithms can attack problems whose full state space is astronomically large (chess has $\sim 10^{43}$ legal positions; we cannot store them all, but we can generate moves from any one of them).



2.3 Example Problems We Will Use Throughout the Course

2.3.1 The Water Jug Problem

Three jugs of capacities 8, 5, and 3 litres.

Start: the 8-litre jug is full; the other two are empty.

Goal: measure exactly 4 litres in any jug.

Allowed moves: pour from one jug to another, stopping when the source is empty or the destination is full. (We don't allow throwing water away — keeps the space finite.)

A state is naturally represented as a tuple (j_8, j_5, j_3) . Start: (8, 0, 0). A typical sequence of moves:

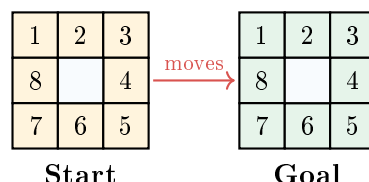
$$(8, 0, 0) \xrightarrow{8 \rightarrow 5} (3, 5, 0) \xrightarrow{5 \rightarrow 3} (3, 2, 3) \xrightarrow{3 \rightarrow 8} (6, 2, 0) \dots (4, 4, 0) \checkmark$$

Intuition

The **state representation** matters enormously. Pick something compact that captures everything you need. Here (j_8, j_5, j_3) is enough; the jugs themselves don't need names.

2.3.2 The 8-Puzzle

A 3×3 grid with eight numbered tiles and one blank. A tile slides into the adjacent blank cell. Moves are usually named by the direction the *blank* moves: U(p), D(own), L(eft), R(ight). The puzzle has $9! / 2 = 181,440$ reachable states (half of all permutations).

**2.3.3 The Man-Goat-Lion-Cabbage (MGLC) Problem**

A man must transport a lion, a goat, and a cabbage across a river using a boat that holds him plus *one* of them. Lion eats goat, goat eats cabbage, unless the man is present. How to do it?

This is a beautiful illustration of **how the choice of state representation changes the size of the state space**:

- **Full representation:** (Left bank, Right bank) both listed. Bulky and redundant.
- **Minimal:** only list the objects on the left bank — the right bank is everything else. Plus, the boat is wherever the man is, so we only track the man's position implicitly via what's on which bank.
- **Even more minimal:** list objects on the same side as the boat (with the boat alternating).

Exam Tip

“Compact state representations” is a favourite exam concept. Always ask: *What's the smallest piece of information that uniquely identifies a situation?* For MGLC: a 3-bit string (one bit per object — which side of the river) plus 1 bit for boat side = 4 bits = 16 states (only some legal).

2.3.4 The N-Queens Problem

Place N queens on an $N \times N$ chess board so that no two attack each other. This is a **configuration problem** — the goal is the *configuration*, not the path. We'll see in Week 3 that this naturally fits *solution space search*, not state space search.

2.3.5 Map Colouring

Colour each region with one of $\{R, B, G, \dots\}$ so that adjacent regions differ. Again a configuration problem (the goal is the assignment).

2.3.6 The Travelling Salesman Problem (TSP)

Given n cities and pairwise distances, find the shortest tour that visits every city exactly once and returns home. Famously **NP-hard**. Khemani calls it “the Holy Grail of computer science.” We meet it repeatedly throughout the course.

2.3.7 Maze / Path Finding

Robotics, games, GPS navigation. From a bird's-eye view a maze is obvious; from inside (what the search algorithm sees) you only know the junctions you've reached.

2.3.8 Planning vs. Configuration Problems

Two flavours of state-space problems

Planning problems	Goal state is known/described; we want the <i>path</i> (the sequence of moves). <i>Examples: river crossing, Rubik's cube, 8-puzzle, cooking, routing.</i>
Configuration problems	Goal is a state satisfying some constraints; the path doesn't matter. <i>Examples: N-queens, Sudoku, map colouring, SAT, crossword puzzles.</i>

Memory Trick

Mnemonic: Planning → Path matters. Configuration → Completed assignment is all you need.

2.4 Search as Treasure Hunt: OPEN and CLOSED

2.4.1 Generate-and-Test

Every search algorithm we'll study has the same skeleton:

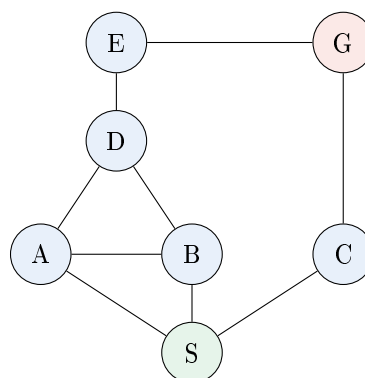
Generate-and-Test

Traverse the space by **generating** new candidates with **MoveGen**, and **testing** each one with **GoalTest**. Maintain a set **OPEN** of candidates waiting to be explored.

The **single question that distinguishes one search algorithm from another**: *Which node do we pick from OPEN next?*

2.4.2 A Tiny Search Problem (running example)

Throughout this week we use this seven-node graph:



MoveGen for this graph:

MoveGen(S) = (A, B, C)	MoveGen(D) = (A, B, E)
MoveGen(A) = (S, B, D)	MoveGen(E) = (D, G)
MoveGen(B) = (S, A, D)	MoveGen(G) = (C, E)
MoveGen(C) = (S, G)	

2.5 SimpleSearch1 — The Naïve First Try

Algorithm 1 SimpleSearch1 — naïve generate-and-test

```

1: OPEN ← {S}
2: while OPEN is not empty do
3:   pick some node N from OPEN
4:   OPEN ← OPEN \ {N}
5:   if GOALTEST(N) then
6:     return N
7:   else
8:     OPEN ← OPEN ∪ MOVEGEN(N)
9:   end if
10: end while
11: return FAILURE

```

Line-by-line.

- Line 1: start with the start node on OPEN.
- Lines 2–7: loop until OPEN is empty.
- Line 3: *the only line that determines the algorithm's identity*. “Some” is deliberately vague — different specifications give DFS, BFS, Best-First, A*, ...
- Line 5: if N is the goal, success.
- Line 7: otherwise put N 's children on OPEN.

2.5.1 The Looping Problem

Trace SimpleSearch1 (picking the leftmost node each time) on the tiny problem:

Step	OPEN
0	(S)
1	(A, B, C)
2	(S, B, D, B, C)
3	(A, B, C, B, D, B, C)
4	(S, A, D, C, B, D, B, C)
⋮ <i>infinite loop!</i>	

Because S links to A and A links back to S , we keep regenerating S . **We need memory of which nodes we've already seen.**

2.6 SimpleSearch2 — Adding CLOSED

Algorithm 2 SimpleSearch2 — with CLOSED list

```

1:  $OPEN \leftarrow \{\text{start}\}; \text{CLOSED} \leftarrow \{\}$ 
2: while  $OPEN$  is not empty do
3:   pick some node  $N$  from  $OPEN$ 
4:    $OPEN \leftarrow OPEN \setminus \{N\}; \text{CLOSED} \leftarrow \text{CLOSED} \cup \{N\}$ 
5:   if GOALTEST( $N$ ) then
6:     return  $N$ 
7:   else
8:      $OPEN \leftarrow OPEN \cup (\text{MOVEGEN}(N) \setminus \text{CLOSED})$ 
9:   end if
10: end while
11: return FAILURE

```

Two rules:

- When you pick N from OPEN, move it to CLOSED.
- When generating new children, *do not add a child that is already on CLOSED*.

Remember This

OPEN = nodes seen but not yet explored (the *frontier*).
CLOSED = nodes already explored (the *explored set*).

2.6.1 Variations and Their Hidden Costs

The line “add $\text{MOVEGEN}(N) \setminus \text{CLOSED}$ ” still allows the *same node* to be added to OPEN *multiple times* (once from each parent). Two stricter variants:

- **V1:** Add only nodes that are not already on OPEN *or* CLOSED. “Each node enters the search tree exactly once.”
- **V2:** Allow duplicates on OPEN. Simpler, but wastes memory.

Quiz Trap

V1 saves OPEN space, but *costs time*: checking whether a node is already on OPEN requires scanning OPEN (or maintaining a hash). For small graphs that’s free; for huge graphs it’s a real cost. Exam questions love asking about this trade-off.

2.7 Recovering the Path: NodePairs

SimpleSearch2 returns the goal node G — but for planning problems we want the *sequence* of moves that got us there. How?

NodePair representation

Replace each “node” in OPEN/CLOSED with a **pair** (currentNode, parentNode). When we find a goal, we trace parent links backwards to the start.

Example trace on our running example, looking for G , picking leftmost (DFS-style):

$$\begin{aligned} & \text{OPEN} = ((S, \text{nil})), \text{CLOSED} = () \\ \xrightarrow{\text{expand } S} & \text{OPEN} = ((A, S), (B, S), (C, S)), \text{CLOSED} = ((S, \text{nil})) \\ \xrightarrow{\text{expand } A} & \text{OPEN} = ((D, A), (B, S), (C, S)), \text{CLOSED} = ((A, S), (S, \text{nil})) \\ \xrightarrow{\text{expand } D} & \text{OPEN} = ((E, D), (B, S), (C, S)), \text{CLOSED} = ((D, A), (A, S), (S, \text{nil})) \\ \xrightarrow{\text{expand } E} & \text{OPEN} = ((G, E), (B, S), (C, S)), \text{CLOSED} = ((E, D), (D, A), (A, S), (S, \text{nil})) \end{aligned}$$

When we pop (G, E) and $\text{GoalTest}(G) = \text{true}$, we trace back: $G \leftarrow E \leftarrow D \leftarrow A \leftarrow S$. **Path:** $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$.

Intuition

This is the same trick you use to reconstruct a shortest path after running BFS on a maze: store a “parent pointer” for every cell.

2.8 From Non-determinism to Deterministic Algorithms

The line “pick some node from OPEN” is non-deterministic. To make a concrete algorithm we turn OPEN into a **LIST** (not a set) and *always pick the head*:

The key insight

Where you insert new children in the list determines the search strategy.

- Insert children **at the head** of OPEN $\Rightarrow$ **stack** $\Rightarrow$ **Depth-First Search**.
- Insert children **at the tail** of OPEN $\Rightarrow$ **queue** $\Rightarrow$ **Breadth-First Search**.
- Insert children **by sorted priority** $\Rightarrow$ **priority queue** $\Rightarrow$ Best-First / A* (Weeks 3, 5).

2.8.1 Ancillary Functions Used in the Algorithms

Function	Purpose
<code>removeSeen(nodeList, OPEN, CLOSED)</code>	Strip from <code>nodeList</code> any node already on OPEN or CLOSED.
<code>occursIn(N, listOfPairs)</code>	True iff N appears as the first element of some pair.
<code>makePairs(nodeList, parent)</code>	Convert $[A, B, C]$ to $[(A, P), (B, P), (C, P)]$.
<code>reconstructPath(goalPair, CLOSED)</code>	Trace parent links to recover the path from start to goal.

2.9 Depth-First Search (DFS)

Definition

DFS. A general search algorithm in which OPEN is a **stack** (LIFO): newly generated children are placed at the *front* of OPEN, so the deepest available node is always expanded next.

Algorithm 3 Depth-First Search

```

1: function DFS(start)
2:    $OPEN \leftarrow [(start, nil)]; \quad CLOSED \leftarrow []$ 
3:   while  $OPEN \neq []$  do
4:      $(N, parent) \leftarrow \text{head}(OPEN)$ 
5:      $OPEN \leftarrow \text{tail}(OPEN)$ 
6:     if GOALTEST( $N$ ) then
7:       return RECONSTRUCTPATH( $(N, parent)$ , CLOSED)
8:     end if
9:      $CLOSED \leftarrow (N, parent) :: CLOSED$ 
10:     $children \leftarrow \text{REMOVESEEN}(\text{MOVEGEN}(N), OPEN, CLOSED)$ 
11:     $newPairs \leftarrow \text{MAKEPAIRS}(children, N)$ 
12:     $OPEN \leftarrow newPairs ++ OPEN$  ▷ prepend — this makes it DFS
13:  end while
14:  return FAILURE
15: end function

```

Intuition. Dive as deep as possible along one branch. Only when that branch is exhausted do you back up and try the next sibling.

2.9.1 DFS Trace on the Tiny Graph

Suppose **MoveGen** returns children in alphabetical order (A before B before C).

Step	OPEN (head left)	CLOSED
0	(S, nil)	$\emptyset$
1	$(A, S), (B, S), (C, S)$	S
2	$(D, A), (B, S), (C, S)$	A, S (A's child B pruned, already on OPEN)
3	$(E, D), (B, S), (C, S)$	D, A, S
4	$(G, E), (B, S), (C, S)$	E, D, A, S
5	G is goal $\Rightarrow$ found path $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$ (length 4).	

2.10 Breadth-First Search (BFS)**Definition**

BFS. A general search algorithm in which **OPEN** is a **queue** (FIFO): newly generated children are placed at the *back* of **OPEN**, so all nodes at depth k are explored before any node at depth $k + 1$.

The only line of DFS that changes for BFS:

$OPEN \leftarrow OPEN ++ newPairs$ (**append** — this makes it BFS)

2.10.1 BFS Trace on the Tiny Graph

Step	OPEN (head left)	CLOSED
0	(S, nil)	$\emptyset$
1	$(A, S), (B, S), (C, S)$	S
2	$(B, S), (C, S), (D, A)$	A, S
3	$(C, S), (D, A)$	B, A, S (B's children S,A,D already seen)
4	$(D, A), (G, C)$	C, B, A, S
5	G pops as goal $\Rightarrow$ found path $S \rightarrow C \rightarrow G$ (length 2 — shortest!).	

Remember This

DFS and BFS, given the same MoveGen, can return different paths.

- DFS found $S \rightarrow A \rightarrow D \rightarrow E \rightarrow G$ (length 4).
- BFS found $S \rightarrow C \rightarrow G$ (length 2).

BFS, by exploring level-by-level, is guaranteed to return a *shortest* path (in terms of number of edges). DFS gives no such guarantee.

2.11 Three Bookkeeping Variants

Different ways to handle the CLOSED list yield subtly different search trees:

-
- Case 1** Only **new** nodes added to OPEN (skip ones already on OPEN *or* CLOSED). Both DFS and BFS explore the same set of nodes, just in different orders.
-
- Case 2** Skip only nodes already on CLOSED. DFS and BFS produce different search trees because both may visit the same node twice via different paths.
-
- Case 3** No CLOSED check at all (no pruning). DFS may go into **infinite loops** on graphs with cycles. BFS still terminates (and still finds the shortest path) because it sweeps level-by-level.
-

Exam Tip

The **infinite-loop** risk of DFS is a favourite exam point. Without CLOSED-checking, DFS on a cyclic graph will go round and round forever — even if a goal is right next door.

2.12 Analysis: Time and Space Complexity

Let:

- b = branching factor (average number of children per node),
- d = depth of the shallowest goal,
- m = maximum depth of the search tree.

2.12.1 Best Case, Worst Case, Average Case

Assume a complete tree (no graph cycles to worry about), with a single goal at depth d .

- **BFS** expands every node at depths $0, 1, \dots, d$ before reaching the goal:

$$N_{\text{BFS}}^{\min} = 1 + b + b^2 + \dots + b^{d-1} + 1 = \frac{b^d - 1}{b - 1} + 1$$

(when the goal is the *first* node at depth d examined). In the worst case it must scan the whole level d :

$$N_{\text{BFS}}^{\max} = \frac{b^{d+1} - 1}{b - 1}.$$

- **DFS** (best case) finds the goal by diving straight down:

$$N_{\text{DFS}}^{\min} = d + 1.$$

DFS worst case is identical to BFS worst case if the goal is the last leaf explored:

$$N_{\text{DFS}}^{\max} = \frac{b^{d+1} - 1}{b - 1}.$$

Average number of nodes inspected.

$$N_{\text{DFS}}^{\text{avg}} = \frac{1}{2} \left((d + 1) + \frac{b^{d+1} - 1}{b - 1} \right) \approx \frac{b^d}{2}. \quad (1)$$

$$N_{\text{BFS}}^{\text{avg}} = \frac{1}{2} \left(\frac{b^d - 1}{b - 1} + 1 + \frac{b^{d+1} - 1}{b - 1} \right) \approx \frac{b^d(b+1)}{2(b-1)}. \quad (2)$$

Dividing:

$$N_{\text{BFS}}^{\text{avg}} \approx \frac{b+1}{b-1} N_{\text{DFS}}^{\text{avg}}$$

For large b , the ratio approaches 1. Both are $O(b^d)$.

Complexity Insight

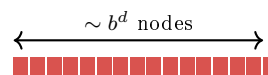
The take-home result: both DFS and BFS have **exponential average-case time complexity** $O(b^d)$. There is no way to escape the exponential branching of the tree using blind search alone.

2.12.2 Space Complexity (the Real Differentiator)

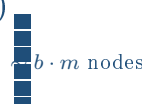
Time is exponential for both. *Space* is where they part dramatically.

- **BFS** must hold an entire frontier of size $\sim b^d$ in OPEN. **Exponential space.**
- **DFS** on a tree of depth m only needs to hold the current path plus siblings at each level: $O(b \cdot m)$. **Linear space.**

BFS frontier (level d)



DFS frontier (current path)



2.12.3 Completeness and Optimality

Definitions

Complete: guaranteed to find a solution if one exists.

Optimal: guaranteed to find a *shortest* (or lowest-cost) solution.

- **BFS** is *complete* on finite graphs (and on infinite graphs with finite branching factor, given a solution exists). It is *optimal* when all edges have unit cost.
- **DFS** is *not complete* on infinite graphs (it may dive forever down an infinite branch and miss the goal). On finite graphs with CLOSED-checking, it *is* complete. It is **not optimal** — it returns whatever path it finds first.

2.12.4 Summary Table: DFS vs. BFS

Criterion	DFS	BFS
Data structure for OPEN	Stack (LIFO)	Queue (FIFO)
Time complexity	$O(b^m)$ (worst), $O(d+1)$ (best)	$O(b^d)$
Space complexity	$O(b \cdot m)$ (linear)	$O(b^d)$ (exponential)
Completeness	Not on infinite spaces; yes on finite with CLOSED	Yes (finite branching, sol. exists)
Optimality	No	Yes (uniform-cost edges)
Behaviour goal at left	Fast (lucky dive)	Slow (sweeps levels)
Behaviour goal at right	Slow (wades through left side)	Same as left (level-by-level)

Memory Trick

Memory trick: DFS = Drill Down Fast, Stingy (on memory). BFS = Broad, First-time finds Shortest.

Quiz Trap

A frequent trap: “Is DFS faster than BFS?” Answer: *neither is faster in general*. DFS can be *much* faster if the goal happens to be deep on the first branch explored, but *much* slower if the goal is at shallow depth in another branch. The average time complexity is exponential for both.

2.13 The Best of Both Worlds: Depth-Bounded DFS and DFID

We want:

- DFS’s linear space, and
- BFS’s optimal (shortest-path) behaviour and completeness.

The answer is a beautiful idea: **Iterative Deepening**.

2.13.1 Step 1 — Depth-Bounded DFS (DBDFS)

Run DFS, but **stop** at depth d . Any node at depth d is treated as a leaf (don’t expand).

Algorithm 4 DBDFS — Depth-Bounded DFS

```

1: function DBDFS(start, depthBound)
2:    $OPEN \leftarrow [(start, nil, 0)]$  ▷ triple: (node, parent, depth)
3:   while  $OPEN \neq []$  do
4:      $(N, parent, dep) \leftarrow \text{head}(OPEN)$ 
5:      $OPEN \leftarrow \text{tail}(OPEN)$ 
6:     if GOALTEST( $N$ ) then return path
7:     end if
8:     if  $dep < \text{depthBound}$  then
9:        $children \leftarrow \text{MOVEGEN}(N)$ 
10:       $OPEN \leftarrow \text{MAKETRIPLES}(children, N, dep+1) ++ OPEN$ 
11:    end if
12:  end while
13:  return FAILURE
14: end function

```

Properties of DBDFS:

- Linear space ✓
- Not complete (might miss goals deeper than d)
- Does not guarantee shortest path

2.13.2 Step 2 — Depth-First Iterative Deepening (DFID)**DFID's beautiful idea**

Repeatedly run DBDFS with $d = 1, 2, 3, \dots$ until the goal is found. The first time the goal is found, it is at the smallest possible depth — exactly what BFS would find — but each individual DBDFS uses only linear space.

Algorithm 5 DFID — Depth-First Iterative Deepening

```

1: function DFID(start)
2:   depthBound  $\leftarrow 1$ 
3:   while true do
4:      $result \leftarrow \text{DBDFS}(start, \text{depthBound})$ 
5:     if  $result \neq \text{FAILURE}$  then return  $result$ 
6:     end if
7:     depthBound  $\leftarrow \text{depthBound} + 1$ 
8:   end while
9: end function

```

2.13.3 Isn't This Wasteful?

Yes... and amazingly, no. DFID re-explores the entire tree at every iteration. Surely that's quadratic blow-up?

Analysis. In a full tree with branching factor b and depth d :

- Number of internal nodes: $I = 1 + b + b^2 + \dots + b^{d-1} = (b^d - 1)/(b - 1)$.
- Number of leaves: $L = b^d$.
- Relationship: $L = (b - 1) \cdot I + 1$, so $L \gg I$ when b is large.

DFID inspects $L + I$ nodes (leaves once, internal nodes multiple times, but their total count is dominated by L). BFS inspects only L nodes at depth d . The ratio:

$$\frac{N_{\text{DFID}}}{N_{\text{BFS}}} \approx \frac{L + I}{L} = 1 + \frac{1}{b - 1} = \frac{b}{b - 1}.$$

For $b = 10$, the ratio is ~ 1.11 — DFID does only 11% more work than BFS. **The extra cost is not significant!**

Remember This

DFID: linear space, optimal solution length, only $\sim 11\%$ more time than BFS (for $b \approx 10$). This is the best of both worlds.

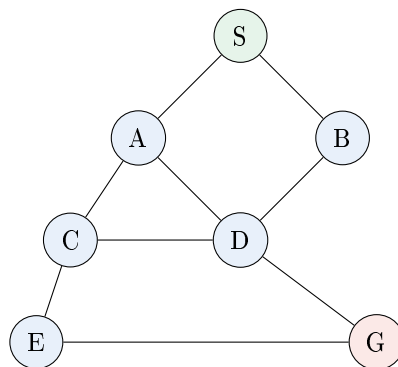
2.13.4 Why DFID Was Invented by Chess Programmers

Computer chess programs face a problem: “you have 3 minutes to make your move.” If you commit to a fixed search depth and run out of time, you have no move. DFID gives you **anytime move generation**: you always have the best move found at the deepest iteration completed so far, no matter when you are stopped.

2.13.5 A Subtle Bug — Be Careful with CLOSED in DFID

Khemani highlights a real subtlety, pointed out by a student (Siddharth Sagar) at IIT Dharwad in 2019.

Consider this state space:



The genuine shortest path is $S \rightarrow B \rightarrow D \rightarrow G$ (length 3). But **if DFID stores all nodes seen in any iteration onto a global CLOSED list**, then when running with depth-bound 3, after exploring $S \rightarrow A \rightarrow C$ and adding D to CLOSED via C , the next attempt from B to reach D finds D already on CLOSED and skips it — missing the optimal path.

Quiz Trap

The fix: reset CLOSED at the start of *every iteration*, or simply don't use CLOSED inside DBDFS (acceptable because depth-bound limits recursion). The cost of dropping CLOSED is a moderate increase in nodes visited (cycles within depth bound), but it's the price for guaranteed optimality.

Trade-off: Without CLOSED, DFID finds the shortest path. *But how do we then reconstruct it?* We can still store parent pointers per node *in the current path stack*, or — for configuration problems — give up on path reconstruction entirely (the goal state itself is the answer).

2.14 The Monster — Combinatorial Explosion

The villain of AI search

The number of nodes in the search tree grows **exponentially** with depth. With branching factor 3, depth 20 already gives $3^{20} \approx 3.5$ billion nodes. *This is what every search algorithm in this course is fighting.*

b	d	nodes
2	10	1,024
2	20	1,048,576
3	10	59,049
3	20	3.5×10^9
10	10	10^{10}
10	20	10^{20}

Remember This

All three algorithms we have studied — DFS, BFS, DFID — are **blind** or **uninformed**. They explore the search space without any knowledge of which nodes are “closer” to the goal. In Week 3 we add *heuristic* guidance, which is our first real weapon against combinatorial explosion.

2.15 Real-World Applications

Real-World Application

DFS is used in: maze solving, topological sorting, cycle detection in graphs, Tarjan’s strongly-connected components, finding bridges/articulation points, generating all permutations / combinations, backtracking-based puzzle solvers (Sudoku, N-queens).

BFS is used in: shortest-path on unweighted graphs (web crawlers, social-network “six degrees” queries), broadcasting in networks, GPS map applications (when all road segments have equal cost), peer-to-peer search (Gnutella), garbage-collection mark-sweep.

DFID is used in: computer game playing (chess, Go) where anytime move generation is essential; IDA* (iterative-deepening A*, Week 6) for memory-bounded heuristic search.

2.16 Common Mistakes Students Make

Common Mistake

1. “DFS always uses less memory than BFS.” *Only true on trees / when using linear-stack implementation. With CLOSED, both can grow to the size of the visited set.*
2. “BFS is optimal for any graph.” *Only when edge weights are uniform.* With variable weights, BFS may miss the cheapest path; use Uniform-Cost Search / Dijkstra instead.
3. “Iterative deepening is wasteful.” *It’s not, because the bottom level dominates the cost.*
4. Mixing up which side new children go on (head/tail) — flipping it turns DFS into BFS and vice versa.
5. Forgetting to check CLOSED — leads to infinite loops on cyclic graphs (DFS) or just wasted work (BFS).

6. Forgetting to reset CLOSED across iterations of DFID — corrupts optimality.

2.17 Quick Revision — Week 2

Quick Revision

- **State space** = implicit graph defined by MoveGen.
- **Search** = **generate-and-test**, with the key question: *which node from OPEN do we expand next?*
- **OPEN** = frontier (seen but unexplored). **CLOSED** = explored.
- **NodePair** = (node, parent) — enables path reconstruction.
- **DFS** = OPEN is a **stack** (prepend new children). Linear space. Not optimal. Not complete on infinite graphs.
- **BFS** = OPEN is a **queue** (append new children). Exponential space. Optimal for unit-cost edges. Complete.
- **Time complexity (avg.)**: both $O(b^d)$. Ratio $N_{\text{BFS}}/N_{\text{DFS}} \approx (b+1)/(b-1)$.
- **DBDFS** = DFS bounded at depth d . Linear space, not complete.
- **DFID** = DBDFS with $d = 1, 2, 3, \dots$. Linear space, optimal, $\sim(b/(b-1))$ times BFS's work.
- **Planning problems**: path matters. **Configuration problems**: only the final state matters.
- Three blind algorithms: DFS, BFS, DFID. All **uninformed** — they ignore any knowledge of where the goal is.

2.18 Most Important Quiz Points — Week 2

Exam Tip

Highest-yield facts:

1. Time complexities: $O(b^d)$ for both DFS (worst) and BFS.
2. Space: DFS $O(bm)$ **linear**; BFS $O(b^d)$ **exponential**.
3. **BFS is optimal** when edges have equal cost.
4. **DFS is not complete** on infinite spaces.
5. DFID combines DFS's linear space with BFS's shortest-path guarantee at $\sim b/(b-1)$ overhead.
6. Choosing between configuration and planning is a matter of *does the path matter?*
7. Storing (N, parent) NodePairs lets you reconstruct the path at termination.
8. The only difference between DFS and BFS in code is whether new children are placed at the **head** or **tail** of OPEN.
9. Without CLOSED, DFS may loop on cyclic graphs; BFS still terminates but wastes work.

10. Three blind algorithms (DFS/BFS/DFID) ignore all problem-specific knowledge — the motivation for heuristic search in Week 3.

2.19 Practice Questions — Week 2

Conceptual.

1. Define *state*, *MoveGen*, *GoalTest*. Why is the state space described as “implicit”?
2. Give state representations for: (i) the water-jug problem, (ii) the 8-puzzle, (iii) MGLC. For each, estimate the size of the state space.
3. Why does the choice between insert-at-head vs. insert-at-tail single-handedly distinguish DFS from BFS?
4. Why is OPEN said to be the “frontier” and CLOSED the “explored set”?
5. Compare planning and configuration problems with one example each.
6. Why doesn’t BFS guarantee shortest-cost (not shortest-length) paths in weighted graphs?
7. Explain why DFID has linear space complexity even though it “re-explores” the tree at each iteration.
8. State and justify the formula $N_{\text{DFID}}/N_{\text{BFS}} \approx b/(b-1)$.
9. Why was DFID invented by chess programmers? What property does it have that fixed-depth DFS lacks?
10. Describe the bug that can arise when DFID maintains a global CLOSED list across iterations.

Numerical.

11. For $b = 4$ and $d = 10$, compute the (worst-case) number of nodes inspected by BFS, DFS, and DFID. Comment.
12. For the tiny graph used in the chapter (vertices S, A, B, C, D, E, G), trace DFS (left-to-right) and BFS, listing OPEN and CLOSED at each step. Recover the path each algorithm returns.
13. Suppose a goal lies at depth $d = 15$ in a tree with $b = 2$. Compare DFS (worst), DFS (best), BFS, and DFID by the number of nodes inspected.
14. Show that for $b = 2$, DFID inspects roughly twice as many nodes as BFS. Verify with $d = 5$.

Algorithmic / proof.

15. Write SimpleSearch1 in pseudocode. Trace it on the tiny example with a deliberate ordering that causes an infinite loop.
16. Modify SimpleSearch2 so that it returns the path to the goal (not just the goal node).
17. Prove that BFS finds the shortest (fewest-edges) path on any graph with unit-cost edges.
18. Prove that DFID finds the shortest-length path provided we do *not* maintain a global CLOSED across iterations.

MCQ-style.

19. Which has **linear** space complexity in the depth of the tree?
 (a) BFS (b) DFS (c) Both (d) Neither.
Answer: (b).

20. For BFS with branching factor $b = 10$ and goal at depth $d = 6$, the order of magnitude of nodes expanded is approximately:
(a) 10^3 (b) 10^6 (c) 10^9 (d) 10^{12} .
Answer: (b) — roughly $b^d = 10^6$.
21. Which combines DFS's space and BFS's optimality?
(a) DBDFS (b) DFID (c) Best-First (d) A*.
Answer: (b).
22. Which of these is a configuration problem (not a planning problem)?
(a) 8-puzzle (b) River crossing (c) N-queens (d) Rubik's cube.
Answer: (c).
23. In SimpleSearch1 (no CLOSED), on a graph with cycles, the algorithm:
(a) is optimal (b) is complete (c) may loop forever (d) is faster than BFS.
Answer: (c).

End of Week 2.

3 Week 3: Heuristic Search, Local Search, and Escaping Local Optima

“Both DFS and BFS are oblivious of the goal. Irrespective of where the goal is, both algorithms set out on the same predetermined trajectories every time. What is needed is a search algorithm with a sense of direction.”

— Deepak Khemani

Goal of Week 3. In Week 2 we built blind algorithms. They work, but they are wasteful: they explore the state space without any clue where the goal lives. This week we give the search algorithm a *heuristic function* $h(n)$ that estimates how close each state is to the goal, then bend our algorithms to follow this gradient. We will meet Best-First Search, Hill Climbing, Solution-Space Search, and a family of clever methods to escape local optima — Beam Search, Variable-Neighbourhood Descent, Tabu Search, Iterated Hill Climbing, and Simulated Annealing.

3.1 From Blind to Informed: the Heuristic Function

3.1.1 The Gravity Analogy

Imagine a stone rolling down a 2-D landscape (assume mass but no momentum). At each instant it “tests” its immediate surroundings and rolls in the direction of steepest descent. It is a **local** agent — it only looks at neighbours — yet the global pull of gravity guides it towards a low point.

Intuition

A heuristic function $h(n)$ *simulates gravity*. It tells the search algorithm which neighbours are “lower” (closer to the goal). The algorithm follows the gradient: pick the neighbour with the smallest h .

3.1.2 The Heuristic Function $h(n)$

Definition

A **heuristic function** $h : \text{States} \rightarrow \mathbb{R}_{\geq 0}$ takes a state n as input and returns a non-negative number that estimates the “distance” from n to a goal.

By convention, $h(\text{goal}) = 0$. We *minimize* h .

Together with `MoveGen` and `GoalTest`, the user now supplies $h(n)$. The search becomes **informed** and **goal-directed** — the same algorithm will follow a different trajectory if you move the goal, unlike DFS/BFS.

3.1.3 Examples of Heuristic Functions

The 8-puzzle. Two famous heuristics:

- $h_1(n) = \text{Hamming distance} = \text{number of tiles that are not in their goal position.}$
- $h_2(n) = \text{Manhattan distance} = \sum_{\text{tiles}} (|\text{row diff}| + |\text{col diff}|) \text{ to each tile's goal location.}$

1	2	3
4	5	6
7	8	

1	2	7
4	5	6
	8	3

Goal: $h_H = 0$, $h_M = 0$ Start: $h_H = 2$, $h_M = 6$

In the right state, tiles 3 and 7 are misplaced: $h_H = 2$. The Manhattan distance: tile 3 is two cells away from its target, tile 7 is four cells away $\Rightarrow h_M = 2 + 4 = 6$.

Geographic route finding. Standard heuristics:

- **Euclidean (straight-line) distance:** $h_E(n) = \sqrt{(x_G - x_n)^2 + (y_G - y_n)^2}$.
- **Manhattan / city-block distance:** $h_M(n) = |x_G - x_n| + |y_G - y_n|$.

Euclidean is appropriate when straight-line movement is allowed (a bird flying); Manhattan when you can only move on a grid (a taxi on city streets).

Blocks world. (We'll meet this domain more deeply in Week 8.) Given a configuration of stacked blocks, possible heuristics:

- h_1 : +1 if a block sits on its correct supporter (block or table); −1 otherwise.
- h_2 : + n if a block is on a *correct* stack of n blocks; − n otherwise. This is *more discriminative* — it penalises a block sitting on the right thing but inside a wrong larger structure.

Remember This

Not all heuristics are equal. Two heuristics can rank neighbours *differently*. A bad heuristic may push search into a dead end. We'll see in Week 5 how to compare heuristic “quality” rigorously (admissibility, consistency, dominance).

3.2 Best-First Search

Definition

Best-First Search. A general search algorithm in which OPEN is a **priority queue** sorted by $h(n)$: the node with the smallest heuristic value is always expanded next.

Algorithm 6 Best-First Search

```

1: OPEN ← [(start, nil, h(start))]; CLOSED ← []
2: while OPEN ≠ [] do
3:   (N, parent, hN) ← head(OPEN)
4:   OPEN ← tail(OPEN)
5:   if GOALTEST(N) then return RECONSTRUCTPATH(...)
6:   end if
7:   CLOSED ← (N, parent, hN) :: CLOSED
8:   children ← REMOVESEEN(MOVEGEN(N), OPEN, CLOSED)
9:   newTriples ← MAKETRIPLES(children, N, h)           ▷ compute h for each child
10:  OPEN ← SORTBYH(newTriples ++ OPEN)                 ▷ key line
11: end while
12: return FAILURE

```

Key idea. Lines 2–7 are identical to DFS/BFS; only line 9 changes — instead of prepending (DFS) or appending (BFS), we sort by h . In practice OPEN is implemented as a **min-heap priority queue** for efficiency: insert and extract-min are both $O(\log |OPEN|)$.

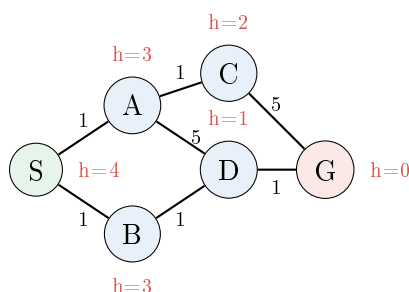
3.2.1 What Best-First Gains Over BFS/DFS

- **Sense of direction.** The search frontier shrinks toward the goal rather than ballooning outward equally in all directions.
- **Often dramatically fewer nodes explored.** For geographic routing, a decent h (Euclidean distance) lets the algorithm head directly toward the goal, occasionally backtracking around obstacles.

3.2.2 What Best-First Does *Not* Guarantee

Quiz Trap

Best-First Search is **NOT optimal**: it may return a longer path than necessary. The heuristic only estimates distance — it doesn't account for the cost already spent (g). For optimal heuristic search we need A^* , which combines g and h — Week 5.



A toy example: from S , both A and B look equally good ($h = 3$). Best-First arbitrarily picks one. If it picks A then C then G , it returns path $S \rightarrow A \rightarrow C \rightarrow G$ of cost $1+1+5 = 7$. The optimal path is $S \rightarrow B \rightarrow D \rightarrow G$ of cost $1+1+1 = 3$. **Best-First found a path, but not the cheapest one.**

3.3 Hill Climbing — Local Search with Constant Space

Best-First Search still keeps a global OPEN list of all unexplored nodes — exponential in size. What if we throw it away entirely and only keep the current node?

Definition

Hill Climbing (HC). A **local search** algorithm that, from the current node N , examines all neighbours, and moves to the best neighbour *only if it improves on N* . Terminates when no neighbour is better.

Note: the standard formulation is a *maximization* problem — we treat h as a quality measure to maximize (sometimes called the *eval* or fitness function). For minimization, flip the inequality. We'll often switch between minimization (distance-to-goal) and maximization (clauses-satisfied) conventions; the algorithm is the same.

Algorithm 7 Hill Climbing (maximization)

```

1: function HILLCLIMBING(start)
2:    $N \leftarrow \text{start}$ 
3:    $\text{newNode} \leftarrow \text{best}(\text{MOVEGEN}(N))$ 
4:   while  $\text{newNode}$  is better than  $N$  do
5:      $N \leftarrow \text{newNode}$ 
6:      $\text{newNode} \leftarrow \text{best}(\text{MOVEGEN}(N))$ 
7:   end while
8:   return  $N$ 
9: end function

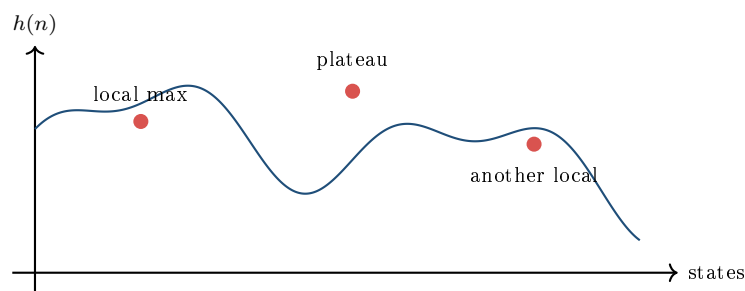
```

3.3.1 Properties**Hill Climbing at a glance**

- **Space:** *constant*. We only remember the current node. A vast improvement over DFS/BFS!
- **Time:** linear in the path length to a local optimum.
- **Termination:** when no neighbour is better.
- **Completeness:** *not complete*. May terminate before reaching the global optimum.
- **Optimality:** *not optimal*. Returns whatever local maximum it climbs.
- **New framing:** treats search as *optimization*, not just goal-finding.

Memory Trick

Mnemonic: “HC = High Climb, Constant memory, Can’t see beyond local hill.”

3.3.2 The Three Curses of Hill Climbing

- **Local maxima.** A hill that’s not the highest. HC climbs it, then is stuck.
- **Plateaus.** Flat regions where every neighbour has the same value. HC has no preference and terminates (or wanders).
- **Ridges.** A long thin maximum that runs diagonally; no axis-aligned neighbour is better. HC cannot move along the ridge.

3.3.3 Worked Example — Blocks World

Recall the two heuristics:

- h_1 : +1 for correctly placed blocks, -1 otherwise (*simple, non-discriminative*).
- h_2 : + n if block is in correct stack of size n , - n if wrong (*discriminative*).

Khemani demonstrates that with h_1 , Hill Climbing gets stuck in a local maximum: the algorithm sees state Q as the best successor of the start (it incorrectly favours placing block A on block E), but from Q all successors are *worse*, so HC terminates there. With h_2 , the heuristic is discriminative enough that HC follows the correct gradient straight to the goal.

Remember This

The lesson: a richer heuristic that “sees further” can be the difference between getting stuck and finding the solution. *Designing a good heuristic is half of solving the problem.*

3.3.4 Effect of Neighbourhood Density

You can change the `MoveGen` function to give yourself more or fewer neighbours. For SAT with n variables represented as bit strings:

- N_1 = flip exactly 1 bit (sparse — n neighbours).
- N_2 = flip exactly 2 bits ($\binom{n}{2}$ neighbours).
- N_{1-n} = flip any subset of bits (2^n neighbours — *brute force!*).

Density trade-off

Denser neighbourhood $\Rightarrow$ fewer local optima (good!), but **more neighbours to inspect at every step** (expensive!).
The right balance depends on the problem.

3.4 The Travelling Salesman Problem (TSP)

The **TSP** is the workhorse benchmark of search algorithms. We will keep returning to it.

Definition

Travelling Salesman Problem. Given n cities and pairwise distances d_{ij} , find a *Hamiltonian cycle* (a tour visiting every city exactly once and returning to start) of **minimum total cost**.

Key facts:

- Every city has exactly 2 edges incident to it in the tour.
- The tour has exactly n edges.
- Number of distinct tours: $(n-1)!/2$ (for symmetric TSP).
- **NP-hard.** No polynomial algorithm known.

3.4.1 Greedy Constructive Heuristics for TSP

These build a tour piece by piece — **synthesis methods**, not local search:

1. Nearest-Neighbour Heuristic.

- Start at some city.
- Repeatedly move to the nearest *unvisited* neighbour.
- Don't close the loop prematurely (only return to start when all cities visited).

Fast, but can be far from optimal (it commits to short edges early, may be forced into very long edges at the end).

2. Greedy-Edges Heuristic.

- Sort all edges by length.
- Add the shortest available edge to the partial tour, provided (a) it doesn't create a degree-3 vertex, and (b) it doesn't close the loop prematurely.

3. Savings Heuristic (Clarke–Wright).

- Pick a base vertex v_0 . Start with $n - 1$ tours of length 2: $v_0 \rightarrow v_i \rightarrow v_0$.
- Repeatedly *merge* two such tours by removing two edges (one from each tour) connected to v_0 and replacing them with a single edge connecting the two hanging vertices.
- Choose the merge that gives *maximal savings* in total cost.
- After $n - 2$ merges you have one tour.

Real-World Application

The savings heuristic is the basis of many real-world vehicle-routing algorithms (postal delivery, garbage collection, school bus routing).

3.4.2 Local Search Operators for TSP

If we already have a tour, we can *perturb* it to try to improve.

- **2-city exchange.** Swap the positions of two cities in the tour.
- **2-edge exchange (2-opt).** Remove two edges from the tour; reconnect the resulting pieces in the only other way that forms a valid tour. ($\binom{n}{2}$ choices.)
- **3-edge exchange (3-opt).** Remove three edges and reconnect (4 ways per choice). ($\binom{n}{3}$ choices, 4 reconnections.)

Intuition

2-opt is the workhorse of TSP local search. Most random tours can be 2-opt'd to within 5–10% of the optimum within seconds, even for $n = 1000$ cities.

3.5 Solution Space (Configuration Space) Search

So far we've been searching in the **state space** — each move is a small change, and the goal is found somewhere in the graph. There is a complementary view:

Definition

Solution Space Search (SSS). Each node in the search space *is itself a candidate solution*. A node is a goal iff it satisfies the goal description. **MoveGen** is now called a *neighbourhood function* — it perturbs a candidate to produce nearby candidates.

- **Configuration problems** (N-queens, SAT, Sudoku, map colouring) fit naturally — every node is an assignment that may or may not satisfy the constraints.
- **Planning problems** can also be solved this way — we are then searching in the *plan space* where each node is a complete (perhaps invalid) plan.

3.5.1 Synthesis vs. Perturbation

Two ways to navigate the solution space:

- **Synthesis (constructive).** Build a candidate piece by piece, starting from an empty assignment. *Examples:* nearest-neighbour TSP, greedy-edges TSP, backtracking N-queens.
- **Perturbation.** Start with a full candidate (maybe random, maybe constructive); modify it. *Examples:* 2-opt for TSP, bit-flipping for SAT, queen-swap for N-queens.

3.5.2 Worked Example — 6-queens via Perturbation

Represent a 6-queens candidate as a permutation $[a, c, e, b, f, d]$ — the i -th element gives the column of the queen in row i . Any permutation is automatically free of row and column conflicts; only diagonal conflicts remain. Perturbation: swap two entries.

3.5.3 Worked Example — SAT

The **Boolean Satisfiability Problem (SAT)**: given a Boolean formula, find an assignment that satisfies it. In Conjunctive Normal Form (CNF):

$$F = (b \vee \neg c) \wedge (c \vee \neg d) \wedge (\neg b) \wedge (\neg a \vee \neg e) \wedge (e \vee \neg c) \wedge (\neg c \vee \neg d)$$

With 5 variables, $2^5 = 32$ candidates. Each candidate is a 5-bit string. Heuristic: $h(n) = \text{number of clauses satisfied}$. Goal: $h(n) = 6$ (all clauses).

Complexity facts.

- **SAT** is NP-complete: $O(2^n)$ in the worst case, but verification is polynomial.
- **TSP** is NP-hard: $O(n!)$ for brute force, no easy verification.
- Despite both being “hard,” TSP’s search space is dramatically larger:

n	2^n (SAT candidates)	$n!$ (TSP candidates)
8	256	40,320
50	$\sim 10^{15}$	$\sim 10^{64}$
100	$\sim 10^{30}$	$\sim 10^{158}$

Inspecting a million SAT candidates per second for $n = 100$: 10^{24} seconds = 10^{16} years. *We cannot solve these by brute force.* Local search is our only hope for large instances.

3.6 Exploration vs. Exploitation

This is one of the deepest concepts in all of AI search:

The trade-off that defines local search

Exploitation pulls the search *along the gradient* (greedy, local, may get stuck).

Exploration leads the search *away from the trodden path* (diverse, may waste time, but can escape local optima).

Hill Climbing is pure exploitation; Random Walk is pure exploration. Every clever algorithm we’ll see is a way to *balance* the two.

3.7 Escaping Local Optima — Deterministic Methods

3.7.1 Beam Search

Idea. Instead of keeping just one “current node” like HC, keep the b best nodes at each level. (Computer memory is cheap.)

Algorithm 8 Beam Search (with beam width b)

```

1:  $beam \leftarrow \{\text{start}\}$ 
2: while no node in  $beam$  is goal do
3:    $candidates \leftarrow \bigcup_{N \in beam} \text{MOVEGEN}(N)$ 
4:    $beam \leftarrow$  best  $b$  candidates by  $h$ 
5: end while
6: return goal
  
```

Properties.

- Space: $O(b)$ — constant (in terms of depth).
- More robust than HC because if one path dies, others continue.
- *Still incomplete*: all b parallel paths could simultaneously die.

Intuition

Beam Search is also called “ k -beam search” or “stochastic beam search” depending on whether you pick the best b deterministically or sample. With $b = 1$, Beam Search degenerates to Hill Climbing. With $b = \infty$, it becomes Best-First Search.

3.7.2 Variable Neighbourhood Descent (VND)

Idea. Have a sequence of **MoveGen** functions, sorted from sparse to dense. Run HC with the sparsest. When it gets stuck, switch to a denser neighbourhood and run HC again. Keep escalating.

Algorithm 9 Variable Neighbourhood Descent

```

1: function VND(start)
2:    $node \leftarrow \text{start}$ 
3:   for  $i \leftarrow 1$  to  $n$  do                                 $\triangleright n$  neighbourhood functions, sparse to dense
4:      $moveGen \leftarrow \text{MOVEGEN}_i$ 
5:      $node \leftarrow \text{HILLCLIMBING}(node, moveGen)$ 
6:   end for
7:   return  $node$ 
8: end function
  
```

Why this is clever. Most of the climbing is done with cheap, sparse neighbourhoods. Only when we are stuck do we pay the cost of inspecting a denser neighbourhood. We get out of *shallow* local optima cheaply.

3.7.3 Best-Neighbour (a.k.a. Steepest Descent with no termination)

Idea. Always move to the best neighbour, *even if it is worse than the current node*. Remember the best node seen.

Algorithm 10 Best-Neighbour

```

1: function BESTNEIGHBOUR(start)
2:    $N \leftarrow \text{start}$ ;  $\text{bestSeen} \leftarrow N$ 
3:   while some external termination criterion not met do
4:      $N \leftarrow \text{best}(\text{MOVEGEN}(N))$ 
5:     if  $N$  better than  $\text{bestSeen}$  then  $\text{bestSeen} \leftarrow N$ 
6:   end if
7: end while
8:   return  $\text{bestSeen}$ 
9: end function

```

Problem. If N^* is a local optimum, the algorithm steps off to a worse neighbour N' . But from N' the best neighbour is... N^* again. So it bounces back. **It needs a memory.**

3.7.4 Tabu Search

Idea (Glover, 1986). Add a **tabu list** — a short memory of recent moves that are temporarily forbidden. This forces the search to wander into new territory after escaping a local optimum.

Algorithm 11 Tabu Search

```

1: function TABUSEARCH(start)
2:    $N \leftarrow \text{start}$ ;  $\text{bestSeen} \leftarrow N$ ;  $\text{tabuList} \leftarrow []$ 
3:   while termination criterion not met do
4:      $N \leftarrow \text{best}(\text{ALLOWED}(\text{MOVEGEN}(N), \text{tabuList}))$ 
5:     if  $N$  better than  $\text{bestSeen}$  then  $\text{bestSeen} \leftarrow N$ 
6:   end if
7:     update  $\text{tabuList}$  ▷ add current move, drop oldest if exceeding tenure
8:   end while
9:   return  $\text{bestSeen}$ 
10: end function

```

Tabu tenure tt : how many cycles a move stays forbidden.

Worked illustration (Khemani's SAT example, $n = 7$, $tt = 2$):

Current candidate	Move made	Tabu memory M
1011010		0000000
1001010 (bit 3 flipped)	flip bit 3	00 2 0000
1001000 (bit 6 flipped)	flip bit 6	00100 2 0
0001000 (bit 1 flipped)	flip bit 1	2 000010

The memory M_i stores how many cycles a bit- i -flip is forbidden. Each cycle, M_i decrements (to zero); a freshly used move is set to tt .

3.7.5 Aspiration Criterion

Tabu has one safety valve. If a forbidden move would lead to a state *better than anything seen so far*, allow it anyway. After all, the point of being tabu is to avoid going in circles — but a record-breaking move can't be “a circle.”

3.7.6 Frequency-Based Tabu (Long-Term Memory)

Maintain a frequency array F : F_i = number of times move i has been used. *Penalise frequently-used moves* by subtracting F_i from their evaluation. This pushes the search into unexplored regions of the space — long-term diversification.

3.8 Stochastic Local Search

Deterministic methods are limited. What if we introduce randomness?

3.8.1 Random Walk — Pure Exploration

Algorithm 12 Random Walk

```

1: function RANDOMWALK(start,  $n$ )
2:    $node \leftarrow start$ ;  $bestNode \leftarrow node$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $node \leftarrow \text{RANDOMCHOOSE}(\text{MOVEGEN}(node))$ 
5:     if  $node$  better than  $bestNode$  then  $bestNode \leftarrow node$ 
6:   end if
7: end for
8:   return  $bestNode$ 
9: end function

```

Pure exploration — ignores the heuristic entirely. Surprisingly effective on some problems, but slow on most.

3.8.2 Iterated Hill Climbing

Why insist on one starting point? Run Hill Climbing many times from random starts; return the best result.

Algorithm 13 Iterated Hill Climbing

```

1: function ITERATEDHC( $k$ )
2:    $bestSeen \leftarrow \text{null}$ 
3:   for  $i \leftarrow 1$  to  $k$  do
4:      $start \leftarrow \text{randomCandidate}()$ 
5:      $node \leftarrow \text{HILLCLIMBING}(start)$ 
6:     if  $node$  better than  $bestSeen$  then  $bestSeen \leftarrow node$ 
7:   end if
8: end for
9:   return  $bestSeen$ 
10: end function

```

Property. The probability of finding the global optimum approaches 1 as $k \rightarrow \infty$ (provided every state has nonzero probability of being a starting point).

3.8.3 Stochastic Hill Climbing

Pick a *random* neighbour (rather than the best); accept it with a probability depending on how much it improves things.

For maximization: let $\Delta E = h(v_{\text{new}}) - h(v_{\text{current}})$.

- $\Delta E > 0$ (better): accept with high probability.
- $\Delta E < 0$ (worse): accept with low probability.

The natural choice is the **sigmoid (Boltzmann)** acceptance probability:

$$P(\text{accept}) = \frac{1}{1 + e^{-\Delta E/T}}$$

where $T > 0$ is a **temperature** parameter.

3.8.4 Simulated Annealing — the Star of the Show

Annealing in metallurgy: heat a metal until atoms can move freely; cool it slowly so atoms settle into a low-energy crystalline state. *The slow cooling is essential* — quench it (cool fast) and you trap defects.

Simulated annealing (Kirkpatrick, Gelatt, Vecchi, 1983): start with high temperature T (lots of random moves accepted), gradually lower T (the search becomes greedier).

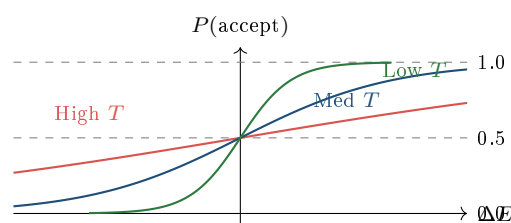
Algorithm 14 Simulated Annealing

```

1: function SIMULATEDANNEALING(start)
2:    $node \leftarrow \text{start}; \text{ bestNode} \leftarrow node$ 
3:    $T \leftarrow T_0$  ▷ some large initial temperature
4:   for time  $\leftarrow 1$  to numEpochs do
5:     for some inner-loop iterations do
6:        $neighbour \leftarrow \text{RANDOMNEIGHBOUR}(node)$ 
7:        $\Delta E \leftarrow \text{EVAL}(neighbour) - \text{EVAL}(node)$ 
8:       if  $\text{RANDOM}(0, 1) < 1/(1 + e^{-\Delta E/T})$  then
9:          $node \leftarrow neighbour$ 
10:      end if
11:      if  $\text{EVAL}(node) > \text{EVAL}(\text{bestNode})$  then  $\text{bestNode} \leftarrow node$ 
12:      end if
13:    end for
14:     $T \leftarrow \text{COOLINGFUNCTION}(T, \text{time})$ 
15:  end for
16:  return bestNode
17: end function

```

3.8.5 Effect of Temperature on the Sigmoid Curve



- $T \rightarrow \infty$: every move (good or bad) accepted with $P \approx 0.5$. Behaves like Random Walk.
- $T = 0$: only improving moves accepted ($\Delta E > 0 \Rightarrow P = 1$, $\Delta E < 0 \Rightarrow P = 0$). Behaves like Hill Climbing.
- Intermediate T : a balance. The hotter the temperature, the more willing to accept bad moves.

3.8.6 Cooling Schedules

Common choices:

- **Linear:** $T_{k+1} = T_k - \delta$.
- **Exponential / geometric:** $T_{k+1} = \alpha T_k$ with $\alpha \in (0.8, 0.99)$.
- **Logarithmic:** $T_k = T_0 / \log(k+1)$. *Theoretically guaranteed to find the global optimum given infinite time, but impractically slow.*

3.8.7 Numerical Example (Michalewicz & Fogel)

Suppose $\text{eval}(v_c) = 107$, $T = 10$:

$\text{eval}(v_n)$	ΔE	$e^{\Delta E/T}$	$P(\text{accept})$
80	-27	0.067	0.06
100	-7	0.497	0.33
107	0	1.000	0.50
120	+13	3.669	0.79
150	+43	73.7	0.99

Effect of temperature for a fixed $\Delta E = -13$ (a bad move):

T	$e^{\Delta E/T}$	$P(\text{accept})$
1	0.000002	0.0000
5	0.0743	0.069
10	0.2725	0.214
20	0.52	0.342
50	0.77	0.435

As T falls, the chance of accepting a bad move shrinks.

3.8.8 When Does Simulated Annealing Shine?

Real-World Application

SA is fantastic on **jagged landscapes with many local optima** but a broad overall trend toward the global optimum. The slow cooling lets the search explore broadly when hot, then settle into a deep valley when cold. *Real applications:* VLSI placement, scheduling, image reconstruction, neural network training (early days), TSP.

3.9 Summary Table: All the Local Search Algorithms of Week 3

Algorithm	Idea	Space	Notes
Best-First Search	sort OPEN by h	exp	informed, not optimal
Hill Climbing	move to best neighbour if better	const	stuck at local opt.
Beam Search	keep b best at each level	$O(b)$	robust to single-path failures
VND	cascade HC with denser neighbourhoods	const	escapes shallow optima
Best-Neighbour	always move to best (record best seen)	const	bounces back without memory
Tabu Search	Best-Neighbour + forbidden list	$O(tt)$	escapes local opt. via memory
Random Walk	random next neighbour	const	pure exploration
Iterated HC	HC from many random starts	const	embarrassingly parallel
Stochastic HC	accept random nbhr by prob.	const	exploration + exploitation
Simulated Annealing	stoch. HC with cooling schedule	const	gold standard for hard landscapes

3.10 Common Mistakes Students Make

Common Mistake

1. Confusing Best-First Search with A^* . Best-First uses only h . A^* uses $f = g + h$. (Week 5.)
2. Assuming Best-First is optimal — it isn't.
3. Calling Hill Climbing “complete” — it isn't, because of local optima.
4. Forgetting that the heuristic must be *non-negative* and $h(\text{goal}) = 0$.
5. Confusing state-space search with solution-space search. State-space: a node is a partial situation; the path matters. Solution-space: a node is itself a candidate; only its quality matters.
6. Using too short a tabu tenure tt (algorithm cycles) or too long (algorithm too restricted).
7. Cooling too fast in SA (“quenching”) — guaranteed to get stuck in a local optimum.
8. Thinking $b = 1$ Beam Search differs from Hill Climbing. It doesn't.

3.11 Quick Revision — Week 3

Quick Revision

- **Heuristic function $h(n)$:** estimates distance to goal. Properties: $h \geq 0$, $h(\text{goal}) = 0$.
- **Best-First Search:** OPEN is a priority queue on h . Informed but not optimal.
- **Hill Climbing:** constant-space, greedy. Stops at local optima.

- Curses of HC: **local maxima, plateaus, ridges**.
- **State space vs. Solution space search**. **Synthesis** (constructive) vs. **Perturbation**.
- TSP heuristics: **Nearest-Neighbour, Greedy-Edges, Savings**. Operators: 2-opt, 3-opt.
- **Exploration vs. exploitation** — the central trade-off.
- **Beam Search**: keep b best nodes (=HC at $b = 1$, =Best-First at $b = \infty$).
- **VND**: cascade HC with denser neighbourhoods. **Tabu Search**: forbidden-list memory.
- **Aspiration**: a tabu move that beats best-seen is allowed.
- **Random Walk** (pure exploration), **Iterated HC** (random restarts), **Stochastic HC** (probabilistic acceptance).
- **Simulated Annealing**: stochastic HC with temperature T that decreases over time. $P(\text{accept}) = 1/(1 + e^{-\Delta E/T})$.
- As $T \rightarrow \infty$ SA $\rightarrow$ Random Walk; as $T \rightarrow 0$ SA $\rightarrow$ Hill Climbing.
- Cooling schedules: linear, geometric, logarithmic. Slow $\Rightarrow$ better solutions.

3.12 Most Important Quiz Points — Week 3

Exam Tip

1. Hill Climbing is **constant space, linear time, not complete, not optimal**.
2. Best-First with $b = 1$ behaves like HC; with $b = \infty$ it is full Best-First.
3. Beam Search with width $b = 1$ is exactly Hill Climbing.
4. The acceptance probability of SA, $1/(1 + e^{-\Delta E/T})$, is a sigmoid in ΔE .
5. SA at high $T \approx$ Random Walk; at $T = 0 \approx$ Hill Climbing.
6. Tabu Search uses a **short-term memory**; the aspiration criterion allows tabu moves if they beat the best-seen.
7. Manhattan distance $\geq$ Hamming distance for the 8-puzzle (more discriminative, more informative).
8. TSP is NP-hard; SAT is NP-complete. Search spaces $n!$ vs. 2^n respectively.
9. Synthesis methods build a solution piece by piece (constructive); perturbation methods modify an existing complete candidate.
10. Iterated HC: probability of finding the global optimum $\rightarrow 1$ as the number of restarts $\rightarrow \infty$.

3.13 Practice Questions — Week 3

Conceptual.

1. Why are DFS and BFS called “blind” or “uninformed”?
2. Define a heuristic function. What two properties must it satisfy?
3. How does Best-First Search differ from BFS and DFS implementation-wise?

4. Explain the exploration-exploitation trade-off using Hill Climbing and Random Walk as the two extremes.
5. Compare Hamming and Manhattan distance heuristics for the 8-puzzle. Which is more informative? Why?
6. Why is Hill Climbing not complete? Illustrate with the 8-puzzle local minimum.
7. In what sense is Beam Search a generalization of Hill Climbing? When does it degenerate to BFS-like behaviour?
8. Explain Variable Neighbourhood Descent. Why does it work?
9. What is the role of the tabu tenure in Tabu Search? What happens if it is too small? Too large?
10. State the aspiration criterion in Tabu Search and explain why it is needed.
11. Why is logarithmic cooling theoretically optimal for SA but impractical?

Numerical.

12. Compute h_H and h_M for the 8-puzzle state (1 3 2 / 4 5 6 / 7 8 _) relative to the standard goal (1 2 3 / 4 5 6 / 7 8 _).
13. For the SAT formula F given in the chapter, list all $2^5 = 32$ candidates and compute h for each. Identify all local optima of HC with the N_1 neighbourhood.
14. In SA with $T = 10$, $\text{eval}(v_c) = 100$, $\text{eval}(v_n) = 92$, find $P(\text{accept})$.
15. Repeat the above with $T = 2$. Comment on the difference.
16. For a TSP with 10 cities, compute the number of tours examined by brute force vs. by 2-opt (one full pass).

Algorithmic / proof.

17. Write Best-First Search in pseudocode. Justify why it is not optimal.
18. Trace Hill Climbing on a small SAT example, showing OPEN and the current node at each step.
19. Prove that Iterated Hill Climbing finds the global optimum with probability 1 as $k \rightarrow \infty$, assuming every state has nonzero probability of being a starting state and HC converges in finite steps.
20. Write Beam Search pseudocode and discuss what happens for $b = 1$ and $b = \infty$.
21. Show that with $T \rightarrow \infty$ SA is equivalent to a Random Walk, and with $T \rightarrow 0$ it is equivalent to Hill Climbing.

MCQ-style.

22. Hill Climbing is:
 - (a) complete and optimal
 - (b) complete only
 - (c) optimal only
 - (d) neither.
 Answer: (d).
23. Which of the following is **not** a problem for Hill Climbing?
 - (a) local maxima
 - (b) plateaus
 - (c) ridges
 - (d) exponential space.
 Answer: (d) — HC uses constant space.
24. The acceptance probability in SA depends on:
 - (a) ΔE only
 - (b) T only
 - (c) both ΔE and T
 - (d) neither.
 Answer: (c).

25. Best-First Search uses:

- (a) $h(n)$ only (b) $g(n)$ only (c) $f(n) = g + h$ (d) $h(n) \cdot g(n)$.

Answer: (a).

26. Beam Search with width 1 is equivalent to:

- (a) Best-First (b) Hill Climbing (c) BFS (d) Random Walk.

Answer: (b).

27. Tabu Search is best characterised as:

- (a) deterministic local search with memory (b) pure exploration (c) stochastic search with random restarts (d) a backtracking algorithm.

Answer: (a).

End of Week 3.

4 Week 4: Population-Based Methods, Genetic Algorithms, and Ant Colony Optimization

“It takes two to invent anything. The one makes up combinations and the other chooses.”

— Paul Valéry

Goal of Week 4. Hill Climbing tracks *one* candidate; Simulated Annealing improves it but still only *one*. What if we maintain a whole *population* of candidates and let them interact? This is the central idea of **population-based search**. We’ll meet **Genetic Algorithms** (inspired by Darwinian evolution and Mendelian genetics) and **Ant Colony Optimization** (inspired by how real ants find shortest paths to food using pheromone trails). These are powerful, general optimisation tools that have solved real industrial problems for decades.

4.1 Why Populations? Recap of Earlier Methods

The progression so far

DFS / BFS / DFID	one node, blind search
Best-First	one node, heuristic-guided
Hill Climbing	one node, greedy local
Simulated Annealing	one node, stochastic local
Genetic Algorithms	a population, evolved through generations
Ant Colony	a population (colony) sharing trail memory

The motivation: a single trajectory can get trapped on a single peak. A *distributed* search — many candidates exploring different regions — is much more robust and can also *combine information* from different candidates.

4.2 Lessons from Nature: Evolution as Search

4.2.1 “Things That Persist, Persist”

Steve Grand, in *Creation: Life and How to Make It*, claims he has uncovered the most important law of nature:

Grand’s Law

Things that persist, **persist**. Things that don’t, **don’t**.

Trivial? Look again. A cloud on a mountain peak appears to “persist” — yet the actual water molecules are constantly moving through. The cloud is a *pattern* that persists because the conditions that create it persist. Your body is the same: the atoms making you up today differ entirely from those that made you as a child, yet “you” persist as a pattern.

Evolution is the biological face of this law: lifeforms that survive, survive. The ones that don’t, don’t. Over billions of years, this trivial-sounding mechanism produced eyes, wings, brains, and us.

4.2.2 Competition and “Survival of the Fittest”

Resources are limited $\Rightarrow$ competition. Species compete (lions vs. hyenas for prey). Crucially *within* a species also: the fastest foxes catch rabbits, the fastest rabbits escape foxes. Darwin called this **natural selection**; Spencer popularised the phrase “survival of the fittest.”

4.2.3 Bisexual Reproduction — Nature’s Genius Hack

If reproduction were cloning, no design would change. Nature’s master move: **mix two genomes into a child**. Every sibling is unique. The child’s two parents have already passed natural selection — so their child starts from already-good genes, mixed in a new way that might be even better.

Richard Dawkins (*The Blind Watchmaker*) calls this a **ratchet mechanism**: a good design, once found, propagates and persists; new combinations are explored; good ones replace bad ones.

4.2.4 Genotype vs. Phenotype

Definition

Genotype: the genetic make-up — the DNA / chromosomes / genes.

Phenotype: the physical expression of those genes — the actual body.

The mixing happens at the *gene* (genotype) level. The competition happens at the *body* (phenotype) level. Both are required.

Intuition

Analogy in computing: the genotype is the *source code*; the phenotype is the *running program*. Genetic algorithms work on source code (bit strings, permutations) and evaluate the running program (the fitness).

4.3 Genetic Algorithms (GAs)

Definition

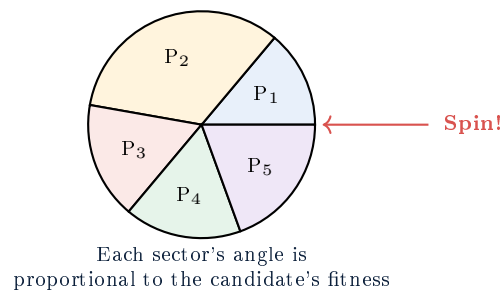
Genetic Algorithm (Holland, 1975; popularized by Goldberg, 1989). A class of *stochastic adaptive heuristic* optimisation algorithms that maintain a **population** of candidate solutions, evaluate them via a **fitness function**, and combine the fittest using **selection**, **crossover**, and **mutation** to produce the next generation.

4.3.1 The Three Operators

1. **Selection (Reproduction).** Clone each candidate *in proportion to its fitness*. Fitter candidates may get multiple copies; less-fit ones may die out.
2. **Crossover (Recombination).** Randomly pair the resulting population; for each pair, mix their genes to produce two offspring.
3. **Mutation.** With small probability, flip one or more genes of an offspring (introduces fresh genetic material).

4.3.2 Selection: The Roulette Wheel

The standard implementation of fitness-proportionate selection:

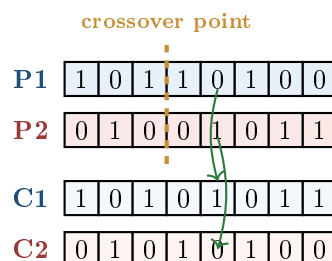


Each candidate P_i in the parent population occupies an angle $\theta_i \propto \text{fitness}(P_i)$. Spin the wheel n times (for a population of size n); each spin selects one parent to clone.

In code: pick a uniform random number $r \in [0, F_{\text{total}}]$ where $F_{\text{total}} = \sum \text{fitness}(P_i)$. Walk through the population accumulating fitness; the first P_i whose accumulated fitness exceeds r is selected.

4.3.3 Crossover: Mixing the Genes

The simplest scheme is **single-point crossover**:



Pick a random crossover point. The first child gets the left part of P_1 and the right part of P_2 ; the second child gets the reverse.

Other operators: **two-point crossover** (swap a middle segment), **uniform crossover** (each gene independently inherited from P_1 or P_2 with probability $\frac{1}{2}$).

4.3.4 Mutation: Injecting Fresh Genetic Material

For a bit-string representation, flip a randomly-chosen bit with small probability (typically 0.001 to 0.05). Without mutation, if a gene happens to be absent from the initial population it can never appear later — diversity dies, and the algorithm stalls.

4.3.5 The Algorithm

Algorithm 15 Genetic Algorithm (basic version)

```

1: function GA()
2:    $P \leftarrow$  random initial population of  $n$  candidates
3:   while not termination do
4:     compute fitness( $c$ ) for each  $c \in P$ 
5:      $S \leftarrow$  select  $n$  candidates from  $P$  with probability  $\propto$  fitness (with replacement)
6:      $O \leftarrow$  pair members of  $S$  randomly; apply crossover to each pair  $\rightarrow n$  offspring
7:     with small probability, mutate each offspring
8:      $P \leftarrow$  replace the  $k$  weakest of  $P$  with the  $k$  strongest of  $O$  (elitist)   or    $P \leftarrow O$ 
9:   end while
10:  return best member of  $P$ 
11: end function
  
```

Remember This

Elitism: keeping the top- k of the previous generation in the new population guarantees that the best fitness never decreases. Almost all real GAs do this.

4.4 Worked Example — Goldberg’s Tiny Problem

Problem. Chromosome = 5-bit string interpreted as a binary integer $x \in \{0, \dots, 31\}$. Fitness $f(x) = x^2$. Population size $n = 4$.

4.4.1 Generation 0

#	Bit string	x	$f(x) = x^2$	P_{sel}	Expected count
1	01101	13	169	0.144	0.58
2	11000	24	576	0.492	1.97
3	01000	8	64	0.055	0.22
4	10011	19	361	0.309	1.23
Total			1170		
Average			293		

$P_{\text{sel}} = f(x_i)/1170$. Expected number of copies = $4 \cdot P_{\text{sel}}$.

4.4.2 After Selection

Suppose the roulette wheel gives copies $\{1, 2, 2, 4\}$ — that is, candidate 2 cloned twice; candidate 3 dropped. New (cloned) population:

01101 | 11000 | 11000 | 10011

4.4.3 After Crossover

Pair (C_1, C_2) and (C_3, C_4) randomly. Suppose crossover points are 4 and 2:

Parent	Crossover at	Child	x	$f(x)$
0110 1	4	0110 0	12	144
1100 0		1100 1	25	625
11 000	2	11 0 11	27	729
10 011		10 0 00	16	256

New total fitness: $144 + 625 + 729 + 256 = 1754$. **Average:** 438.5.

The population has improved: average fitness rose from 293 to 438.5; the best individual rose from 24 ($f = 576$) to 27 ($f = 729$).

Intuition

This is the key empirical claim of GAs: *the population becomes fitter, generation after generation*. Goldberg's tiny example shows the effect after just one cycle.

4.4.4 Diversity Warning

Watch what happens in cycle 2 if both “11000” and one “10011” get heavy representation: you may end up with *four* candidates, all of which have a 0 in some position. That gene is **lost forever from the population** (no crossover of zeros can produce a one). The optimum 11111 becomes unreachable without mutation.

Quiz Trap

The diversity problem: with a small population, alleles can drift out and never return.
Solutions: (a) use a *large* population, (b) inject mutation, (c) periodically restart.

4.4.5 The Cheetah Cautionary Tale

The cheetah is a magnificent hunter, but its population has very low genetic diversity (the species passed through a near-extinction bottleneck $\sim 10,000$ years ago). It is exquisitely fit *for its old environment* but cannot evolve quickly enough to handle a changing world. This is exactly the GA pitfall: too much exploitation, too little diversity.

4.5 Solving TSP with Genetic Algorithms

Setup. Each candidate is a permutation of cities $[c_1, c_2, \dots, c_n]$ (the tour visits them in this order and returns to c_1). Fitness = (negative of) tour length — minimisation problem.

4.5.1 Why Single-Point Crossover Doesn't Work for TSP

Take two parents $P_1 = [1, 2, 3, 4, 5, 6, 7, 8]$ and $P_2 = [3, 7, 1, 5, 8, 2, 6, 4]$, and cut at position 4:

$$C_1 = [1, 2, 3, 4, \mid 8, 2, 6, 4]$$

C_1 visits city 2 twice and city 4 twice; cities 5, 7 are missing — **not a valid tour!** TSP needs *permutation-preserving* crossover operators.

4.5.2 Cycle Crossover (CX)

Find “cycles” between P_1 and P_2 where positions index the same cities. Copy alternate cycles from each parent.

4.5.3 Partially Mapped Crossover (PMX, Goldberg & Lingle, 1985)

Choose two crossover points. Copy the segment between them from P_1 to C_1 . For the rest of C_1 , copy from P_2 *unless* that city is already in C_1 — in which case use a mapping derived from the swapped segment.

4.5.4 Order Crossover (OX, Davis, 1985)

Choose two crossover points. Copy the segment from P_1 to C_1 . Fill the rest of C_1 by reading P_2 from *after* the second crossover point, in cyclic order, skipping cities already in C_1 .

4.5.5 Alternating-Edges Crossover (Adjacency Representation)

Represent a tour as adjacency list. Start at some city X . Choose the next city Y from P_1 . From Y , choose the next from P_2 . Alternate. (If the alternation creates a premature cycle, pick a random unvisited city.)

4.5.6 Heuristic Crossover

From each city, choose the next-edge from whichever parent has the *shorter* edge to an unvisited city. This injects heuristic information into the crossover itself.

4.5.7 Ordinal Representation (clever encoding)

Instead of storing the actual sequence $[3, 1, 4, 2, 5]$, store the *position within a remaining list* at each step:

- Reference list: $[1, 2, 3, 4, 5]$.
- Pick city at position **3**: that's 3. Reference becomes $[1, 2, 4, 5]$.
- Pick position **1**: that's 1. Reference becomes $[2, 4, 5]$.
- Pick position **2**: that's 4. Reference becomes $[2, 5]$.
- Pick position **1**: that's 2. Reference becomes $[5]$.
- Pick position **1**: that's 5.

So the tour $[3, 1, 4, 2, 5]$ encodes as $[3, 1, 2, 1, 1]$.

Remember This

Beautiful property of ordinal representation: *any* string with i -th component in $\{1, \dots, n - i + 1\}$ is a valid tour. So **single-point crossover on ordinal strings always produces valid children!**

4.6 Practical Tips for Genetic Algorithms

GA design checklist

1. **Encoding** — design genes/chromosomes so that small changes correspond to small phenotype changes (good “locality”).
2. **Fitness** — must be cheap to compute (you’ll call it millions of times) and meaningful (avoid plateaus).
3. **Population size** — usually 50–500 for small problems; thousands for hard ones. Larger = more diverse but slower.
4. **Crossover rate** — typically 0.6–0.95.
5. **Mutation rate** — typically 0.001–0.05. Too high = random walk; too low = premature convergence.
6. **Selection** — roulette wheel, tournament (pick k random candidates, take the fittest), rank-based.
7. **Elitism** — keep top few candidates intact across generations.
8. **Termination** — fixed generations, no improvement for N generations, or target fitness reached.

4.7 Ant Colony Optimization (ACO)

4.7.1 Real Ants and Pheromones

Real ants find shortest paths to food sources without any central coordination. The trick is **stigmergy** — indirect communication via the environment:

1. An ant wanders randomly looking for food.
2. Finding food, it returns to the nest, depositing **pheromone** on its trail.
3. Other ants smell the pheromone and probabilistically prefer to follow it.
4. More ants follow $\Rightarrow$ more pheromone $\Rightarrow$ even more ants follow (positive feedback).
5. **Crucial:** pheromone **evaporates** over time.

Why shorter trails dominate. A shorter trail is traversed more often per unit time, so it accumulates pheromone faster, while evaporation gradually wipes the longer trails.

This is a classic example of **emergent behaviour**: simple agents following simple rules collectively solve a hard problem.

4.7.2 ACO Algorithm (Dorigo, 1992)

Definition

Ant Colony Optimization. An optimisation algorithm inspired by the bio-semiotic pheromone trail communication of ants. A *colony of artificial ants* repeatedly constructs solutions via a stochastic greedy method that combines a heuristic function with the strength of pheromone trails on the graph.

For TSP: imagine the cities as a graph; each edge (i, j) has a pheromone level τ_{ij} . Each iteration, every ant constructs a complete tour stochastically.

Probabilistic city selection. At city i , ant k moves to city j with probability:

$$P_{ij}^k = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{u \in \text{allowed}_k} [\tau_{iu}]^\alpha \cdot [\eta_{iu}]^\beta}$$

where:

- τ_{ij} = pheromone on edge (i, j) (learnt over iterations).
- $\eta_{ij} = 1/d_{ij}$ = heuristic “desirability” of edge (i, j) (shorter edge = more desirable).
- α, β = exponents controlling relative importance.
- “allowed _{k} ” = cities not yet visited by ant k .

Pheromone update. After all ants finish their tours, each ant k deposits pheromone on its tour edges:

$$\Delta\tau_{ij}^k = \begin{cases} Q/L_k & \text{if edge } (i, j) \text{ is in ant } k\text{'s tour} \\ 0 & \text{otherwise} \end{cases}$$

where L_k is the length of ant k 's tour and Q is a constant. Shorter tours $\Rightarrow$ more pheromone.

Evaporation. The pheromone on every edge decays:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_k \Delta\tau_{ij}^k$$

$\rho \in (0, 1)$ is the evaporation rate (typically 0.1–0.5).

4.7.3 The Algorithm

Algorithm 16 Ant Colony Optimization (for TSP)

```

1: initialise  $\tau_{ij}$  to small constant for all edges
2: for  $t = 1$  to maxIterations do
3:   for each ant  $k = 1, \dots, m$  do
4:     place ant  $k$  at random starting city
5:     while tour incomplete do
6:       choose next city  $j$  from current  $i$  with probability  $P_{ij}^k$ 
7:     end while
8:     compute  $L_k$  = length of ant  $k$ 's tour
9:   end for
10:  for each edge  $(i, j)$  do
11:     $\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \sum_k \Delta\tau_{ij}^k$ 
12:  end for
13:  (optional) update bestSoFar tour
14: end for
15: return bestSoFar

```

4.7.4 Why ACO Works

Two complementary forces:

- **Exploitation:** edges on good tours accumulate pheromone, attracting more ants.
- **Exploration:** stochastic choices and evaporation prevent the colony from getting locked into a single tour too early.

Remember This

ACO is a **swarm intelligence** method — a member of a broader family including Particle Swarm Optimisation (PSO), Bee Colony Optimisation, Firefly Algorithms, etc. They all share the philosophy of *decentralised, indirect coordination among simple agents*.

4.8 Where Do These Algorithms Shine?**Real-World Application**

Genetic Algorithms have been used for: antenna design (NASA's ST5 spacecraft antenna), engineering design optimisation, scheduling (job-shop, university timetabling), evolved circuit design, financial portfolio optimisation, neural network architecture search.

Ant Colony Optimization has been used for: vehicle routing, telecommunications network routing, protein folding, scheduling, and combinatorial optimisation generally — anywhere you can map the problem to “find a good path through a graph.”

4.9 Comparison Table: Single-Point vs. Population-Based Methods

Aspect	Single-point (HC, SA, Tabu)	Population-based (GA, ACO)
State held	1 candidate	many candidates / pheromone trails
Memory	$O(1)$ to $O(\text{tabu list})$	$O(\text{population size})$
Combines info from many candidates	no	yes (key power)
Embarrassingly parallel	no (one trajectory)	yes (evaluate each candidate independently)
Diversity preserved by	temperature, restarts	population, mutation, evaporation

4.10 Common Mistakes**Common Mistake**

1. Using *single-point crossover* for a permutation problem (TSP, scheduling). Use PMX, OX, CX, or ordinal representation.
2. Setting the mutation rate too high — the GA degenerates to random search.
3. Setting the mutation rate to zero — diversity dies, you get premature convergence.
4. Using too small a population — alleles get lost.
5. Forgetting evaporation in ACO — the strongest path becomes irreversibly locked in.
6. Confusing fitness with cost in a minimisation problem. Either flip signs, or use $\text{fitness} = 1/(1 + \text{cost})$.
7. Thinking that GAs guarantee the optimum. They don't; they are stochastic heuristics.

4.11 Quick Revision — Week 4

Quick Revision

- **Population-based search** keeps many candidates and lets them interact.
- **GA Operators:** Selection (fitness-proportionate, e.g. roulette wheel), Crossover (single-point / multi-point / uniform / PMX / OX / cycle / heuristic / ordinal), Mutation (low probability bit flip).
- **Genotype** = encoding (chromosome of genes); **phenotype** = the candidate solution / fitness.
- **Fitness function** is the key user-defined input; computed many times, so must be cheap.
- Diversity is essential. Mutation and large populations preserve it.
- **Elitism** = keep top- k of each generation; guarantees fitness never decreases.
- TSP needs special crossover (**PMX, OX, cycle, alternating-edges, ordinal**) to preserve permutations.
- **Ant Colony Optimization (Dorigo, 1992):** colony of artificial ants; transition probability blends pheromone τ (learnt) with heuristic η (problem-specific); pheromone deposit $\propto 1/L_k$; evaporation rate ρ keeps the search exploring.
- Both GAs and ACO are **embarrassingly parallel** — natural fit for multicore / GPU.
- Connection to Week 1: ACO is the algorithmic version of the **biosemiotics** mentioned by Khemani (sign-mediated coordination).

4.12 Most Important Quiz Points — Week 4

Exam Tip

1. GAs were devised by **John Holland (1975)**; ACO by **Marco Dorigo (1992)**.
2. In a GA, *selection* is fitness-proportionate; *crossover* mixes genes from two parents; *mutation* introduces fresh material.
3. Roulette-wheel selection assigns each candidate an angle $\propto$ fitness.
4. Single-point crossover does **not** work for permutation problems like TSP.
5. PMX, OX, CX are permutation-preserving crossover operators for TSP.
6. ACO transition probability: $P_{ij}^k \propto [\tau_{ij}]^\alpha [\eta_{ij}]^\beta$.
7. ACO pheromone deposit by ant k on its tour edges: $\Delta\tau = Q/L_k$.
8. Pheromone evaporation factor $(1 - \rho)$ each iteration.
9. Without evaporation, ACO locks into a single tour permanently — exploration dies.
10. Both GA and ACO are stochastic, not guaranteed to find global optima.

4.13 Practice Questions — Week 4

Conceptual.

1. Distinguish genotype and phenotype with an example.

2. Why does single-point crossover fail for TSP? Give a small example.
3. Explain the roulette-wheel selection mechanism, and outline pseudocode for it.
4. What is elitism? Why is it almost always used?
5. Why is mutation necessary? What happens if the mutation rate is too high? Too low?
6. Compare GAs and Simulated Annealing along (a) state held, (b) parallelism, (c) source of diversity.
7. Describe stigmergy in real ant colonies. How is it abstracted in ACO?
8. Explain the role of α and β in the ACO transition probability. What happens in the limits $\alpha = 0$ and $\beta = 0$?
9. Why is evaporation essential in ACO?
10. Pick one real-world problem and outline how you would solve it with (i) a GA, (ii) ACO.

Numerical.

11. Goldberg's 5-bit problem: starting population [01101, 11000, 01000, 10011] with $f(x) = x^2$. Verify the selection probabilities and expected counts given in the chapter.
12. For a population of 6 candidates with fitnesses {10, 30, 20, 40, 50, 25}, list the cumulative fitness and the candidate selected for each of three random spins $r_1 = 85$, $r_2 = 125$, $r_3 = 160$.
13. In ACO, suppose $\tau_{ij} = 2.0$, $\eta_{ij} = 0.5$, $\alpha = 1$, $\beta = 2$ for one edge; and for an alternative edge $\tau = 1.0$, $\eta = 1.0$, with the same α, β . Find the relative probability of choosing each.
14. Compute the pheromone deposit for an ant whose tour length is 25, with $Q = 100$.

MCQ-style.

15. Which is NOT a genetic operator?
 (a) Selection (b) Crossover (c) Mutation (d) Annealing.
Answer: (d).
16. Roulette-wheel selection assigns each candidate an angle proportional to:
 (a) the rank (b) the fitness (c) the chromosome length (d) random.
Answer: (b).
17. Which crossover operator works on permutation chromosomes (TSP)?
 (a) single-point (b) PMX (c) uniform (d) all of the above.
Answer: (b). Single-point and uniform crossover can produce illegal permutations.
18. In ACO the pheromone update for an ant with tour length L_k is proportional to:
 (a) L_k (b) $1/L_k$ (c) L_k^2 (d) $\log L_k$.
Answer: (b).
19. Evaporation in ACO mainly enables:
 (a) faster convergence (b) exploration (c) parallelism (d) elitism.
Answer: (b).

End of Week 4.

5 Week 5: Finding Optimal Paths – Branch & Bound, Dijkstra, and Algorithm A*

“Best-First Search has a sense of direction, but no sense of cost. Branch & Bound has a sense of cost, but no sense of direction. A has both — and is provably optimal.”*

Goal of Week 5. Last week’s algorithms found a solution; this week we want the *cheapest* solution. We start from the slow-but-correct **Branch & Bound**, refine it via **Dijkstra’s algorithm**, and then add a heuristic to get **A*** — the most celebrated algorithm in classical AI. We will prove A* is *admissible* (always returns optimal) and also *the most efficient possible* among admissible algorithms with the same heuristic.

5.1 Two Search Spaces, Two Forms of Branch and Bound

5.1.1 Heuristic Refinement and the “Refinement Space”

There is an alternative, more abstract way to view search. Instead of incrementally extending a partial *path* (state-space search), we can incrementally *refine* an abstract description of a complete solution (refinement-space search).

Heuristic Refinement / Branch and Bound on the solution set

- The initial “solution set” contains *all* possible solutions.
- At each step, **partition** a set into two smaller sets.
- Each set is tagged with an *estimated* (lower-bound) cost.
- Refine the set with the *smallest* estimated cost next.
- Terminate when a fully specified solution is cheaper than the lower bound of every other set.

For TSP, refinement steps might be “all tours containing edge (A, B) ” vs. “all tours *not* containing edge (A, B) ”. The cost estimate sums up known committed-edge costs plus a lower bound for the unfinished part.

Intuition

Branch = partition the candidate-set; **Bound** = use a lower-bound cost estimate to prune sets that can’t beat what we already have. *The higher the lower bound, the more pruning.*

5.1.2 The Key Principle of Branch and Bound

Soundness condition for B&B

An optimal solution is **guaranteed** if the estimated cost is a *lower bound* on the actual cost.

A solution will never be cheaper than its estimate. So if a fully refined solution is cheaper than the estimate of a partially refined one, the latter *cannot* contain a better solution and can be pruned.

Trade-off:

- **Estimate = 0 (the trivial bound):** always valid, but prunes nothing.
- **Higher estimates:** more pruning, more computation per node.
- **Perfect estimate (h^*):** no search at all needed, but generally impossible to compute.

5.2 Branch and Bound in State Space — the Cheapest-First Method

In state-space terms, Branch & Bound is simply “always extend the partial path with the smallest known cost so far.”

Algorithm 17 Branch & Bound (state-space, no heuristic)

```

1:  $OPEN \leftarrow [(start, nil, g=0)]; \quad CLOSED \leftarrow []$ 
2: while  $OPEN \neq []$  do
3:    $(N, parent, g) \leftarrow$  node from  $OPEN$  with smallest  $g$ 
4:    $OPEN \leftarrow OPEN \setminus \{(N, parent, g)\}$ 
5:   if GOALTEST( $N$ ) then return path
6:   end if
7:    $CLOSED \leftarrow (N, parent, g) :: CLOSED$ 
8:   for all  $(c, edgeCost)$  in MoveGen( $N$ ) do
9:      $g_c \leftarrow g + edgeCost$ 
10:    insert  $(c, N, g_c)$  into  $OPEN$ , replacing any old entry if  $g_c$  is smaller
11:   end for
12: end while
13: return FAILURE

```

Properties. B&B always finds the optimal-cost path on graphs with non-negative edge costs. But it has **no sense of direction** — it expands cheap nodes anywhere in the graph, regardless of whether they head toward the goal.

5.3 Dijkstra’s Algorithm — Branch and Bound with Relaxation

Dijkstra (1959) is essentially the same idea, formalised:

Algorithm 18 Dijkstra's Shortest-Path Algorithm

```

1: for all nodes  $v$ :  $\text{dist}[v] \leftarrow \infty$ ,  $\text{colour}[v] \leftarrow \text{white}$ 
2:  $\text{dist}[\text{start}] \leftarrow 0$ 
3: while any node is white do
4:    $u \leftarrow$  white node with smallest  $\text{dist}[u]$ 
5:    $\text{colour}[u] \leftarrow \text{black}$ 
6:   for all neighbours  $v$  of  $u$  do
7:     if  $\text{dist}[u] + w(u, v) < \text{dist}[v]$  then  $\triangleright$  relaxation
8:        $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ 
9:        $\text{parent}[v] \leftarrow u$ 
10:    end if
11:  end for
12: end while

```

Relaxation is the key step (line 7): whenever we find a cheaper way to reach v , we update v 's distance and parent.

Why Dijkstra works. The first time a node turns black, its distance is provably optimal — because all white nodes have distance $\geq$ the current node's distance, and edge weights are non-negative.

Remember This

Dijkstra is essentially B&B with the additional bookkeeping of relaxation. It finds shortest paths from the start to *all* other nodes (single-source shortest paths). Its drawback is the same as B&B's: **no sense of direction**.

5.4 Algorithm A* — Combining Cost and Heuristic

Peter Hart, Nils Nilsson, Bertram Raphael (Stanford Research Institute, 1968). A* married Branch & Bound (cost) with Best-First Search (heuristic) into one algorithm.

Definition

Algorithm A*. A search algorithm that always expands the node n with the smallest *f*-value:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = cost of the best path from start to n found so far (B&B contribution),
- $h(n)$ = heuristic estimate of the cost from n to the goal (Best-First contribution).

5.4.1 Pseudocode of A*

Algorithm 19 A* search

```

1:  $OPEN \leftarrow$  priority queue  $\{(\text{start}, \text{nil}, g=0, h(\text{start}))\}$ 
2:  $CLOSED \leftarrow \{\}$ 
3: while  $OPEN \neq \emptyset$  do
4:    $(N, \text{parent}, g_N, h_N) \leftarrow$  extract-min by  $f = g_N + h_N$  from  $OPEN$ 
5:   if GOALTEST( $N$ ) then return path
6:   end if
7:   move  $N$  to  $CLOSED$ 
8:   for all successors  $c$  of  $N$  with edge cost  $w$  do
9:      $g_c \leftarrow g_N + w$ ;  $f_c \leftarrow g_c + h(c)$ 
10:    if  $c \notin OPEN$  and  $c \notin CLOSED$  then
11:      insert  $(c, N, g_c, h(c))$  into  $OPEN$  ▷ Case 1: new node
12:    else if  $c \in OPEN$  and  $g_c <$  current  $g$ -value of  $c$  then
13:      update  $c$ 's  $g$ -value and parent in  $OPEN$  ▷ Case 2: better path to OPEN node
14:    else if  $c \in CLOSED$  and  $g_c <$  current  $g$ -value of  $c$  then
15:      remove  $c$  from  $CLOSED$ , insert with new  $g, f$  into  $OPEN$  ▷ Case 3: better path to CLOSED node
16:    end if
17:  end for
18: end while
19: return FAILURE
  
```

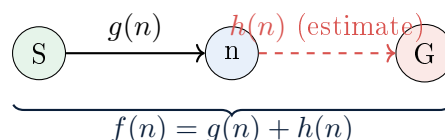
5.4.2 Why Three Cases? Why “Re-open” CLOSED Nodes?

This is the most subtle point. In Dijkstra, once a node is black, its distance is provably optimal. In A* this is *not* the case — A* sorts by $f = g + h$, not just g , so a node may be picked early because of a small h , before we’ve found its cheapest g .

Quiz Trap

Crucial fact: unless h is *monotone* (Week 6), A* may discover a cheaper path to a node that’s already been closed. The Case-3 “re-opening” handles this. Forgetting Case 3 is a classic A* bug.

5.4.3 Visual Intuition for $f = g + h$



$f(n)$ estimates the cost of the cheapest path *through* n from start to goal. A* always picks the node whose f is smallest — the path most likely to be cheap overall.

5.4.4 When $h \equiv 0$, A* Becomes Dijkstra

If $h(n) = 0$ for all n , then $f = g$, and A* expands nodes in increasing order of g — that is Dijkstra. Without a heuristic, A* loses its sense of direction but is still correct.

Remember This**Special-case lineage:**

- $h \equiv 0$, all edges unit cost $\Rightarrow A^* = \text{BFS}$.
- $h \equiv 0$, general edges $\Rightarrow A^* = \text{Dijkstra}$ (= uniform-cost search).
- $g \equiv 0 \Rightarrow A^* = \text{Best-First Search}$ (loses optimality).

5.5 Admissibility of A***5.5.1 Definitions****Definition**

Admissible algorithm. A search algorithm is **admissible** if it is guaranteed to return an optimal-cost solution whenever one exists.

Admissible heuristic. A heuristic h is admissible if $h(n) \leq h^*(n)$ for every node n , where $h^*(n)$ is the *true* optimal cost from n to the nearest goal. (I.e., h never overestimates.)

5.5.2 Notation

For the proofs we use these standard notations:

- $g^*(n)$ = optimal-path cost from start S to node n .
- $h^*(n)$ = optimal-path cost from node n to a goal G .
- $f^*(n) = g^*(n) + h^*(n)$ = optimal-path cost from S to G *via* n .
- Note: $g^*(n) \leq g(n)$ always (we might have found a non-optimal path so far).

5.5.3 Conditions for A* to Be Admissible

A* is admissible if and only if all three hold:

1. **Finite branching factor.** Otherwise we cannot even generate successors.
2. **Edge costs bounded below by some $\varepsilon > 0$.** Earlier statements only required “positive cost,” but Arvind Narayanan (an IIT Madras student in the mid-1990s) pointed out that with edge costs $1, \frac{1}{2}, \frac{1}{4}, \dots$ you can have an infinite path of finite total cost — A* could spend forever exploring it.
3. **h is admissible:** $h(n) \leq h^*(n)$ for all n .

5.5.4 Proof Outline — Six Lemmas

The proof proceeds through six lemmas (after Hart, Nilsson & Raphael; refined by Khemani):

Lemma L1 — Termination on Finite Graphs

Claim. On a finite graph, A* always terminates.

Proof. Each main-loop iteration moves one node from OPEN to CLOSED. With finitely many nodes, this can happen only finitely many times. Either A* finds a goal (success) or OPEN empties (correctly reporting no path). $\square$

Lemma L2 — An Optimal-Path Node Is Always on OPEN

Claim. If a path from S to a goal exists, then at any time before A* terminates, OPEN contains some node n' on an optimal path. Moreover, $f(n') \leq f^*(S)$.

Proof sketch. Let $P = (S, n_1, n_2, \dots, G)$ be optimal. Initially $S \in \text{OPEN}$. Whenever a node on P leaves OPEN (moves to CLOSED), its successor on P is generated. So the optimal path always has at least one “descendant” on OPEN.

For the f bound, let n' be that descendant. Then:

$$\begin{aligned}
 f(n') &= g(n') + h(n') \\
 &= g^*(n') + h(n') \quad (\text{because } n' \text{ is on the optimal path, } g(n') = g^*(n')) \\
 &\leq g^*(n') + h^*(n') \quad (\text{admissibility of } h) \\
 &= f^*(n') \\
 &= f^*(S) \quad (\text{since } n' \text{ is on an optimal path}). \square
 \end{aligned}$$

Lemma L3 — A* Terminates Even on Infinite Graphs (Given a Solution)

Claim. If a path to the goal exists, A* terminates — even on infinite graphs (provided finite branching and edge costs $\geq \varepsilon$).

Proof. Every step of A* either picks the goal or extends a partial path whose g -value grows by at least ε . With finite branching, there are only finitely many partial paths with $g(n) < f^*(S)$. After exploring them all in finite time, the cheapest unfinished partial path has $f(n) \geq f^*(S)$. By L2 a node n' on the optimal path also satisfies $f(n') \leq f^*(S)$. So A* picks an optimal-path node and eventually the goal. $\square$

Lemma L4 — A* Returns the Optimal-Cost Path (THE MAIN THEOREM)

Claim. When A* terminates with goal G , the cost of the returned path equals $f^*(S)$.

Proof (by contradiction). Suppose A* terminates by picking G' with $g(G') > f^*(S)$. At the moment A* was about to pick G' , by L2 there was some optimal-path node n' in OPEN with $f(n') \leq f^*(S) < g(G')$.

But A* picks the minimum- f node. Since $h(G') = 0$ (goal), $f(G') = g(G')$. So $f(n') < f(G')$ — A* would have picked n' , **contradiction**.

Hence $g(G') \leq f^*(S)$, and since $f^*(S)$ is by definition the minimum cost, $g(G') = f^*(S)$. $\square$

Lemma L5 — Every Node A* Expands Has $f(n) \leq f^*(S)$

Claim. For every node n that A* expands (not just the optimal-path nodes), $f(n) \leq f^*(S)$.

Proof. A* picks n in preference to any other node on OPEN, including the L2 node n' . So $f(n) \leq f(n') \leq f^*(S)$. $\square$

Lemma L6 — “More Informed Means Less Search”

Claim. Let A_1 and A_2 use admissible heuristics h_1, h_2 with $h_2(n) > h_1(n)$ for all non-goal n (i.e., h_2 is *strictly more informed*). Then every node expanded by A_2 is also expanded by A_1 :

$$\forall n \text{ (Expands}(A_2, n) \Rightarrow \text{Expands}(A_1, n))$$

Sketch of proof (by induction on depth d).

Basis ($d = 0$): both expand S .

Inductive step. Assume true for all nodes at depth $\leq d$. Consider a node n' at depth $d + 1$ expanded by A_2 . Then $f_2(n') = g(n') + h_2(n') \leq f^*(S)$ (L5 applied to A_2). Since $h_1(n') < h_2(n')$, we get $f_1(n') = g(n') + h_1(n') < f_2(n') \leq f^*(S)$. With strictly smaller f , A_1 would have selected n' to expand. $\square$

5.5.5 Practical Consequences**Remember This**

L6 is profound. It says that as long as your heuristic is *closer* to h^* pointwise, you do strictly less work. Designing a tight admissible heuristic is therefore the central engineering task in applying A*.

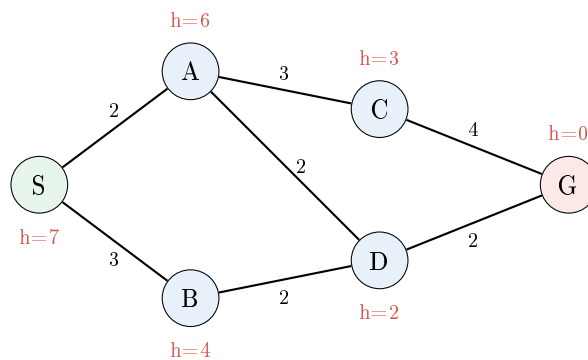
5.5.6 Example: Heuristic Dominance for 8-Puzzle

- h_1 = Hamming distance (tiles misplaced).
- h_2 = Manhattan distance.

For every state n in the 8-puzzle: $h_1(n) \leq h_2(n) \leq h^*(n)$. So Manhattan dominates Hamming; A* with Manhattan expands a strict subset of the nodes A* with Hamming expands.

5.6 A* in Action — Worked Example (Geographic Routing)

A small map. Edges are road costs; node labels show h = straight-line distance to goal G :



Trace.

Step	OPEN ($f = g + h$)	CLOSED
0	$(S, -, 0 + 7 = 7)$	—
1	$(A, S, 2 + 6 = 8), (B, S, 3 + 4 = 7)$	S
2	$(A, S, 8), (D, B, 5 + 2 = 7)$	B, S
3	$(A, S, 8), (G, D, 7 + 0 = 7)$	D, B, S
4	Goal G popped with $g(G) = 7$. Optimal path: $S \rightarrow B \rightarrow D \rightarrow G$, cost 7.	

Note A* correctly chose B over A at step 1 even though A has a slightly smaller h , because A 's g is smaller — f is what matters.

5.7 A* vs. Best-First vs. Branch and Bound

Algorithm	Cost function	Optimal?	Complete?
BFS	implicit (depth)	yes if unit cost	yes (finite graph)
Best-First	$h(n)$	no	yes
Branch & Bound / Dijkstra	$g(n)$	yes	yes
A* (admissible h)	$g(n) + h(n)$	yes	yes (above conditions)

Memory Trick

Mnemonic for $f = g + h$: “**G**round truth (so far) plus **H**ope (estimate).”

5.8 Designing Admissible Heuristics

5.8.1 Heuristics from Relaxed Problems

A reliable way to invent an admissible heuristic: **remove some constraints** from the original problem, and use the cost of the optimal solution to the easier problem as h . Since you’ve made the problem easier, the optimal cost can only *drop*, so $h \leq h^*$ automatically.

8-puzzle examples.

- *Original*: tiles move only to an adjacent empty cell.
- *Relaxation 1* — “*tiles can teleport.*” Optimal cost = Hamming distance (number of misplaced tiles).
- *Relaxation 2* — “*tiles can move to any adjacent cell, ignoring the blank.*” Optimal cost = Manhattan distance.

Both are guaranteed admissible because they solve relaxed problems.

5.8.2 Combining Heuristics

If you have multiple admissible heuristics $h_1, h_2, \dots, h_k$, then

$$h(n) = \max(h_1(n), h_2(n), \dots, h_k(n))$$

is also admissible *and dominates each component* (so by L6, it’s better).

5.8.3 Pattern Databases

For very hard problems (15-puzzle, Rubik’s cube), **pattern databases** pre-compute optimal costs for subproblems involving only a subset of pieces. The lookup-table approach yields very strong admissible heuristics. (Beyond Week 5’s scope, but a striking real-world application of L6.)

5.9 Common Mistakes

Common Mistake

1. Using an *overestimating* heuristic and expecting optimality. If $h(n) > h^*(n)$ even at one node, A* may return a suboptimal path.
2. Forgetting the Case-3 “reopen CLOSED” step. Without it, A* is not optimal for general admissible h .
3. Confusing h (estimate to goal) with g (cost so far).
4. Stopping A* the moment the goal is *generated* (added to OPEN) instead of when it is *expanded* (popped from OPEN). Stopping early can yield a non-optimal cost.
5. Using “negative cost edges.” A* (and Dijkstra) don’t handle them; you need Bellman–Ford.
6. Calling Best-First Search “A*” — they are different.

5.10 Real-World Applications of A*

Real-World Application

Game pathfinding (every commercial game from *Age of Empires* to *The Witcher* uses A* or a variant). **GPS navigation systems** (with carefully designed straight-line heuristics). **Robotics motion planning**. **Theorem proving** (proof search). **Network routing** (OSPF protocol). **Bioinformatics: sequence alignment** (Week 6). **Cleaning planners, factory schedulers**.

A* is so widely used that “A*-pathfinding” is essentially a synonym for “find a shortest path” in the games industry.

5.11 Quick Revision — Week 5

Quick Revision

- **Branch & Bound:** extend cheapest partial path; bound = lower bound on completion. The higher the bound, the more pruning.
- **Dijkstra’s algorithm:** B&B with relaxation; finds shortest paths from a single source on non-negative weighted graphs.
- **A* (Hart, Nilsson, Raphael, 1968):** $f(n) = g(n) + h(n)$, expand smallest f .
- Special cases: $h \equiv 0 \Rightarrow$ Dijkstra; $g \equiv 0 \Rightarrow$ Best-First.
- **Admissibility of h :** $h(n) \leq h^*(n)$ everywhere (never overestimates).
- **Admissibility of A*:** returns optimal-cost solution. Holds iff (a) finite branching, (b) edge costs $\geq \varepsilon > 0$, (c) h admissible.
- Six lemmas: L1 termination, L2 OPEN has optimal-path node with $f \leq f^*(S)$, L3 termination on infinite graphs, L4 optimal cost returned, L5 every expanded node has $f \leq f^*(S)$, L6 more-informed dominates.
- **L6: “More informed means less search”** — the central engineering motto for A*.
- Heuristic design tricks: relaxed-problem heuristics, max of admissible heuristics, pattern

databases.

- Case-3 reopen is needed in A* when h is admissible but not monotone (Week 6).

5.12 Most Important Quiz Points — Week 5

Exam Tip

1. A* expands the node with the smallest $f(n) = g(n) + h(n)$.
2. Admissible means $h(n) \leq h^*(n)$ for all n (never overestimates).
3. Admissible heuristic $\Rightarrow$ A* returns optimal cost (subject to finite branching, ε -cost edges).
4. $h \equiv 0$ in A* reduces to Dijkstra. $g \equiv 0$ reduces to Best-First.
5. Strictly more informed admissible heuristic $\Rightarrow$ strictly fewer expansions (Lemma L6).
6. A 50p/c trap: forgetting to reopen CLOSED nodes when finding a better path.
7. The Arvind Narayanan condition: edge costs $\geq \varepsilon > 0$, not just positive (else infinite finite-cost paths possible).
8. $h^*(n)$ itself is the perfect heuristic — leads to zero search, but is generally unknown.
9. Goal test on *pop*, not on *generation*: this preserves optimality.
10. Relaxed-problem heuristics are always admissible.

5.13 Practice Questions — Week 5

Conceptual.

1. Define admissible algorithm and admissible heuristic. Give an example of each.
2. Why does A* check Case 3 (better path to a CLOSED node)? Construct a small example illustrating the need.
3. Compare A* and Best-First Search. Why is one optimal and the other not?
4. Compare A* and Dijkstra. When are they equivalent?
5. State and prove Lemma L4 (A* finds the optimal-cost path).
6. State Lemma L6 in your own words. Why is it surprising?
7. Suppose $h_1(n) = h_2(n)$ for all n except one node v where $h_2(v) > h_1(v)$. Does A_2 expand strictly fewer nodes than A_1 ? Explain.
8. Why is the condition “edge costs $\geq \varepsilon$ ” rather than “positive” needed for A* to terminate on infinite graphs?
9. Show that the maximum of two admissible heuristics is itself admissible. Is the average?
10. Give two ways to construct an admissible heuristic for the 15-puzzle.

Numerical / algorithmic.

11. Trace A* on the worked geographic example with the values given.
12. Construct a small graph and an admissible h such that A* finds a path $S \rightarrow \dots \rightarrow G$ different from the path Best-First Search would return.

13. Construct an example where forgetting Case 3 causes A* to return a non-optimal path.
14. For the 8-puzzle state $\begin{smallmatrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & - & 5 \end{smallmatrix}$ relative to the standard goal, compute h_{Hamming} and $h_{\text{Manhattan}}$.

MCQ-style.

15. Which is NOT required for A* to be admissible?
 (a) admissible h (b) edge costs $\geq \varepsilon$ (c) monotone h (d) finite branching factor.
Answer: (c). Monotonicity is sufficient but not necessary.
16. A* with $h \equiv 0$ behaves like:
 (a) DFS (b) BFS (c) Dijkstra (d) Best-First.
Answer: (c).
17. If $h_1(n) \geq h_2(n)$ for all n and both are admissible, then A_1 relative to A_2 :
 (a) expands more nodes (b) expands the same nodes (c) expands fewer nodes (d) cannot be compared.
Answer: (c) — by Lemma L6.
18. Goal test in A* should be done:
 (a) when a node is generated (b) when a node is popped (c) when a node is on CLOSED (d) never.
Answer: (b) — ensures optimality.
19. In the function $f(n) = g(n) + h(n)$, $g(n)$ represents:
 (a) heuristic estimate to goal (b) known cost so far (c) average path cost (d) random number.
Answer: (b).

End of Week 5.

6 Week 6: The Monotone Condition, Space-Saving A*, and Sequence Alignment

“As the weight of $h(n)$ increases, search requires less space — but eventually becomes inadmissible.”

— The fundamental trade-off

Goal of Week 6. Last week we proved that A* is admissible (always finds an optimal path) under three mild conditions. But A* has a darker side: it can keep *enormous* OPEN and CLOSED lists. This week we (i) prove a stronger property of heuristics — the **monotone (consistency) condition** — that makes the algorithm Dijkstra-like and lets us simplify Case 3 of A*, (ii) meet a beautiful motivating problem from biology — **Sequence Alignment** — where the search space blows up quadratically with input length, (iii) study a family of **space-saving variants of A*** (Frontier Search, SMGS, Breadth-First Heuristic Search, Beam Search, Beam Stack Search, Divide-and-Conquer BSS) that progressively trade away memory for time.

6.1 Recap: the Three Conditions for A*'s Admissibility

A* is admissible (finds the optimal cost path) when:

1. **Finite branching factor.** Each node has finitely many neighbours.
2. **Edge cost lower bound.** Every edge cost $\geq \varepsilon > 0$.
3. **Heuristic admissibility.** $h(n) \leq h^*(n)$ for every node n , where $h^*(n)$ is the true cost of the cheapest path from n to a goal.

Remember This

Admissibility says *the heuristic is optimistic*. It never claims a node is worse than it really is. We saw in Week 5 that under these conditions, when A* picks the goal, $g(\text{goal}) = g^*(\text{goal})$. But on the way, A* may revisit nodes in CLOSED with better paths (Case 3). This makes the algorithm more complex than it needs to be.

6.2 The Monotone (Consistency) Condition

Definition

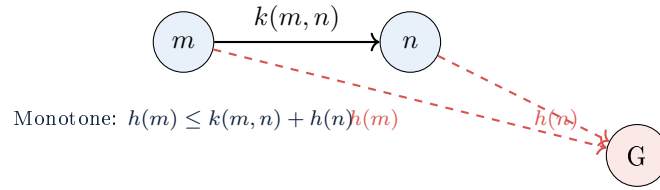
Monotone / Consistency Property. A heuristic h is **monotone** (also called **consistent**) if, for every edge (m, n) with cost $k(m, n)$,

$$h(m) - h(n) \leq k(m, n).$$

Equivalently, $h(m) \leq k(m, n) + h(n)$ — the heuristic estimate from m is at most one edge step plus the estimate from n .

6.2.1 The Triangle-Inequality Interpretation

Think of h as an estimated distance to the goal. The monotone condition says *the heuristic itself obeys a triangle inequality* along every edge: stepping from m to n cannot reduce the heuristic estimate by more than the cost of the step. The heuristic “underestimates the cost of each individual edge” in addition to the global optimistic bound.



6.2.2 Example: a Monotone Heuristic

In a 2-D map where edge costs are road distances and h is the Manhattan (or Euclidean) distance to the goal, monotonicity holds automatically: the change in h across one road segment is bounded by the segment's length. Khemani's example graph in the lectures uses Manhattan distance with edges of cost $\geq$ Manhattan distance — so for instance the edge $H \rightarrow O$ has cost 23 while $|h(H) - h(O)| = |120 - 100| = 20 \leq 23$.

6.3 Consequence 1: Non-decreasing f -Values

Theorem (monotonicity $\Rightarrow$ non-decreasing f)

If h is monotone, then along any path $S = n_0 \rightarrow n_1 \rightarrow n_2 \rightarrow \dots$ generated by A*, the f -values are **non-decreasing**:

$$f(n_0) \leq f(n_1) \leq f(n_2) \leq \dots$$

Proof. Let n be a successor of m via the edge of cost $k(m, n)$. Then $g(n) = g(m) + k(m, n)$, so

$$\begin{aligned} f(n) &= g(n) + h(n) \\ &= g(m) + k(m, n) + h(n) \\ &\geq g(m) + h(m) \quad (\text{by monotonicity: } k(m, n) \geq h(m) - h(n)) \\ &= f(m). \end{aligned} \tag{3}$$

□

Intuition

With a monotone heuristic, f -values climb (or stay flat) as we descend the search tree. A* is therefore *also* exploring nodes in order of increasing f — it never has to “go back” to revise an estimate.

6.4 Consequence 2: A* Finds Optimal Paths to *Every* Expanded Node

Theorem (the “never look back” theorem)

When A* uses a monotone heuristic, then **the first time** A* picks any node n from OPEN, it has already found the optimal cost path to n :

$$g(n) = g^*(n) \quad \text{when } n \text{ is moved from OPEN to CLOSED.}$$

6.4.1 Proof Sketch (by contradiction)

Suppose A* is about to expand a node n with $g(n) > g^*(n)$ — i.e. A* has found a sub-optimal path to n . Then there must exist a **cheaper** path $S \rightarrow n_1 \rightarrow n_2 \rightarrow \dots \rightarrow n_\ell \rightarrow n_{\ell+1} \rightarrow \dots \rightarrow n$ that achieves cost $g^*(n)$. Let n_ℓ be the last node on this optimal path that has been *expanded*

so far. Then $n_{\ell+1}$ must currently be on OPEN with $g(n_{\ell+1}) = g^*(n_{\ell+1})$ (since A* found n_ℓ optimally and propagated cost optimally to $n_{\ell+1}$).

By the first consequence (non-decreasing f), and since $n_{\ell+1}$ lies on an optimal path to n ,

$$f(n_{\ell+1}) \leq f^*(n) = g^*(n) + h(n).$$

But A* picked n before $n_{\ell+1}$, so $f(n) \leq f(n_{\ell+1})$, i.e.

$$g(n) + h(n) \leq g^*(n) + h(n) \implies g(n) \leq g^*(n).$$

Combined with $g(n) \geq g^*(n)$ (the optimal is by definition the minimum), we get $g(n) = g^*(n)$ — a contradiction with our assumption. $\square$

6.4.2 What this Buys Us: Case 3 of A* Disappears

Recall A*'s three cases when handling a child n' of the expanded node n :

1. **Case 1:** n' is new — add to OPEN.
2. **Case 2:** n' is on OPEN with a worse g — update its g and parent.
3. **Case 3:** n' is on CLOSED with a worse g — *this is the painful case*: reopen n' , update its descendants' g -values, propagate changes through the search tree.

Remember This

Under monotonicity, Case 3 cannot happen. Once a node is on CLOSED, its g -value is already optimal — no later path can improve it. The algorithm is identical in spirit to **Dijkstra's algorithm**, which is what A* with $h \equiv 0$ degenerates to.

6.4.3 Consistency Implies Admissibility

Lemma (consistency $\Rightarrow$ admissibility)

If h is monotone (consistent), then h is admissible. The converse is *not* true — admissible heuristics need not be monotone.

Proof. Pick any node n , and let $n \rightarrow n_1 \rightarrow \dots \rightarrow n_k = G$ be the optimal path of cost $h^*(n)$. By monotonicity along each edge:

$$h(n) - h(n_1) \leq k(n, n_1), \quad h(n_1) - h(n_2) \leq k(n_1, n_2), \dots$$

Telescoping these and using $h(G) = 0$:

$$h(n) \leq \sum k(\cdot) = h^*(n). \quad \square$$

$\square$

Exam Tip

A favourite exam question: “Is consistency stronger than admissibility?” **Yes.** Consistent $\Rightarrow$ admissible. But *admissible* does not imply *consistent*: you can craft heuristics that underestimate globally but violate the local triangle-inequality on some edges.

6.5 Motivating Problem: Sequence Alignment in Biology

Why do we want to prune CLOSED? Because for some real-world problems, **CLOSED grows faster than OPEN**. The canonical example is sequence alignment.

6.5.1 The Biology

DNA is a long linear polymer made of four nucleotides, denoted by the letters $\{A, C, G, T\}$ (adenine, cytosine, guanine, thymine). RNA uses U (uracil) in place of T . The **sequence** of nucleotides encodes the information that directs every function of a living organism.

In bioinformatics, biologists routinely need to know how *similar* two DNA or protein sequences are. Similar sequences across species often signal evolutionary relationships; similar sequences within an organism may indicate shared function.

6.5.2 The Sequence Alignment Problem

Definition

Given two strings $X = x_1x_2\dots x_m$ and $Y = y_1y_2\dots y_n$ over an alphabet Σ (e.g. $\{A, C, G, T\}$), find an **alignment** that places the two strings side-by-side, possibly inserting *gaps* (denoted “—”) into either string, so as to maximise similarity (equivalently, minimise total penalty). Two kinds of penalty are charged:

- **Mismatch penalty:** when x_i is aligned with y_j and $x_i \neq y_j$.
- **Indel penalty:** when a character is aligned with a gap (“insertion or deletion”).

Example. Align $X = CG$ with $Y = CA$. Suppose indel = 3, mismatch = 7.

- Direct alignment $\begin{pmatrix} C & G \\ C & A \end{pmatrix}$ pays 7 (one mismatch).
- Gap-inserted alignment $\begin{pmatrix} C & G & - \\ C & - & A \end{pmatrix}$ pays $3 + 3 = 6$ (two indels).

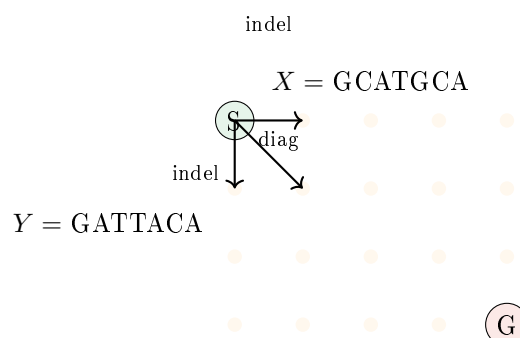
So with these costs, gap insertion is cheaper.

If we instead set mismatch = 5, the direct alignment is cheaper. **The optimal alignment depends on the penalty structure**, which biologists choose based on biological evidence (mismatch matrices, gap-opening vs. gap-extending costs, etc.).

6.5.3 Alignment as a Graph Search

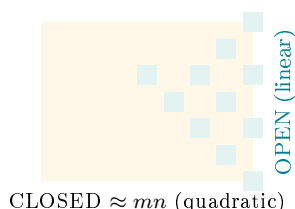
Model the problem on an $(m+1) \times (n+1)$ grid:

- **Nodes:** cells (i, j) representing “ i characters of X and j characters of Y have been processed.”
- **Start:** $(0, 0)$; **Goal:** (m, n) .
- **Moves at (i, j) :**
 - *Diagonal* to $(i+1, j+1)$: align x_{i+1} with y_{j+1} (mismatch cost or zero).
 - *Horizontal* to $(i, j+1)$: insert gap in X (indel cost).
 - *Vertical* to $(i+1, j)$: insert gap in Y (indel cost).



Any path from start to goal represents a valid alignment. The number of nodes is $(m+1)(n+1) \approx mn$ — grows **quadratically**. The number of paths grows combinatorially: the number of pure horizontal/vertical paths alone is $\binom{m+n}{m}$, and including diagonals it explodes further.

6.5.4 Why CLOSED Grows Quadratically But OPEN Linearly



A* sweeping the alignment grid produces an OPEN list that runs along an anti-diagonal “wavefront” of width $\sim m+n$ — *linear* in input size. But the CLOSED region behind it covers a quadratic area $\sim mn$.

Remember This

For DNA sequences of length 10^5 , $mn = 10^{10}$ — **ten gigabytes of CLOSED at a single byte per node**. This is why we need to *prune CLOSED*.

6.6 The Two Roles of CLOSED

To understand how to remove CLOSED, first remember *why* we kept it:

1. **Loop prevention.** When we expand n and generate a child n' already in CLOSED, we discard n' to avoid revisiting it. Under monotonicity (the key property of this week), *nothing useful comes from revisiting* — CLOSED is just a stop-the-leak mechanism.
2. **Path reconstruction.** When the goal pops, we trace parent pointers (stored in CLOSED) back to the start.

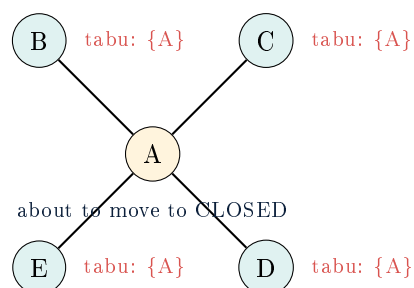
The space-saving algorithms we will now study address these two roles separately:

- **Frontier Search (Korf & Zhang, 2000)** solves leak prevention by storing per-node “tabu lists” inside OPEN, then reconstructs the path with **relay nodes** and recursion.
- **Smart Memory Graph Search (SMGS, Zhou & Hansen, 2003)** is adaptive: it keeps full CLOSED while memory allows, then converts to a sparse relay representation when memory runs out.

6.7 Frontier Search (Korf & Zhang, 2000)

6.7.1 The Idea: Replace CLOSED with Per-Node Tabu Lists

When A* expands node A , generate A ’s neighbours $\{B, C, D, E\}$ and decide which ones to put on OPEN. Frontier Search adds one extra step: for every neighbour X added to OPEN, **record A in X ’s personal tabu list** — meaning “when X is eventually expanded, do not regenerate A .”



Then we *forget* A . When some other node, say E , is later expanded and generates its children, those children check E 's tabu list — which now includes $\{A, D, H, I, \dots\}$, the neighbours of E — and refuse to regenerate any of them. The search cannot “leak back” even though we no longer have a CLOSED list.

Intuition

Each OPEN node carries a small backpack of names it must not regenerate. CLOSED has been deconstructed and *distributed* into these backpacks.

6.7.2 Reconstruction via Relay Nodes (Divide-and-Conquer Frontier Search)

But we've destroyed our path-reconstruction information! Korf & Zhang's fix: at strategic intermediate depths — typically when $g(n) \approx h(n)$ (roughly half-way through the search) — pick certain OPEN nodes and mark them as **relay nodes**, retaining them in memory. Every subsequent OPEN node also remembers a pointer to “which relay node it descended from.”

When the goal is found, we know the relay node that gave rise to it. So we have one “check-point” on the path. To get the actual path, we now run **two recursive Frontier Searches**: from start to relay, and from relay to goal.

DCFS time complexity

Let $T(d)$ be the time to solve one A* of depth d . DCFS recursively splits, giving

$$T_{\text{DCFS}}(d) = T(d) + 2T(d/2) + 4T(d/4) + \dots$$

If T is exponential in d , then $T_{\text{DCFS}}(d) \approx d \cdot T(d)$ — a **factor- d time penalty for huge memory savings**.

6.8 Smart Memory Graph Search (SMGS, Zhou & Hansen, 2003)

6.8.1 Why SMGS Improves on DCFS

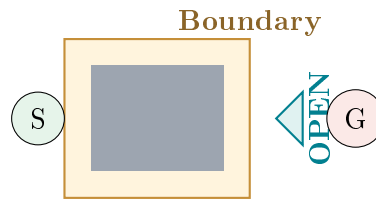
Frontier Search always pays the divide-and-conquer cost, even when memory is plentiful. SMGS asks: **why not just run A* with full CLOSED while memory allows, and only switch to the sparse representation when memory runs out?** Memory is getting cheaper; let's be smart about using it.

6.8.2 Kernel and Boundary

SMGS distinguishes two kinds of CLOSED node:

- **Boundary nodes:** a CLOSED node that has at least one child on OPEN. These are the nodes “touching” the frontier.

- **Kernel nodes:** a CLOSED node *all* of whose neighbours are also CLOSED. These nodes are fully enclosed by the frontier.



Crucial observation: To prevent leak-back, we only need the **boundary** of CLOSED, not the kernel. A child of a frontier node cannot regenerate a kernel node because every path from the frontier into the kernel must first cross the boundary.

6.8.3 The SMGS Algorithm in One Paragraph

Run A* with full CLOSED. When memory pressure crosses a threshold, invoke a **PruneClosed** routine: **promote all current boundary nodes to relay nodes, then delete all kernel nodes**. Continue A* with the (now sparse) CLOSED. If pressure rises again, prune again — accumulating more relay layers. At termination, the relay layers form a *sparse solution path* that can be densified with recursive calls.

Remember This

SMGS is adaptive — it pays the divide-and-conquer overhead only when forced to. On small problems it behaves exactly like A*; on huge problems it gracefully degrades.

6.9 Pruning OPEN — Weighted A* and Beam Search

So far we've attacked CLOSED. But OPEN grows exponentially for general (tree-shaped) problems, so we need to prune OPEN as well.

6.9.1 Recap: Weighted A* (wA*)

In Week 5 we saw that A* uses $f(n) = g(n) + h(n)$. **Weighted A*** (Pohl, 1970) generalises this to

$$f(n) = g(n) + w \cdot h(n), \quad w \geq 1.$$

As w increases, the search becomes more greedy:

Algorithm	Nodes	Cost	Comment
Branch & Bound ($w = 0$)	23	148	exact, explores most
A* ($w = 1$)	14	148	optimal, fewer nodes
wA* ($w > 1$)	9	153	inadmissible, much smaller frontier
Best-First ($w = \infty$)	8	195	ignores g , can be terrible

wA* explicitly trades optimality for memory and time.

6.9.2 Beam Search Revisited (now in the Optimal-Path Setting)

Recall Beam Search from Week 3 (used as a Hill-Climbing variant on h). Here we re-purpose it for finding an optimal path, ordering by f instead of h :

Algorithm 20 Beam Search for optimal paths (beam width w)

```

1: beam  $\leftarrow$  {start}
2: while no goal in beam do
3:   candidates  $\leftarrow \bigcup_{n \in \text{beam}} \text{MOVEGEN}(n)$ 
4:   beam  $\leftarrow$  the  $w$  candidates with lowest  $f$ -value (even if not better)
5: end while
6: return path to goal

```

Key change from Week 3: since f -values *increase* with depth (under monotonicity), the “only if better” termination no longer makes sense. We must keep the w best successors even if they all have worse f than the parent — otherwise we never reach the goal. Termination is goal-found instead.

Beam Search trade-off

Total space = $w \times d$ (linear in depth, constant in width). **Will find a path**, but not necessarily the optimal one — *inadmissible*. The cost of the path it finds is, however, a useful **upper bound** U on the optimal cost.

6.10 Breadth-First Heuristic Search (BFHS, Zhou & Hansen 2004)**6.10.1 The Upper-Bound Trick**

If we have a known upper bound U on the optimal cost (perhaps obtained by running Beam Search first), then *any node n with $f(n) > U$ cannot be on the optimal path*. Why? Because f -values are non-decreasing along any path (monotonicity), so all of n ’s descendants have $f \geq f(n) > U$.

Intuition

U is a “ceiling.” Any node above the ceiling is pruned. The smaller U is, the more we prune. *Beam Search gives us a cheap ceiling.*

6.10.2 The BFHS Algorithm

Run **Breadth-First Search** (which has nothing to do with the goal!), but restrict yourself to nodes with $f(n) \leq U$.

Algorithm 21 Breadth-First Heuristic Search

```

1: Find any feasible solution path; let its cost be  $U$ 
2: OPEN  $\leftarrow$  {start}; CLOSED  $\leftarrow \emptyset$ 
3: while OPEN  $\neq \emptyset$  do
4:   expand all nodes in OPEN level-by-level
5:   for each child  $n'$  generated: if  $f(n') > U$ , discard; else add to next-level OPEN
6:   if goal generated then update  $U$  to cost of new path and continue
7:   end if
8: end while
9: return best path found

```

Remember This

BFHS sounds like a contradiction in terms — “Breadth-First *Heuristic* Search.” It is a blind BFS *wrapped in a heuristic-based pruning rule*. Zhou & Hansen showed empirically that BFHS often expands a *smaller* frontier than plain A* on graph-like problems, because BFS naturally avoids the priority-queue overhead and zigzag exploration that A* sometimes does.

6.11 Beam Stack Search (Zhou & Hansen, 2005)**6.11.1 The Problem with Plain Beam Search**

Beam Search drops nodes that don’t make the cut, never to be reconsidered. If the optimal path happens to require a node that fell off the beam at some level, beam search will miss it. The result is *inadmissible*.

6.11.2 The Fix: Beam + Explicit Backtracking

Beam Stack Search (BSS) retains beam search’s small frontier but adds an *explicit backtracking mechanism*. The trick is a clever data structure: the **beam stack**.

Definition

The **beam stack** is a stack of pairs $[f_{\min}, f_{\max}]$, one entry per level of the search tree.

- $f_{\min}$ = smallest f -value in the current beam at that level.
- $f_{\max}$ = smallest f -value *not* in the beam (i.e. the next candidate to consider on backtrack).

When the algorithm finds a path of cost U and wants to verify optimality, it backtracks: pop the top of the beam stack, slide the window to $[f_{\max}, \dots)$ — i.e. re-explore the level with the *next-cheapest* batch of children, and so on.

$f_{\min}=3, f_{\max}=7$ (top of stack)

beam 3 4 5 7 9 12 15

On backtrack, slide: $f_{\min} \leftarrow 7, f_{\max} \leftarrow 12$

The beam stack provides *just enough memory* to regenerate the correct children at every level. **BSS sweeps the entire feasible region with f -values $\leq U$** , like a controlled, monotonically-advancing wavefront.

6.11.3 Divide-and-Conquer Beam Stack Search

DCBSS combines the BSS backtracking idea with relay-layer divide-and-conquer:

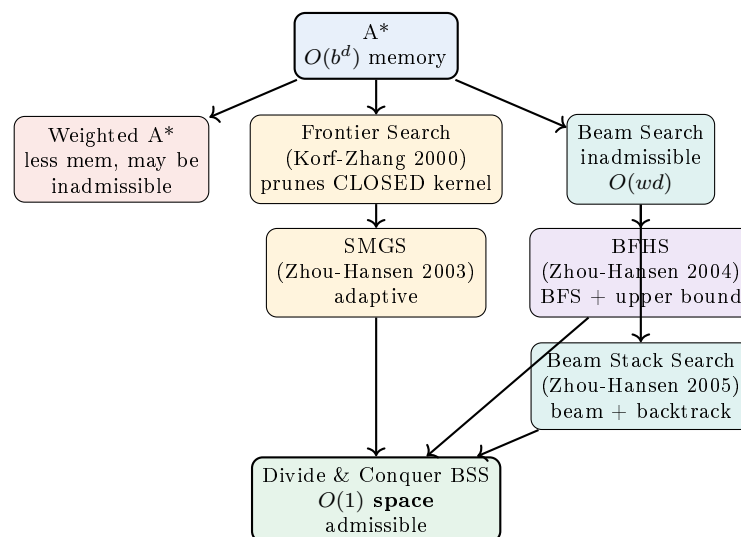
- Keep only **three layers** of constant width w : open, boundary, and one relay layer.
- On backtracking, **regenerate from the start** (guided by the beam stack).
- Space: $O(1)$ in the depth, ignoring the beam stack itself which is $O(d)$.

DCBSS = ultimate space-saving A*

We've come full circle:

- Plain A*: $O(b^d)$ memory.
- Frontier Search: deletes kernel of CLOSED.
- Beam Search: linear $O(wd)$, but inadmissible.
- BFHS: linear with admissibility.
- BSS: linear with admissibility, sweeps the bounded region.
- **DCBSS**: $O(1)$ memory (plus beam stack), admissible.

The cost? More time, due to the regeneration of nodes during backtracking.

6.12 The Full Family Tree of Space-Saving A*

6.13 Comparison: When to Use Which Variant

Algorithm	Space	Admissible?	Best for
A*	$O(b^d)$	Yes (if h admissible)	Standard, small problems
Weighted A*	similar to A*	No ($\text{cost} \leq w \cdot \text{opt}$)	When “good enough” is OK
Frontier Search	deletes kernel	Yes (with monotone h)	Sequence alignment, graphs with quadratic CLOSED
SMGS	adaptive	Yes	Mixed regimes; “best of both”
Beam Search	$O(wd)$	No	Quick upper bound
BFHS	frontier + upper bound	Yes	When BFS frontier < A* frontier
BSS	frontier + beam stack	Yes	Tight memory, want optimality
DCBSS	$O(1)$ + beam stack	Yes	Massive problems, tight memory

6.14 Common Mistakes

Common Mistake

1. Confusing **admissibility** with **consistency**. Consistency is strictly stronger.
2. Claiming Case 3 of A* never matters. It vanishes only under consistency. Under plain admissibility you may genuinely need to reopen CLOSED.
3. Beam Search with the Week-3 “only if better” termination — this fails for f since f only ever grows.
4. Using Frontier Search without storing the per-node tabu lists — the search will leak back into the kernel and find sub-optimal solutions.
5. Picking too small a beam width: the optimal path may not lie in the beam, and inadmissible algorithms have no way to know.
6. Forgetting that DCBSS regenerates from the start on backtrack — the time cost is hidden but real.

6.15 Real-World Applications

Real-World Application

Sequence Alignment. Tools like BLAST, FASTA, and Needleman–Wunsch underpin essentially every comparative genomics analysis. DNA sequences in the millions of bases use BFHS-style algorithms with banded heuristics.

Robotic Planning. For massive state spaces (e.g. 24-puzzle, Rubik’s cube), IDA*, frontier search and SMGS-style methods are practical only with consistent heuristics.

Speech Recognition / NLP. Beam Search is the workhorse of speech decoding and neural machine translation, where memory and latency budgets are tight and locally suboptimal but globally good paths are acceptable.

Game AI. Beam search variants and bounded-suboptimal heuristics are used in real-time domains like StarCraft AI, where a 100 ms move latency rules out full A*.

6.16 Quick Revision — Week 6

Quick Revision

- **Monotone condition:** $h(m) - h(n) \leq k(m, n)$ for every edge. Equivalently, $h(m) \leq k(m, n) + h(n)$ (heuristic triangle inequality).
- **Consistency $\Rightarrow$ Admissibility** (but not vice versa).
- **Two consequences** of monotonicity: (1) f -values are non-decreasing along any path; (2) A* finds optimal paths to every CLOSED node, not just the goal. Case 3 of A* therefore vanishes.
- **Sequence Alignment** models alignment of two strings as a grid search problem; CLOSED grows quadratically with input length, OPEN linearly — motivating space-saving.
- **Two roles of CLOSED:** loop prevention and path reconstruction.
- **Frontier Search (Korf-Zhang, 2000):** replace CLOSED with per-node tabu lists; use *relay nodes* + Divide-and-Conquer for path reconstruction.
- **SMGS (Zhou-Hansen, 2003):** keep full CLOSED while memory allows; on memory pressure, prune kernel and promote boundary to relay nodes. Adaptive.
- **Pruning OPEN:** use an upper bound U on cost (e.g. from Beam Search) to prune nodes with $f(n) > U$.
- **Breadth-First Heuristic Search:** blind BFS bounded by U — often smaller frontier than A*.
- **Beam Search:** keep top w by f at each level. Inadmissible but linear-space.
- **Beam Stack Search:** beam search with explicit backtracking via a stack of $[f_{\min}, f_{\max})$ pairs. Admissible, linear space.
- **DCBSS:** BSS + three constant-width layers = $O(1)$ space (modulo beam stack), admissible.

6.17 Most Important Quiz Points — Week 6

Exam Tip

1. **State the monotone condition** in both forms: $h(m) - h(n) \leq k(m, n)$ and $h(m) \leq k(m, n) + h(n)$.
2. **Consistency $\Rightarrow$ admissibility**, by telescoping along the optimal path.
3. **Under monotonicity**, f -values are non-decreasing — and Case 3 of A* never fires.
4. In sequence alignment, the search grid is $(m+1) \times (n+1)$, with three moves (diago-

nal=align, horizontal=gap in X , vertical=gap in Y).

5. CLOSED in sequence alignment grows as $O(mn)$ (quadratic), while OPEN is only $O(m+n)$ — *this is why we prune CLOSED*.
6. **Frontier Search** stores per-node tabu lists; relay nodes enable path reconstruction via two recursive calls.
7. **Time penalty** of Divide-and-Conquer Frontier Search is roughly a factor of d .
8. **Kernel vs Boundary**: a CLOSED node is on the *boundary* iff it has at least one OPEN neighbour; otherwise it's in the *kernel*. Only the boundary is needed to prevent leak-back.
9. **Beam Search** ordered by f keeps top w even if worse; inadmissible. Use f -values, not h .
10. Upper bound U allows pruning any node with $f(n) > U$ because f -values only grow.
11. DCBSS achieves $O(1)$ space — the smallest among all admissible variants.

6.18 Practice Questions — Week 6

Conceptual.

1. State and explain the monotone (consistency) condition. How is it related to the triangle inequality?
2. Prove: consistency implies admissibility. Give a counter-example to the converse.
3. Show that under a consistent heuristic, f -values are non-decreasing along any path.
4. Show that under a consistent heuristic, A* never has to reopen a CLOSED node (Case 3 vanishes).
5. Explain the two roles of CLOSED in A*. Which one does Frontier Search address, and how?
6. Describe how Frontier Search uses per-node “tabu lists” to prevent the search from leaking back.
7. What is a relay node? Why is it placed roughly where $g(n) \approx h(n)$?
8. State the recurrence for DCFS time and explain why the overhead is $\sim d$ for exponential-time problems.
9. Distinguish kernel and boundary in SMGS. Why is only the boundary needed?
10. Why does Beam Search ordered by f require “keep w best even if worse” as its rule, unlike Hill-Climbing-style Beam Search?

Sequence alignment.

11. Model the alignment of $X = \text{GCAT}$ and $Y = \text{GAT}$ as a search problem. Draw the search grid, mark start and goal, and label the three move types.
12. Given indel = 2 and mismatch = 1, find the optimal alignment of $X = \text{ACGT}$ and $Y = \text{AGCT}$. Show all costs.
13. Prove that for sequence alignment, CLOSED grows as $O(mn)$ while OPEN grows as $O(m+n)$.
14. Compute the number of distinct alignments (paths in the grid) of two strings of length 3 each. (Include diagonal moves.)

Algorithmic.

15. Write the pseudocode for Beam Search ordered by f , with width w , terminating on goal.
16. Write the pseudocode for BFHS using an upper bound U .
17. Trace Beam Stack Search on a simple tree (5 levels, branching factor 3, beam width 2) and show the beam stack at each step.
18. Suppose A* with consistent h expands 1000 nodes on a problem. Estimate (qualitatively) how many nodes BFHS would expand if U equals the optimal cost.

MCQ-style.

19. Which of the following is the monotone condition?
 (a) $h(n) \leq h^*(n)$ (b) $h(m) - h(n) \leq k(m, n)$ (c) $h(\text{goal}) = 0$ (d) f is constant.
Answer: (b).
20. Consistency implies:
 (a) inadmissibility (b) admissibility (c) constant f (d) zero h .
Answer: (b).
21. Under monotonicity, f -values along any path are:
 (a) decreasing (b) non-decreasing (c) constant (d) unpredictable.
Answer: (b).
22. In sequence alignment with strings of length m, n , the search grid has:
 (a) mn nodes (b) $m + n$ nodes (c) $(m + 1)(n + 1)$ nodes (d) 2^{m+n} nodes.
Answer: (c).
23. In Frontier Search, what replaces the CLOSED list?
 (a) per-node tabu lists (b) a hash table (c) the OPEN list (d) nothing — it's the same.
Answer: (a).
24. Beam Search ordered by f is:
 (a) admissible (b) inadmissible (c) complete only on trees (d) optimal.
Answer: (b).
25. DCBSS's space complexity (ignoring the beam stack) is:
 (a) $O(b^d)$ (b) $O(bd)$ (c) $O(d)$ (d) $O(1)$.
Answer: (d).
26. In SMGS, a CLOSED node is in the kernel if:
 (a) it has at least one OPEN neighbour (b) all its neighbours are CLOSED (c) it is on the optimal path (d) $f(n) = g(n)$.
Answer: (b).

End of Week 6.

7 Week 7: Game Playing – Minimax, Alpha-Beta, and SSS*

“A game is a model of an interactive situation among rational players. The key to game theory is that one player’s payoff is contingent on the strategy of the other.”

— The foundational idea

Goal of Week 7. For six weeks we have assumed the world is a passive arena that we move through alone (Assumption 3 from Week 1: “one agent only”). This week we relax that assumption: **the world contains an opponent**. Every move you make will be answered by an adversary who is trying to maximise their own gain — which, in zero-sum games, means minimising yours. We will build the foundational algorithms of adversarial search: **Minimax**, **Alpha-Beta pruning**, and a clever best-first alternative called **SSS***. We will also touch on game theory, types of games, the prisoner’s dilemma, evaluation functions, and the horizon effect.

7.1 A First Look at Game Theory

Definition

Game Theory. The branch of mathematics concerned with the analysis of strategies for dealing with competitive situations, where the outcome of a participant’s choice of action depends critically on the choices of *other* participants.

Pioneered by mathematicians **John von Neumann** and **John Nash** (Nobel laureate, 1994) and economist **Oskar Morgenstern**, game theory has been applied to war, business, biology, and politics. It is the *theory* behind multi-agent AI.

7.1.1 Nash Equilibrium

Nash Equilibrium

An outcome of a game is a **Nash equilibrium** if no player can unilaterally change their strategy and improve their payoff. Equivalently: “once everyone has decided, no one has regrets.”

7.1.2 The Prisoner’s Dilemma

The classic illustration (Flood & Dresher, RAND, 1950; named by Tucker):

Two criminals are arrested. Each can either **Cooperate** (stay silent) or **Defect** (betray the other). Sentences:

- Both Cooperate: each gets 1 year.
- Both Defect: each gets 2 years.
- One Defects, one Cooperates: the defector goes free; the silent one gets 3 years.

Payoff matrix (negative = years in prison):

	B: Cooperate	B: Defect
A: Cooperate	(−1, −1)	(−3, 0)
A: Defect	(0, −3)	(−2, −2)

The paradox. Whichever choice B makes, A is individually better off defecting. By symmetry, B should defect too. So both defect — getting 2 years each — even though both cooperating would have been better. This is a Nash equilibrium that is collectively *worse* than another outcome.

Intuition

The prisoner's dilemma generalises: "Rational self-interest can be self-defeating." It models situations from arms races to climate-change negotiations to evolutionary biology. The general payoff structure is $T > R > P > S$ (Temptation, Reward, Punishment, Sucker).

7.2 Classifying Games

7.2.1 By Payoff

- **Zero-sum:** total payoff is zero — one's gain is the other's loss. (*Chess: someone wins, someone loses, or draw.*)
- **Positive-sum:** total payoff is positive — cooperation possible.
- **Negative-sum:** total is negative — most or all players lose. (*Price wars.*)

7.2.2 By Number of Players

- **Two-player:** chess, checkers, Go.
- **Multi-player:** poker (6+), Risk, real-time strategy.
- **Team games:** bridge (2 vs 2), football (11 vs 11). Often modelled as two-player with each player being a team.

7.2.3 By Information and Determinism

- **Perfect information:** all players see the full state. *Chess, Go.*
- **Imperfect information:** hidden state. *Poker (opponent's cards), war (enemy plans).*
- **Deterministic:** no randomness. *Chess.*
- **Stochastic:** dice rolls, card draws. *Backgammon, poker, Monopoly.*

Remember This

The course focuses on **two-player, zero-sum, perfect-information, deterministic, alternate-move games** — the cleanest setting. This covers chess, checkers, tic-tac-toe, Othello, Go. We do *not* cover poker, backgammon, or bridge in detail.

7.3 A Whirlwind History of Computer Game Playing

- **1770** — Wolfgang von Kempelen's **Mechanical Turk**: a chess-playing automaton (revealed to be a hoax with a human hidden inside).
- **1912** — Leonardo Torres y Quevedo builds a genuine machine that plays King + Rook vs. King endgames.

- **1950** — **Claude Shannon**: “Programming a Computer for Playing Chess.”
- **1951** — **Alan Turing** designs (on paper) the first program that could play a full game of chess.
- **1952** — **Arthur Samuel**’s Checkers program (IBM 701): the first self-learning game program. “Eventually beat its own creator.”
- **1956** — **John McCarthy** invents the alpha-beta search algorithm (independently credited to others).
- **1967** — **Mac Hack VI** (Greenblatt et al., MIT) introduces *transposition tables*; first program to beat a person in tournament play.
- **1968** — **David Levy** bets no computer will beat him in 10 years. He wins the bet in 1978.
- **1988** — **Deep Thought** shares first place at a major tournament, ahead of ex-World-Champion Tal.
- **1997** — **Deep Blue** (IBM) defeats World Champion Garry Kasparov, 3.5–2.5.
- **2016** — **AlphaGo** (DeepMind) defeats Go World Champion Lee Sedol.
- **2017** — **AlphaGo Zero**: learns from *zero* human games via self-play — beats AlphaGo comprehensively.
- **2019** — **Pluribus** (CMU) beats top professionals at six-player no-limit Texas Hold’em — the first AI to crack multi-player poker.

7.3.1 Why Chess Was a Big Deal: Size of the Game Tree

Shannon’s number

Chess: ~ 35 legal moves per position, average game length ~ 80 ply $\Rightarrow \sim 35^{80} = 10^{120}$ possible games. The number of *atoms in the observable universe* is about 10^{80} . *Chess is unimaginably larger.*

Even at 10^{12} leaf-evaluations per second per machine, with 10^{75} machines (one per particle in the universe), exploring the chess tree would take $\sim 10^{22}$ centuries. **Brute force is impossible.** We need (a) finite lookahead with evaluation functions, (b) aggressive pruning.

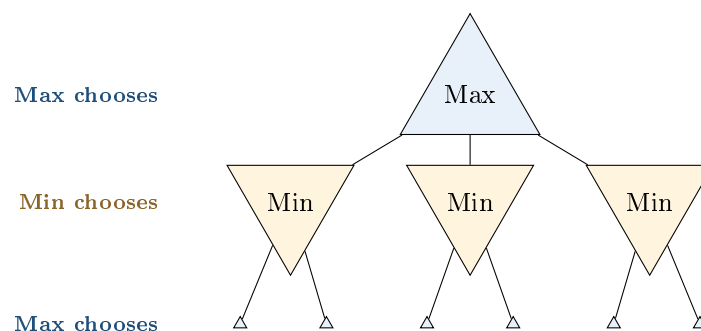
7.4 Board Games and Game Trees

Definition

A **game tree** is a layered tree whose root is the initial position, whose internal nodes are positions, and whose leaves are terminal positions. The levels alternate between two players:

- **Max** (us): wants to maximise the outcome. Drawn as $\triangle$ in textbooks.
- **Min** (opponent): wants to minimise it. Drawn as ∇ .

We always evaluate from Max’s perspective.



7.4.1 Leaves are Labelled with Outcomes

For tic-tac-toe and similar, leaf values can be $\{W, D, L\}$ (win, draw, loss for Max) — equivalently $\{+1, 0, -1\}$. For chess we typically use a much richer value (an *evaluation function*; see below) because we cannot reach actual terminal positions in lookahead.

7.5 The Minimax Algorithm

7.5.1 The Minimax Backup Rule

Minimax backup rule

At a leaf, the value is the leaf label. At an internal node N :

$$\text{Minimax}(N) = \begin{cases} \max_{c \in \text{children}(N)} \text{Minimax}(c) & \text{if } N \text{ is a Max node} \\ \min_{c \in \text{children}(N)} \text{Minimax}(c) & \text{if } N \text{ is a Min node} \end{cases}$$

Intuition. “If *I* move (Max), I’ll pick the best for me. If *they* move (Min), assume they pick the worst for me.”

Intuition

We *assume the opponent plays perfectly*. This is the **minimax assumption**. If the opponent plays badly, the actual outcome can only be better — so minimax is a **safe lower bound** on our final score. Good defensive design.

7.5.2 Strategies and the Value of a Game

Definition

A **strategy for Max** is a subtree of the game tree that picks one move at every Max node (the actual game played depends on Min’s response). The **value of a strategy** is the value Min would force if Max followed it — i.e. the *minimum* over leaves reachable through that strategy.

The **minimax value of the game** is the maximum over Max’s strategies of the value of each. Equivalently: the Minimax backup at the root.

7.5.3 Algorithm Minimax (recursive)

Algorithm 22 Minimax (Depth-First, Max's perspective)

```

1: function MINIMAX(node)
2:   if ISLEAF(node) then return EVAL(node)
3:   end if
4:   if node is Max then
5:     value  $\leftarrow -\infty$ 
6:     for child  $c$  of node do
7:       value  $\leftarrow \max(\text{value}, \text{MINIMAX}(c))$ 
8:     end for
9:   else ▷ Min node
10:    value  $\leftarrow +\infty$ 
11:    for child  $c$  of node do
12:      value  $\leftarrow \min(\text{value}, \text{MINIMAX}(c))$ 
13:    end for
14:  end if
15:  return value
16: end function

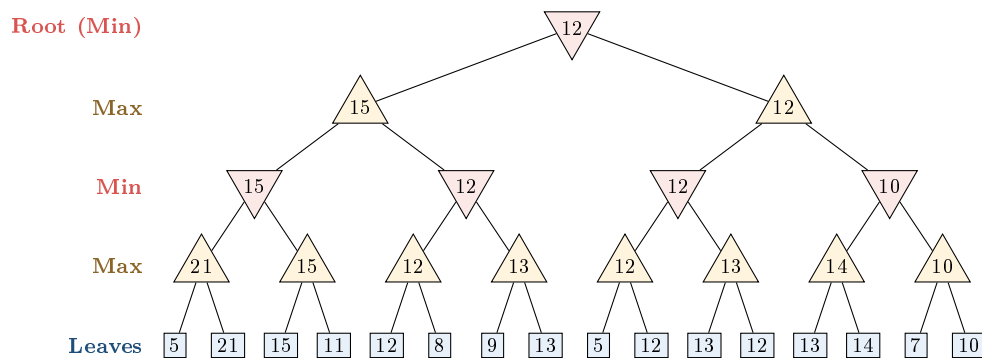
```

It's a **Depth-First Search**. Linear space ($O(bd)$). The whole tree must be inspected — $O(b^d)$ time. This is why we need pruning.

7.5.4 Worked Example

Consider a balanced binary tree of depth 4, with leaf values

5, 21, 15, 11, 12, 8, 9, 13, 5, 12, 13, 12, 13, 14, 7, 10.



Backup the values:

- Max nodes at depth 1: $\max(5, 21) = 21$, $\max(15, 11) = 15$, $\max(12, 8) = 12$, $\max(9, 13) = 13$, $\max(5, 12) = 12$, $\max(13, 12) = 13$, $\max(13, 14) = 14$, $\max(7, 10) = 10$.
- Min nodes at depth 2: $\min(21, 15) = 15$, $\min(12, 13) = 12$, $\min(12, 13) = 12$, $\min(14, 10) = 10$.
- Max nodes at depth 3: $\max(15, 12) = 15$, $\max(12, 10) = 12$.
- Root (Min): $\min(15, 12) = \boxed{12}$.

The **minimax value of the game is 12** — what perfect play by both yields.

7.6 Finite Lookahead and Evaluation Functions

Real games are too big to search to terminal nodes. **Finite lookahead**: search to a fixed depth k (called the *horizon*); estimate each horizon node with an **evaluation function** $h(N)$; back the values up using minimax.

Evaluation function h

$h : \text{Positions} \rightarrow [-\text{Large}, +\text{Large}]$. Positive values favour Max, negative favour Min. Typically a sum of weighted features:

- **Material strength**: piece values (king = 200, queen = 9, rook = 5, etc.)
- **Positional strength**: mobility, king safety, pawn structure, control of the centre, ...

7.6.1 Shannon's Chess Evaluation (1950)

$$\begin{aligned} f(p) = & 200(K - K') \\ & + 9(Q - Q') + 5(R - R') \\ & + 3(B - B' + N - N') + 1(P - P') \\ & - 0.5(D - D' + S - S' + I - I') \\ & + 0.1(M - M') \end{aligned}$$

where primed letters are the opponent's counts. K, Q, R, B, N, P are kings, queens, rooks, bishops, knights, pawns. D, S, I are doubled, blocked, isolated pawns. M is mobility. Deep Blue's evaluator had ~ 8000 **components**.

7.6.2 Tic-Tac-Toe Evaluation

A very simple one:

$$h(N) = (\# \text{ free rows/cols/diagonals for Max}) - (\# \text{ free for Min}).$$

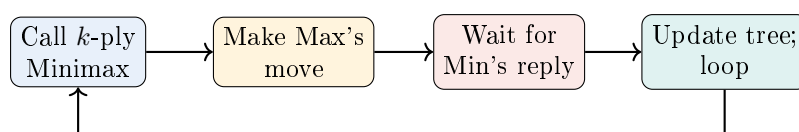
At empty board, the value is 0; in mid-game it shifts based on which lines each side can still potentially complete.

7.6.3 Evaluation vs. Lookahead Trade-off

Why bother with lookahead if we have an evaluation function?

- Grandmasters (**Adriaan de Groot's** studies) recognise thousands of patterns and can play one-ply by intuition.
- But for computers, a perfect h is impossible — so we combine modest h with deep lookahead.
- Lookahead can *reveal* things the evaluator misses. Example: a fork (a knight attacking two pieces) — the evaluator sees just one knight; lookahead reveals the inevitable loss of a major piece.

7.6.4 The Game-Playing Loop



The crucial trick: *while waiting for the opponent*, the computer can deepen its search.

7.7 The Zugzwang and the Horizon Effect

7.7.1 Zugzwang

Zugzwang (German for “compulsion to move”) is a chess situation where any move you make worsens your position — you’d rather pass. Evaluation functions ignorant of “turn to move” can misvalue zugzwang positions badly. Many strong programs include a “side to move” feature in h .

7.7.2 The Horizon Effect

Horizon Effect

A bad event can be *pushed beyond the search horizon* by playing meaningless intervening moves. The program then thinks it has avoided the danger — when in fact it has only postponed it past the lookahead depth.

Classic example: Deep Thought II (1990s), lookahead 10–11 ply, captured a poisoned rook in a contrived chess study because the consequences fell beyond ply 11. Humans saw the trap instantly. The fix is **secondary search** — extend the search selectively along promising/threatening lines before committing.

7.8 Alpha-Beta Pruning

7.8.1 The Intuition: “We’ve Already Seen Better”

Imagine you (Max) are evaluating a node with children c_1, c_2, c_3 . After evaluating c_1 you know your value is at least 12. While evaluating c_2 (which is a Min node), you find one of its grandchildren has value 5. Min at c_2 will pick at most 5 — *worse than 12 you already have*. So **you don’t need to inspect any more of c_2 ’s grandchildren**. Cut it off.

Intuition

The whole idea of alpha-beta: *once you know a node can’t improve on what’s already established elsewhere, stop exploring it*. The minimax value of the game is unchanged; the algorithm is simply faster.

7.8.2 Alpha and Beta Bounds

Definition

- α : the highest value Max has guaranteed so far along the current path. A *lower bound* for Max. Can only go up.
- β : the lowest value Min has guaranteed so far. An *upper bound* for Min. Can only go down.

7.8.3 The Two Cutoff Rules

Cutoff conditions

At any node during search, maintain (α, β) inherited from ancestors. When $\beta \leq \alpha$, *prune the remaining children* — they cannot affect the result.

- **α -cutoff** (at a Min node): one of Min's children turned out to be $\leq \alpha$, so Max above will never let us get here. Stop.
- **β -cutoff** (at a Max node): one of Max's children turned out to be $\geq \beta$, so Min above will never let us get here. Stop.

7.8.4 The Algorithm

Algorithm 23 Alpha-Beta Pruning

```

1: function ALPHABETA(node,  $\alpha$ ,  $\beta$ )
2:   if ISLEAF(node) then return EVAL(node)
3:   end if
4:   if node is Max then
5:      $v \leftarrow -\infty$ 
6:     for child  $c$  of node do
7:        $v \leftarrow \max(v, \text{ALPHABETA}(c, \alpha, \beta))$ 
8:        $\alpha \leftarrow \max(\alpha, v)$ 
9:       if  $\alpha \geq \beta$  then break ▷  $\beta$ -cutoff
10:      end if
11:    end for
12:   else ▷ Min node
13:      $v \leftarrow +\infty$ 
14:     for child  $c$  of node do
15:        $v \leftarrow \min(v, \text{ALPHABETA}(c, \alpha, \beta))$ 
16:        $\beta \leftarrow \min(\beta, v)$ 
17:       if  $\beta \leq \alpha$  then break ▷  $\alpha$ -cutoff
18:       end if
19:     end for
20:   end if
21:   return  $v$ 
22: end function

```

Initial call: AlphaBeta(root, $-\infty$, $+\infty$).

7.8.5 Worked Example on the Same Tree

Run alpha-beta left-to-right on the 16-leaf tree from earlier. The result is the same value 12. Two cutoffs fire:

- After Max-node $\max(12, 8) = 12$ on the left subtree, the Min node $\min(15, \dots) = 15$ already has 15 from its sibling 21. The 12 means the second child of this Min is ≤ 12 , but the Min parent already has ≤ 15 — actually the cutoff happens at the depth-3 Max node above, which has $\alpha = 15$ from its left subtree, and the depth-2 Min child (12) tries to bring the value down. We get a cutoff.

- Similar cutoffs further right.

Remember This

Alpha-Beta returns exactly the same minimax value as plain **Minimax** — it only *prunes* branches that provably cannot change the answer. It is therefore equivalent, just faster.

7.8.6 Complexity of Alpha-Beta

Knuth–Moore (1975) result

With **optimal move ordering** (best child first), alpha-beta examines only $O(b^{d/2})$ nodes — **the square root of Minimax's** $O(b^d)$.

Practically: a fixed-depth search can go **twice as deep** for the same cost. The branching factor is effectively halved.

Worst case (worst ordering): alpha-beta degrades to $O(b^d)$ — no pruning at all.

Random ordering: $O(b^{3d/4})$.

Best ordering: $O(b^{d/2}) = O((\sqrt{b})^d)$.

7.8.7 Move Reordering — Free Lunch From Idle Time

Alpha-Beta's efficiency *depends on the order* in which siblings are visited. **Best first** → most cutoffs. While waiting for the opponent's move, a clever program reorders moves based on values from the previous iteration of search. This is **move reordering** — same minimax value, faster runtime.

7.9 Influence Analysis (Pearl)

Why does Alpha-Beta work? Judea Pearl's *influence analysis* gives the clean argument.

For a node J deep in the tree with value v to actually **influence the root**, v must satisfy:

- Every Min ancestor between J and the root has $v < (\text{its current } \beta)$.
- Every Max ancestor between J and the root has $v > (\text{its current } \alpha)$.

Combining: v must lie strictly between the tightest α (from any Max ancestor) and the tightest β (from any Min ancestor):

$$\alpha < v < \beta.$$

If at any time $\alpha \geq \beta$, no value v can be influential. The current subtree is pruned. This is the cutoff rule, beautifully justified.

7.10 Algorithm SSS* (Stockman, 1979)

7.10.1 The Motivation

Alpha-Beta is depth-first. But the search space here is the space of *strategies*, not nodes. A best-first approach over strategies — refining the most promising one first — could be even better. This is what **SSS*** does.

7.10.2 Solution Space (Strategies) View

Recall a **strategy** for Max is a subtree picking one move at every Max node, with all Min children kept. The number of strategies grows multiplicatively with the number of Max nodes. Each strategy has a **value** (determined by Min's choices) which gives an *upper bound* on what Max can achieve via that strategy.

7.10.3 Clusters of Strategies

SSS* doesn't enumerate every strategy. It groups them into **clusters**: partial strategies represented by "at every Max node take all children; at every Min node take one child (leftmost by convention)."

SSS* core loop

1. Maintain a priority queue OPEN of nodes labelled **LIVE** (cluster not yet evaluated) or **SOLVED** (cluster evaluated, value known), ordered by their value (largest first).
2. Pop the best cluster. *Refine* it: if its Min child is **LIVE**, generate its children; if it has reached the horizon, evaluate.
3. When a Max node becomes **SOLVED**, propagate up via the minimax rule (and prune dominated clusters).
4. Terminate when the root pops out with **SOLVED** status.

7.10.4 The State Triple

Each entry on OPEN is a triple (N , status, value):

- **LIVE** means we have an upper bound (initially $+\infty$) but haven't finished evaluating.
- **SOLVED** means we know the exact value.

7.10.5 Walking Through an Example

On the 16-leaf binary tree from earlier (root is a Min):

1. Initial cluster: pick all from root (Max), one from each Min child (the leftmost). Four such initial clusters, each rooted at one of the four depth-2 Min nodes. Value = $+\infty$.
2. Pop the highest-value cluster (any of them, since all = $+\infty$). Refine to horizon.
3. Evaluate horizon leaves; back up using minimax.
4. As a Max node becomes solved at value v , add its sibling (the other Max child of the same Min parent) but *with an upper bound of v* — that sibling matters only if it's lower.
5. Continue, deleting dominated clusters (alpha-cutoff equivalent), until root is **SOLVED**.

Key invariant: SSS* never pursues a cluster whose upper bound is below the current best.

7.10.6 SSS* vs. Alpha-Beta

SSS* dominates Alpha-Beta in node count

For any uniform-depth game tree, the set of leaf nodes visited by SSS* is a **subset** of those visited by Alpha-Beta with left-to-right ordering (Stockman, 1979). However:

- SSS* needs an *exponential-size priority queue* — vs. alpha-beta's linear stack.
- Maintenance overhead (priority queue ops) can outweigh the saved leaf evaluations.

In practice, **alpha-beta with good move ordering matches SSS*** and uses far less memory. SSS* is mostly of theoretical / pedagogical interest.

7.11 Comparison Table: Minimax vs. Alpha-Beta vs. SSS*

Criterion	Minimax	Alpha-Beta	SSS*
Search style	DFS, full traversal	DFS with cutoffs	Best-First on strategies
Returns value of	game (with finite-depth h)	same as Minimax	same as Minimax
Time (worst)	$O(b^d)$	$O(b^d)$	$O(b^{d/2})$
Time (best ordering)	$O(b^d)$	$O(b^{d/2})$	$O(b^{d/2})$
Space	$O(bd)$ stack	$O(bd)$ stack	$O(b^{d/2})$ PQ
Move-order dep.	No	Yes (critical)	No
Practical?	Only as baseline	Yes — standard	Mostly theoretical

Memory Trick

Mnemonic: “Minimax = always look at Many. Alpha-Beta = A void (the obviously bad). SSS* = pick the Seemingly best strategy first.”

7.12 Common Mistakes

Common Mistake

1. Confusing α and β : α belongs to Max (lower bound), β to Min (upper bound).
2. Claiming Alpha-Beta is “better than Minimax” in terms of *value* — it returns the *same value*, just faster.
3. Forgetting move ordering: random alpha-beta is far worse than best-first alpha-beta.
4. Treating the horizon node's evaluation as terminal: it is just an estimate, subject to the horizon effect.
5. Applying minimax to a non-zero-sum game without further care: minimax is the right strategy only when “my gain = your loss.”
6. Assuming SSS* is always faster — its priority-queue overhead and memory cost often cancel its node savings.
7. Confusing “Max” (the player) with “maximisation” (the operation) — useful in tutorials,

but sometimes a maximising player has good moves that minimise something else (e.g. piece loss).

7.13 Real-World Applications

Real-World Application

Computer Chess (Deep Blue, Stockfish): alpha-beta + transposition tables + quiescence search + extensive move ordering heuristics (killer moves, history heuristic). Depths of 30+ ply are now routine.

Go (AlphaGo, AlphaZero): the search tree is so huge that pure alpha-beta is unworkable. AlphaGo combines **Monte Carlo Tree Search** with deep neural networks; AlphaZero learns everything from self-play.

Real-time game AI: bounded-depth alpha-beta is used in turn-based games (Civilization, XCOM). For real-time games (RTS, fighting), heavily abridged search with heuristic shortcuts.

Adversarial planning: minimax appears in robust control, security games (allocating airport screeners against an adversary), and worst-case design for safety-critical systems.

Auctions and markets: game-theoretic equilibrium analysis underpins ad auctions (e.g. Google's AdSense bid pricing), spectrum auctions (telecom), and market design (Roth's Nobel-winning work on kidney exchange).

7.14 Quick Revision — Week 7

Quick Revision

- **Game theory**: study of strategy among rational players. Nash equilibrium = no player wants to deviate unilaterally.
- **Prisoner's dilemma**: rational defection $\Rightarrow$ collectively bad outcome ($T > R > P > S$).
- **Game classification**: zero-sum / positive / negative; 2-player / multi-player; perfect / imperfect info; deterministic / stochastic.
- **Course scope**: two-player zero-sum perfect-info deterministic alternate-move games.
- **Game tree**: layered tree alternating Max (Δ) and Min (∇) levels.
- **Minimax backup rule**: Max picks max of children, Min picks min.
- **Minimax algorithm**: DFS, $O(b^d)$ time, $O(bd)$ space.
- **Finite lookahead**: search to depth k , evaluate horizon with h , back up via minimax.
- **Evaluation function** (Shannon for chess): material + positional, ~ 8000 components in Deep Blue.
- **Horizon effect**: pushing bad news past the lookahead horizon. Cure: secondary search.
- **Zugzwang**: situation where moving is bad — evaluator should include side-to-move.
- **Alpha-beta pruning**: α = Max's lower bound, β = Min's upper bound. Cutoff when $\alpha \geq \beta$.

- **Alpha-Beta complexity:** $O(b^{d/2})$ with best move ordering — effectively doubles search depth.
- **Move reordering:** while opponent thinks, reorder to put likely-best moves first.
- **SSS* (Stockman, 1979):** best-first search over strategy clusters. Same value as minimax, fewer nodes, but exponential priority-queue space.

7.15 Most Important Quiz Points — Week 7

Exam Tip

1. The four characteristics of a board game in our scope: two-player, zero-sum, perfect-info, deterministic, alternate-move.
2. Minimax assumption: *opponent plays perfectly*. Result is a safe lower bound on Max's payoff.
3. Time complexity of Minimax = $O(b^d)$, of Alpha-Beta = $O(b^{d/2})$ (best ordering).
4. **Alpha-Beta returns the same value as Minimax** — it merely prunes provably irrelevant subtrees.
5. Alpha-cutoff fires at Min nodes; beta-cutoff at Max nodes. Trigger: $\alpha \geq \beta$.
6. Influence analysis: for a node's value v to influence the root, $\alpha < v < \beta$ must hold.
7. The *horizon effect*: postponing bad consequences beyond the lookahead depth. Fix: quiescence / secondary search.
8. *Zugzwang*: include “side to move” in evaluator to handle.
9. Move reordering can be done “for free” during the opponent's thinking time.
10. SSS* expands a subset of the nodes alpha-beta expands — but its priority queue grows exponentially.
11. Deep Blue (1997), AlphaGo (2016), AlphaZero (2017), Pluribus (2019) — milestone wins.
12. Chess tree size is $\sim 35^{80} = 10^{120}$ — *Shannon's number*. Brute force is impossible.

7.16 Practice Questions — Week 7

Conceptual.

1. Define: zero-sum game, perfect information, Nash equilibrium.
2. Why is the prisoner's dilemma considered “a Nash equilibrium that is collectively suboptimal”?
3. Explain why we always evaluate game trees from Max's perspective.
4. State and explain the minimax backup rule for a Max node and for a Min node.
5. Why does Alpha-Beta return exactly the same value as Minimax?
6. Define α and β . When does an α -cutoff fire? A β -cutoff?
7. Explain Pearl's influence-analysis argument for why $\alpha \geq \beta$ implies the current subtree can be pruned.

8. What is the horizon effect, and how is secondary search used to mitigate it?
9. Why does move ordering matter so much for alpha-beta efficiency?
10. Compare evaluation functions for chess (Shannon's formula) and for tic-tac-toe (rows/cols/-diagonals).
11. Explain "finite lookahead" and how it interacts with the evaluation function.
12. What is a *strategy* in a game tree? What is the value of a strategy?
13. Sketch the SSS* algorithm. Why does it use the data structure $(N, \text{status}, \text{value})$ on its priority queue?
14. Compare SSS* and Alpha-Beta in terms of (i) time, (ii) space, (iii) practical use.

Numerical / trace.

15. Given the tree with leaves $[3, 5, 6, 9, 1, 2, 0, -1]$, rooted at Max with alternating Min/Max levels, compute the minimax value.
16. For the same tree, run Alpha-Beta left-to-right and identify which subtrees are pruned.
17. Construct an 8-leaf example where alpha-beta with left-to-right ordering prunes *nothing*, then show how reversing the order causes many cutoffs.
18. Trace SSS* on a small 4-leaf tree (one Max root, two Min children, each with two leaves).
19. Compute $b^{d/2}$ for chess if $b = 35$, $d = 12$. How does this compare with b^d ? What does this mean for the search depth one can afford?

Algorithmic.

20. Write recursive pseudocode for Minimax.
21. Write recursive pseudocode for Alpha-Beta with parameters (α, β) .
22. Modify Minimax to return not just the value but also the best move from the root.
23. Add a transposition table to Alpha-Beta. Why does it work especially well in chess?

MCQ-style.

24. In a zero-sum game, the values of Max and Min:
 - (a) sum to zero (b) are equal (c) are independent (d) are both positive.
 Answer: (a).
25. Alpha-Beta with the best move ordering expands:
 - (a) $O(b^d)$ (b) $O(b^{d/2})$ (c) $O(\log b^d)$ (d) $O(d)$ nodes.
 Answer: (b).
26. The horizon effect occurs because:
 - (a) the heuristic is bad (b) bad consequences are pushed past the depth limit (c) alpha-beta over-prunes (d) the tree is unbalanced.
 Answer: (b).
27. At a Max node during Alpha-Beta, a β -cutoff fires when:
 - (a) $v \leq \alpha$ (b) $v \geq \beta$ (c) $\beta \leq \alpha$ (d) v is unchanged.
 Answer: (b) and (c) are equivalent.
28. SSS* expands which set of leaves compared to left-to-right alpha-beta?
 - (a) the same (b) a strict subset (c) a strict superset (d) incomparable.
 Answer: (b).

29. Deep Blue defeated Kasparov in:

- (a) 1989 (b) 1992 (c) 1997 (d) 2002.

Answer: (c).

30. Which is *not* a zero-sum game?

- (a) chess (b) tic-tac-toe (c) prisoner's dilemma (d) checkers.

Answer: (c) — both can defect or cooperate; the payoffs don't sum to zero.

31. The minimax value computed for a Max root means:

- (a) the worst Max can do (b) the value Max can guarantee against perfect Min (c) the expected score (d) the lowest leaf value.

Answer: (b).

End of Week 7.

8 Week 8: Automated Domain-Independent Planning

“Until now we have viewed problem-solving as state-centered. The planning community takes an action-centric view: a solution is a sequence of actions, not a sequence of states.”

— The shift of perspective

Goal of Week 8. For seven weeks we have searched for *states*. This week we change perspective and search for *actions* — the building blocks of plans. We will introduce the **STRIPS** formalism, the **Blocks World** as our running example, then study three planning paradigms: **Forward State Space Planning (FSSP)**, **Backward State Space Planning (BSSP)**, and the hybrid **Goal Stack Planning (GSP)**. We will encounter **Sussman’s Anomaly**, see why goal ordering matters, and finally meet the elegant solution: **Partial Order Planning (POP)**, which plans in plan space rather than state space.

8.1 From State-Search to Action-Search

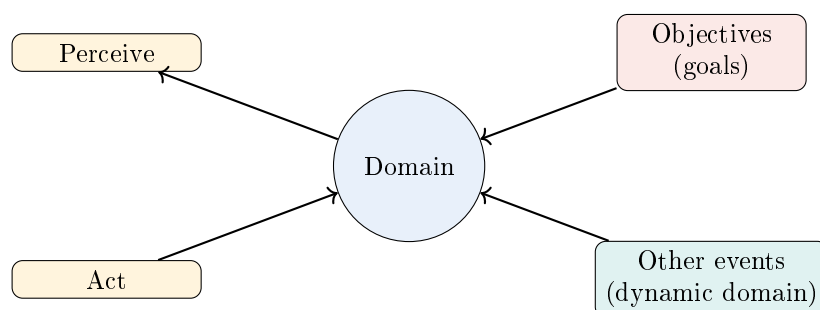
For chess we cared about states (board positions). But many problems are naturally about *doing things*: stacking blocks, baking a cake, scheduling a flight. The **planning community** (Fikes, Nilsson, McDermott, Sacerdoti, Tate, Kambhampati, Ghallab, Nau, Traverso, ...) thinks in terms of **actions and plans**.

8.1.1 The Reasoning Side of Acting

An autonomous agent in some domain:

- Has **goals** to achieve,
- Has access to a **repository of actions / operators**,
- Uses **search or reasoning** to find a plan,
- **Executes** the plan, **monitors** it, and may **replan** when things go wrong.

This is the *sense-plan-act* loop, the central organising structure of an intelligent agent (cf. Week 1).



8.2 Classifying Planning Domains

Real-world planning problems vary in complexity along several axes:

Axis	Choices
Perception	<i>Full</i> (complete info) vs. <i>Partial</i> (sensors limited).
Goal type	<i>Satisfaction</i> (hard constraints on end state) vs. <i>Soft preferences</i> ; <i>Trajectory constraints</i> (also on intermediate states).
Actions	<i>Deterministic</i> vs. <i>Stochastic</i> ; <i>Instantaneous</i> vs. <i>Durative</i> (with start/end times); with/without costs.
Other agencies	<i>Solo</i> vs. <i>Multi-agent</i> (collaborative, adversarial, or competing for resources).
Environment events	<i>Static</i> (only the agent acts) vs. <i>Dynamic</i> (external events: rain, shops opening, etc.).

We focus on the simplest setting: *STRIPS* domains.

8.3 STRIPS Domains

STRIPS = **S**Tanford **R**esearch **I**nstitute **P**lanning **S**ystem (Fikes & Nilsson, 1971). It is the simplest formal model of planning and the ancestor of all modern planners.

Definition

A **STRIPS domain** has:

- A **finite** set of states S and actions A .
- **Complete** information — the agent always knows the current state.
- **No other agents**; the world is *static* (only the planning agent changes it).
- **Deterministic** actions — no chance, no execution errors.
- **Instantaneous** actions — no time.
- Goals are **hard constraints on the final state**.

This is exactly Khemani's **SCOAR-D** setting from Week 1.

8.3.1 State-Transition System

A STRIPS planning problem is a triple $\Sigma = (S, A, \gamma)$ where $\gamma : S \times A \rightarrow S$ is the **state-transition function**. If action a is applicable in state s , then $\gamma(s, a)$ is the resulting state.

8.4 The Planning Domain Description Language (PDDL)

PDDL, invented around 1998 for the International Planning Competition, brought uniformity to planning research. Successive versions added expressivity:

PDDL 1.0	STRIPS
PDDL 1.2	Conditional effects
PDDL 2.1	Numerical fluents (model fuel, money, etc.), plan metrics, durative actions
PDDL 2.2	Derived predicates, timed initial literals (model exogenous events)
PDDL 3.0	State trajectory constraints, preferences (soft constraints)
PDDL 3.1	Object fluents (functions returning objects)

A PDDL planning task has five ingredients:

1. **Objects**: things in the world (blocks, rooms, robots).
2. **Predicates**: properties / relationships ($\text{on}(X,Y)$, $\text{clear}(X)$).
3. **Initial state**: full description of the start.
4. **Goal specification**: partial description — set of things that must be true.
5. **Actions / Operators**: ways of changing the state.

Remember This

For this course we will stay within **STRIPS** / **PDDL 1.0**.

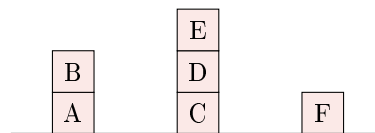
8.5 The Blocks World — Our Running Example

The Blocks World Domain

A finite set of identical blocks on a table. A one-armed robot picks them up and stacks them. Predicates:

- $\text{on}(X,Y)$ — block X is on block Y .
- $\text{ontable}(X)$ — block X is on the table.
- $\text{clear}(X)$ — nothing is on block X .
- $\text{holding}(X)$ — the arm is holding X .
- AE (“arm empty”) — the arm is empty.

Restrictions: a block has at most one block on it; the table is arbitrarily large; the arm holds at most one block.



An initial Blocks World state

8.5.1 The Four Blocks-World Operators

Each operator has a name with arguments, **preconditions**, **positive effects** (the ADD list), and **negative effects** (the DELETE list).

Operator	Preconditions	Adds	Deletes
$\text{pickup}(X)$	$\text{ontable}(X)$, AE	$\text{clear}(X)$, $\text{holding}(X)$	$\text{ontable}(X)$, $\text{clear}(X)$, AE
$\text{putdown}(X)$	$\text{holding}(X)$	$\text{ontable}(X)$, AE	$\text{clear}(X)$, $\text{holding}(X)$
$\text{stack}(X,Y)$	$\text{holding}(X)$, $\text{clear}(Y)$	$\text{on}(X,Y)$, $\text{clear}(X)$, AE	$\text{holding}(X)$, $\text{clear}(Y)$
$\text{unstack}(X,Y)$	$\text{on}(X,Y)$, $\text{clear}(X)$, AE	$\text{holding}(X)$, $\text{clear}(Y)$	$\text{on}(X,Y)$, $\text{clear}(X)$, AE

Intuition

Each operator captures a small physical fact. `pickup` requires the block be on the table, not under anything, and the arm empty. After picking up, the arm is full and the block is off the table — *the delete list captures what is no longer true*. Together the four operators describe everything the robot can do.

8.5.2 Actions vs. Operators**Definition**

A **planning operator** is parameterised (with object types). An **action** (or *ground operator*) is an operator with concrete objects substituted in. For example, `pickup(X)` is an operator; `pickup(A)` is an action.

8.6 Forward State Space Planning (FSSP)**8.6.1 Definitions****Key formulas for FSSP**

- **Applicable action.** Action a is applicable in state S if $\text{pre}(a) \subseteq S$.
- **Progression.** Applying a to S :

$$S' = \gamma(S, a) = (S \cup \text{effects}^+(a)) \setminus \text{effects}^-(a).$$

- **Plan.** Sequence $\langle a_1, a_2, \dots, a_n \rangle$.
- **Valid plan** for goal G from S_0 : there exist states $S_1, \dots, S_n$ with $\gamma(S_{i-1}, a_i) = S_i$ and $G \subseteq S_n$.

8.6.2 The Branching Problem

From a Blocks World state with many clear blocks, there are *many* applicable actions: pick up any of the clear top blocks, or unstack any clear block from another, etc.

For the 17-block example state at the top, the number of applicable actions can be more than ten, and the tree's branching factor explodes.

Remember This

FSSP suffers from high branching. The initial state is fully described, so every operator whose preconditions are satisfied is applicable — and many are. Without a good heuristic, FSSP is impractical for non-trivial problems.

8.7 Backward State Space Planning (BSSP)

The goal is usually small (e.g. “ $\text{on}(B, C) \wedge \text{on}(A, B)$ ”). Why not start from the goal and work backwards?

8.7.1 Relevant Actions and Regression

Key formulas for BSSP

- **Relevant action.** Action a is relevant to goal G if it adds something we need and deletes nothing we need:

$$\text{effects}^+(a) \cap G \neq \emptyset \quad \text{and} \quad \text{effects}^-(a) \cap G = \emptyset.$$

- **Regression.** Applying a to a goal G gives a new goal (subgoal):

$$G' = \gamma^{-1}(G, a) = (G \setminus \text{effects}^+(a)) \cup \text{pre}(a).$$

That is: remove the goals a achieves, add the preconditions a needs.

- **Termination.** Regression ends when $G_0 \subseteq S_0$ — every remaining subgoal is true in the start state.

8.7.2 The Spurious Sub-Goals Problem

The arrow of time problem. Planning operators are designed to be applied *forward*. When we regress through them, the resulting set G' may be *infeasible* — describe a state that is physically impossible.

Example: stacking A on B requires $\text{holding}(A) \wedge \text{clear}(B)$. If we then need $\text{on}(B, C)$, regressing through $\text{stack}(B, C)$ requires $\text{holding}(B) \wedge \text{clear}(C)$. Combined goal: $\{\text{holding}(A), \text{clear}(B), \text{holding}(B), \text{clear}(C)\}$ — but the robot's arm can hold only one block! **Infeasible.**

Quiz Trap

This is a famous trap. BSSP is *unsound* — it can generate spurious sub-goal combinations. We need an extra “validity check” (forward simulation) to confirm a candidate plan actually works.

8.7.3 FSSP vs. BSSP — Summary

Aspect	FSSP	BSSP
Start from	Start state S_0	Goal description G
Move test	Action applicable: $\text{pre}(a) \subseteq S$	Action relevant: $\text{effects}^+(a) \cap G \neq \emptyset, \text{effects}^-(a) \cap G = \emptyset$
Transition	Progression: $S' = (S \cup E^+) \setminus E^-$	Regression: $G' = (G \setminus E^+) \cup \text{pre}(a)$
Branching	High (many actions applicable)	Low (few actions relevant)
Soundness	Sound	Not sound — needs validity check
Plan construction	$\pi \leftarrow \pi \circ a$ (append)	$\pi \leftarrow a \circ \pi$ (prepend)

8.8 Goal Stack Planning (GSP)

Goal Stack Planning (Newell, Simon, and Nilsson, 1960s; in Strips form): *search backwards* (low branching) but *build the plan forward* (soundness).

8.8.1 The Big Idea

Use a stack to hold:

- Goals (atomic and compound),
- Actions (whose preconditions still need to be true).

Pop items from the stack one at a time. The stack discipline (LIFO) ensures that subgoals are achieved before they are needed.

8.8.2 PushSet — A Basic Operation

PushSet(G , stack)

For a compound goal $G = \{g_1, g_2, \dots, g_n\}$:

1. Push the compound G first.
2. Then push each g_i in some order (heuristic-driven, ideally).

The individuals will be popped (and solved) first; the compound is popped *last*, as a check that solving them individually didn't break the conjunction.

8.8.3 The GSP Algorithm

Algorithm 24 Goal Stack Planning (GSP)

```

1: function GSP(givenState, givenGoal, actions)
2:    $S \leftarrow \text{givenState}$ ;  $\pi \leftarrow ()$ ; stack  $\leftarrow \emptyset$ 
3:   PUSHSET(givenGoal, stack)
4:   while stack is not empty do
5:      $x \leftarrow \text{POP}(\text{stack})$ 
6:     if  $x$  is an action  $a$  then
7:        $\pi \leftarrow \pi \circ a$  ▷ add to plan
8:        $S \leftarrow \text{PROGRESS}(S, a)$ 
9:     else if  $x$  is a compound goal  $G$  and  $G \not\subseteq S$  then
10:      PUSHSET( $G$ , stack)
11:    else if  $x$  is an atomic goal  $g$  and  $g \notin S$  then
12:      choose a relevant action  $a$  achieving  $g$ 
13:      PUSH( $a$ , stack)
14:      PUSHSET(pre( $a$ ), stack)
15:    end if
16:  end while
17:  return  $\pi$ 
18: end function

```

Line-by-line.

- Line 2: maintain a current state S , an empty plan π , and an empty stack.
- Line 3: seed the stack with the goal description.
- Line 5: pop the top.
- Lines 6–8: if it's an action, it must be applicable (we maintained S); add to plan and progress S .
- Lines 9–10: if it's a still-unsatisfied compound goal, push it as a set.
- Lines 11–14: if an atomic goal isn't true, choose a relevant action to achieve it; push the action then push its preconditions.

8.8.4 Worked Example — The Right Goal Order

Problem: Start: $\{B \text{ on } C, A \text{ alone on table}\}$. Goal: $\text{on}(A, B) \wedge \text{on}(B, C)$.

If we push $\text{on}(B, C)$ first, then $\text{on}(A, B)$:

1. Pop $\text{on}(A, B)$ — false. Push $\text{stack}(A, B)$, then preconds $\{\text{holding}(A), \text{clear}(B)\}$.
2. Pop $\text{clear}(B)$ — true. Continue.
3. Pop $\text{holding}(A)$ — false. Push $\text{pickup}(A)$, then preconds.
4. Pop $\text{ontable}(A), \text{clear}(A), \text{AE}$ — all true. Pop $\text{pickup}(A)$, apply $\Rightarrow \pi = [\text{pickup}(A)]$.
5. Pop $\text{stack}(A, B)$, apply $\Rightarrow \pi = [\text{pickup}(A), \text{stack}(A, B)]$.
6. Pop $\text{on}(B, C)$ — false (we put A on B , B is no longer clear).
7. Push $\text{stack}(B, C)$ and its preconds.
8. Continue: need $\text{holding}(B)$, so $\text{unstack}(A, B)$, $\text{putdown}(A)$, $\text{pickup}(B)$, $\text{stack}(B, C)$.

Result: 8 actions. *Suboptimal!* The wrong goal order forced us to undo and redo work.

Right goal order: push $\text{on}(A, B)$ first, then $\text{on}(B, C)$ (so $\text{on}(B, C)$ pops first):

1. Achieve $\text{on}(B, C)$: $\text{pickup}(B), \text{stack}(B, C)$.
2. Achieve $\text{on}(A, B)$: $\text{pickup}(A), \text{stack}(A, B)$.

Result: 4 actions. *Optimal!*

Remember This

Goal ordering matters. In some domains, the wrong order merely produces longer plans. In others, it leads to dead ends and requires backtracking.

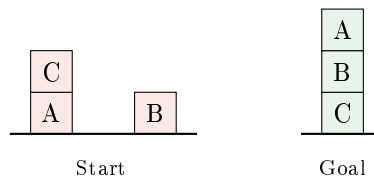
8.9 Sussman's Anomaly — When No Goal Order Works

In 1973 **Gerald Sussman** pointed out a tiny three-block problem that broke early linear planners.

8.9.1 The Setup

Start: $C \text{ on } A, A \text{ on table}, B \text{ on table}$.

Goal: $\text{on}(A, B) \wedge \text{on}(B, C)$.



8.9.2 Why Both Orders Fail

Try $\text{on}(B, C)$ first:

- Pickup B , stack B on C .
- Now to achieve $\text{on}(A, B)$: we need to unstack B (already placed) to free B . . . wait, we don't need to unstack B — we just need to put A on B .
- But A is under C . So: unstack C from A , putdown C , pickup A , stack A on B . That *works*, with 6 actions total.

Try on(A, B) first:

- Unstack C , putdown C , pickup A , stack A on B — 4 actions, $\text{on}(A, B)$ true.
- Now achieve $\text{on}(B, C)$: but A is on B , so B isn't clear. Unstack A , putdown A , pickup B , stack B on C — 4 more actions. But now $\text{on}(A, B)$ is false again!
- We then need to redo $\text{on}(A, B)$: pickup A , stack A on B — 2 more.
- Total: 10 actions. **And we undid and redid a goal we'd already achieved.**

The optimal plan is 6 actions and requires interleaving the two goals.

8.9.3 Non-Serializable Subgoals

Non-Serializable Subgoals

A planning problem has **non-serializable subgoals** when there is *no* ordering of the goals such that each can be solved independently without undoing the previous. The Rubik's cube, the 8-puzzle, and Sussman's Anomaly all share this property. **Linear planners like GSP cannot solve them optimally.**

The optimal solution to Sussman's Anomaly **interleaves** the two goals:

1. Unstack C , putdown C — *not directly serving either goal, but enabling both.*
2. Pickup B , stack B on C — completing $\text{on}(B, C)$.
3. Pickup A , stack A on B — completing $\text{on}(A, B)$.

Intuition

The fix is to *not commit to a goal order in advance*. Build the plan piece-by-piece, adding actions and ordering constraints as needed. This is **Partial Order Planning**.

8.10 Plan Space Planning — A New Search Space

State-space planners (FSSP, BSSP, GSP) search the *state space*: nodes are world states. **Plan-space planners** search a different space: nodes are *partial plans*, and edges are *plan-refinement steps*.

8.10.1 Anatomy of a Partial Plan

Definition

A **partial plan** π is a 4-tuple $\langle A, O, L, B \rangle$:

- A — a set of (partially instantiated) actions.
- O — a set of **ordering links** $(A_i \prec A_j)$ — “ A_i happens before A_j .”
- L — a set of **causal links** (A_i, P, A_j) — “ A_i 's effect P supports A_j 's precondition P .”
- B — a set of **binding constraints** on variables (e.g. $?X = A$, $?X \neq Y$).

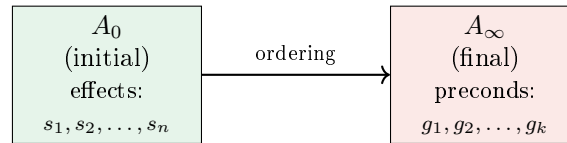
8.10.2 The Initial Plan π_0

Every plan-space search starts from the same skeleton:

$$\pi_0 = \langle \{A_0, A_\infty\}, \{(A_0 \prec A_\infty)\}, \emptyset, \emptyset \rangle$$

where:

- A_0 — the **initial action**: no preconditions, effects = initial state.
- A_∞ — the **final action**: preconditions = goal description, no effects.



This plan “stands for” the set of *all possible plans* for the problem. Each refinement step shrinks the set.

8.10.3 Flaws — What’s Missing

Definition

A partial plan has two kinds of **flaws**:

- **Open goal**: a precondition of some action A_p that has no causal link supporting it.
- **Threat**: an action A_t that could potentially delete the proposition P of a causal link (A_i, P, A_j) if A_t runs between A_i and A_j .

A **solution plan** is a partial plan with no flaws — every precondition is supported, no threats exist.

Plan-Space Search: systematically remove flaws by adding actions, causal links, ordering, and binding constraints.

8.11 Partial Order Planning (POP) — Resolving Flaws

8.11.1 Resolving an Open Goal

To resolve an open precondition P of action A_p :

1. **Reuse**: if an existing action A_e has P as a positive effect, add the causal link (A_e, P, A_p) and ordering $(A_e \prec A_p)$.
2. **Add**: otherwise, insert a new action A_{new} that produces P , with its own causal link and ordering.

The second route may itself introduce new open goals — the preconditions of A_{new} — which must also be resolved.

8.11.2 Detecting Threats

A causal link (A_i, P, A_j) is threatened by A_t if all three hold:

1. A_t has an effect $\neg Q$ such that P and Q can be unified.
2. A_t happens *after* A_i (or could).
3. A_t happens *before* A_j (or could).

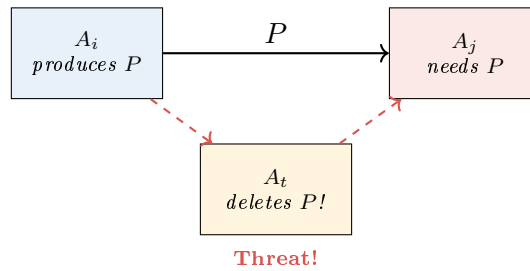
If all three are unavoidable, the threat **materializes**.

8.11.3 Three Ways to Resolve a Threat

Threat resolution strategies

- **Promotion:** schedule A_t *before* A_i . Add ordering ($A_t \prec A_i$).
- **Demotion:** schedule A_t *after* A_j . Add ordering ($A_j \prec A_t$).
- **Separation:** add a binding constraint so that P and Q cannot unify. (Only possible when P or Q has variables.)

8.11.4 Visualising a Threat



8.11.5 The POP Algorithm in Outline

Algorithm 25 Partial Order Planning

```

1: function POP( $\pi_0$ , operators)
2:    $\pi \leftarrow \pi_0$ 
3:   while  $\pi$  has flaws do
4:     pick a flaw  $f$  (open goal or threat) — heuristic choice
5:     if  $f$  is an open goal ( $P, A_p$ ) then
6:       nondeterministically choose: reuse existing action OR add new action providing  $P$ 
7:       add appropriate causal link, ordering, and binding constraints
8:     else  $\triangleright f$  is a threat by  $A_t$  on causal link ( $A_i, P, A_j$ )
9:       nondeterministically choose: promotion / demotion / separation
10:    end if
11:    if the resulting  $\pi$  is inconsistent (cyclic ordering, contradictory bindings) then back-track
12:    end if
13:  end while
14:  return any linearisation of  $\pi$  consistent with  $O$ 
15: end function

```

8.11.6 POP for the Sussman Anomaly

POP builds the plan without committing to a goal order:

1. Initial π_0 : A_0 (start state effects), A_∞ (goals $\text{on}(A, B), \text{on}(B, C)$).
2. **Open goal** $\text{on}(A, B)$: add $\text{stack}(A, B)$ with causal link to A_∞ .
3. **Open goal** $\text{on}(B, C)$: add $\text{stack}(B, C)$ with causal link to A_∞ .
4. **Open goals from stack's preconditions**: add $\text{pickup}(A), \text{pickup}(B), \text{unstack}(C, A), \text{putdown}(C)$.

5. **Resolve threats:** `stack(A, B)` deletes `clear(B)`, threatening `stack(B, C)`'s precondition. **Demote:** schedule `stack(A, B)` after `stack(B, C)`.
6. Linearise the resulting partial order $\Rightarrow$ optimal 6-action plan.

POP gracefully solves Sussman's Anomaly because it never linearised the goals.

8.12 Comparison: GSP vs. POP

Aspect	Goal Stack Planning (GSP)	Partial Order Planning (POP)
Search space	State space (with stack)	Plan space
Plan shape	Linear (totally ordered)	Partial order until linearised
Commitment	Eager: pick goal order up-front	Least commitment: defer ordering until forced
Non-serializable goals	Fails (or backtracks heavily)	Handles naturally
Algorithm style	Search through stack states	Search through partial plans
Worst-case	Exponential	Exponential, but smaller in practice
Famous example	Sussman's Anomaly (fails)	Sussman's Anomaly (succeeds)

Memory Trick

Mnemonic: “Plan less, commit later.” POP's principle of **least commitment** is one of the most important ideas in classical AI. Don't make a decision until forced to — then there's more information available to make it well.

8.13 Common Mistakes

Common Mistake

1. Confusing **applicable** (used in FSSP, $\text{pre} \subseteq S$) with **relevant** (used in BSSP, $\text{effects}^+ \cap G \neq \emptyset$ etc.).
2. In BSSP, forgetting that regression can produce spurious sub-goals — the validity check is essential.
3. In GSP, pushing the conjunction *after* the individual goals instead of before — breaks the verification step.
4. Forgetting to push the preconditions when an action is added to the stack.
5. Claiming GSP can solve all blocks-world problems optimally — Sussman's Anomaly disproves this.
6. In POP, forgetting that adding an action requires adding all four pieces: $\text{action} \in A$, ordering links, causal link to the consumer, possibly binding constraints.
7. Confusing *open goals* with *threats* — both are flaws but they need different fixes.
8. In threat resolution, attempting separation when neither P nor Q has variables — only ordering changes will work.

8.14 Real-World Applications

Real-World Application

Spacecraft autonomy: NASA's Remote Agent (1999) — used a hierarchical planner on Deep Space 1, the first AI to plan in space autonomously.

Logistics: the famous DARPA logistics planning saved hundreds of millions of dollars in the 1991 Gulf War by automating airlift scheduling. Modern descendants run UPS, FedEx, and Amazon supply chains.

Manufacturing scheduling: factories using PDDL-style planners coordinate hundreds of machines, parts, and tools.

Robotics: planning under partial information powers warehouse robots, autonomous cars, and the Mars rovers. Many use HTN (hierarchical task network) planners — a higher-level cousin of POP.

Game AI: games like F.E.A.R. (2005) introduced **Goal-Oriented Action Planning (GOAP)**, a STRIPS-style planner that drove the AI behaviour of enemies.

Domain-independent planning competitions (IPC): biennial events with hundreds of benchmark problems drive research forward. Modern planners regularly solve million-state problems in seconds.

8.15 Quick Revision — Week 8

Quick Revision

- **Action-centric view:** solutions are sequences of actions, not states.
- **STRIPS** = finite, static, deterministic, single-agent, complete-info, instantaneous-action planning.
- **Operator** = name + preconditions + add-list + delete-list. **Action** = grounded operator.
- **Blocks World predicates:** on, ontable, clear, holding, AE.
- **FSSP:** progress from S_0 with applicable actions. Sound but high branching.
- **BSSP:** regress from G with relevant actions. Low branching but unsound — needs validity check.
- **Progression:** $S' = (S \cup E^+) \setminus E^-$. **Regression:** $G' = (G \setminus E^+) \cup \text{pre}(a)$.
- **Goal Stack Planning (GSP):** backward search, forward plan construction. Uses a stack.
- **Goal ordering matters** in GSP — wrong order $\Rightarrow$ longer plan or dead end.
- **Sussman's Anomaly:** 3-block problem with non-serializable subgoals — linear planners fail.
- **Non-serializable subgoals:** no order lets you solve goals independently.
- **Plan Space Planning:** nodes are partial plans $\langle A, O, L, B \rangle$.
- **Open goals** and **threats** are the two kinds of flaws.
- **POP** resolves threats by **promotion**, **demotion**, or **separation**.

- **Principle of least commitment:** defer decisions until forced.
- POP handles Sussman's Anomaly naturally by interleaving the goals.

8.16 Most Important Quiz Points — Week 8

Exam Tip

1. STRIPS = STanford Research Institute Planning System (Fikes & Nilsson, 1971).
2. Operator = name + pre + add-list + delete-list. Add list = positive effects, delete list = negative effects.
3. FSSP applicability: $\text{pre}(a) \subseteq S$. BSSP relevance: $E^+(a) \cap G \neq \emptyset \wedge E^-(a) \cap G = \emptyset$.
4. Progression: $S' = (S \cup E^+) \setminus E^-$. Regression: $G' = (G \setminus E^+) \cup \text{pre}(a)$.
5. FSSP has high branching, BSSP has low branching. BSSP can produce spurious subgoals (unsound).
6. GSP idea: search backward, plan forward, use a stack.
7. GSP pushes compound goal first, then individual goals, so individuals pop first.
8. Sussman's Anomaly is non-serializable — no goal order lets GSP solve it optimally.
9. POP's partial plan = $\langle A, O, L, B \rangle$.
10. Flaws = open goals + threats.
11. Threat resolution: promotion / demotion / separation.
12. POP follows the principle of *least commitment*.
13. Names to remember: **Fikes, Nilsson, Sacerdoti, Sussman, Tate, McAllester, Penberthy.**

8.17 Practice Questions — Week 8

Conceptual.

1. Define a STRIPS domain. List the five characterising properties.
2. Distinguish a planning operator from an action.
3. Write the four Blocks World operators with their preconditions, add and delete lists.
4. State the formulas for progression and regression. Give a numerical example of each.
5. Why does FSSP suffer from high branching while BSSP does not?
6. Why is BSSP not sound? Illustrate with an example.
7. Describe Goal Stack Planning in your own words. Why is it called “goal stack”?
8. Why does GSP push the compound goal before the individual goals?
9. What is Sussman's Anomaly? Why does linear planning fail on it?
10. Define **non-serializable subgoals**. Give two examples beyond Sussman's Anomaly.
11. Define a partial plan and its four components.
12. What are the two types of flaws? How does POP resolve each?

13. State the principle of least commitment and give an example of how POP applies it.

Worked planning problems.

14. For the Blocks World start state $\{\text{on}(A, B), \text{ontable}(B), \text{ontable}(C), \text{clear}(A), \text{clear}(C), \text{AE}\}$, and goal $\{\text{on}(B, C)\}$, find an optimal plan by hand. How many actions?
15. Trace GSP on the problem in Q14 with two different goal orderings. Compare plan lengths.
16. Solve Sussman's Anomaly using POP. Draw the resulting partial-order plan.
17. Construct a Blocks World problem (with at least 4 blocks) where BSSP's spurious subgoals would mislead a naive planner. Show the offending sub-goal set.

Algorithmic.

18. Write the full GSP pseudocode and explain each line.
19. Modify GSP to use heuristic-driven goal ordering at each PushSet step.
20. Write the high-level POP algorithm. Identify the non-deterministic choice points.
21. Show by example how each of the three threat-resolution strategies (promotion, demotion, separation) works.

MCQ-style.

22. In STRIPS, the delete list of an action contains:
 (a) preconditions (b) negative effects (c) variables (d) goals.
Answer: (b).
23. Progression takes:
 (a) goal to subgoal (b) state to state (c) plan to plan (d) state to plan.
Answer: (b).
24. Backward State Space Planning is:
 (a) sound (b) unsound (needs validity check) (c) the same as FSSP (d) the same as POP.
Answer: (b).
25. Sussman's Anomaly has:
 (a) two serializable subgoals (b) two non-serializable subgoals (c) no solution (d) infinitely many goals.
Answer: (b).
26. POP resolves threats by:
 (a) promotion (b) demotion (c) separation (d) all of the above.
Answer: (d).
27. A partial plan in POP is a 4-tuple:
 (a) (S, A, γ, G) (b) (A, O, L, B) (c) (pre, add, del, name) (d) (g, h, f, w) .
Answer: (b).
28. The initial action A_0 in POP has:
 (a) no preconditions, effects = start state (b) all goals as preconditions (c) no effects
 (d) the entire state space as effects.
Answer: (a).
29. The principle of least commitment says:
 (a) commit to everything early (b) defer decisions until forced (c) backtrack early (d) ignore ordering.
Answer: (b).

End of Week 8.

9 Week 9: Problem Decomposition and Algorithm AO*

“Problem decomposition takes a goal-directed view: break the problem into smaller sub-problems. A solution is not a sequence — it is a subtree.”

— The shift in perspective

Goal of Week 9. For eight weeks our solutions have been *linear sequences* of states or actions. This week the view becomes *hierarchical*: a goal can be **decomposed** into independent sub-goals, and each sub-goal solved separately. The natural data structure is an **AND-OR tree** (**goal tree**), and the algorithm is **AO***, which generalises A* from path-search to subtree-search. Goal trees underpin Means-Ends Analysis, expert systems like DENDRAL, the Prolog programming language, and modern hierarchical task network (HTN) planners.

9.1 From Linear Plans to Hierarchical Decomposition

Imagine planning an evening out with friends: *pick an activity, then a movie, then a restaurant for dinner*. A naive depth-first planner would try:

- Visit Mall + The Matrix + Saravana Bhavan — friend rejects (mall).
- Visit Mall + The Matrix + Pizza Hut — friend rejects (mall).
- Visit Mall + AI movie + Saravana Bhavan — friend rejects (mall).
- (*many more, all rejected because of “mall”*)...

The **culprit** is *Visit Mall*. The other two choices (movie, dinner) are independent of it. Chronological DFS backtracking wastes enormous effort exploring all combinations beneath the bad choice.

Intuition

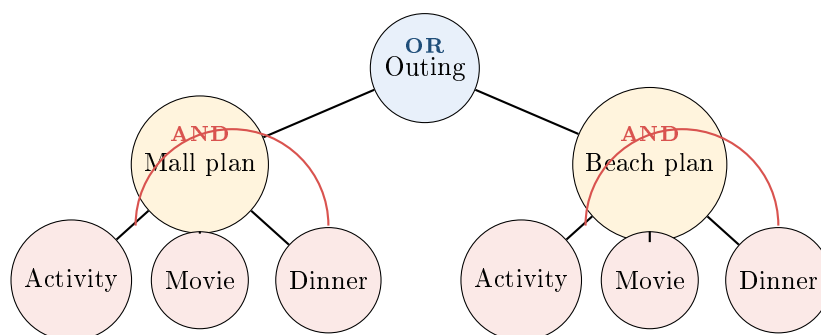
The problem is that the three sub-goals — *activity, movie, restaurant* — are **independent**. They should be solved separately. If the friend rejects the mall, we should switch to “visit beach” without re-trying every (movie, restaurant) pair beneath it.

9.2 AND-OR Trees (Goal Trees)

Definition

A **goal tree** (also called an **AND-OR tree**) is a tree where:

- **OR nodes** represent goals with multiple alternative solutions. To solve an OR node, solve *any one* child.
- **AND nodes** (drawn with an arc joining the outgoing edges) represent goals that decompose into independent sub-goals. To solve an AND node, solve *all* children.
- A **solution** is a *subtree* that, for every OR node, picks one child, and for every AND node, includes all children. Every leaf of this subtree must be a SOLVED primitive.



Remember This

The **AND-arc** is the visual marker for “all children must be solved.” Without the arc, we have an **OR node** — any one child suffices. The solution is a *subtree*, not a path.

9.2.1 Layering AND and OR Nodes

A node can be mixed (some AND, some OR children) but is usually **converted to a canonical form**: alternating AND and OR levels. A goal with k different decompositions becomes an OR node with k AND children — each AND child representing one decomposition.

9.3 Edge Costs, Heuristics, and the Cost of a Solution

Edges in a goal tree have **costs** representing the work needed to transform a problem. Nodes can have **heuristic values** $h(n)$ estimating the cost to solve them. **Solved leaf nodes** (primitive solutions) may have non-zero cost.

9.3.1 Backed-Up Cost

The cost of a partial solution rooted at node n is computed bottom-up:

- Leaf SOLVED: cost = $h(\text{leaf})$ (or 0 if a primitive).
- Leaf LIVE (not yet expanded): cost = $h(\text{leaf})$ (heuristic estimate).
- Internal AND node n with children $c_1, \dots, c_k$ and edge costs $e_1, \dots, e_k$:

$$\text{cost}(n) = \sum_i (e_i + \text{cost}(c_i)).$$

- Internal OR node n with children $c_1, \dots, c_k$:

$$\text{cost}(n) = \min_i (e_i + \text{cost}(c_i)).$$

Intuition

AND = “sum the children” (all must be solved). OR = “take the minimum” (pick the cheapest). This is what gives goal-tree search its character — and what makes propagation back to the root non-trivial.

9.4 Algorithm AO* (Martelli-Montanari, 1978; Nilsson, 1980)

Just as A* was a best-first search over paths, **AO*** is a best-first search over partial solution subtrees.

9.4.1 Core Idea

At any moment, AO* maintains the **partial graph** explored so far. Every choice point (OR node) carries a *marker* indicating the currently best child. Following these markers from the root yields the **current best partial solution**. AO*'s loop:

AO* main loop

1. **Forward phase**: starting at the root, follow marked edges to a set of unsolved leaves U . Pick one $n \in U$ and *refine* (expand) it.
2. **Backward phase**: re-compute n 's cost from its newly generated children. Propagate the revised cost up through all ancestors, possibly re-marking OR nodes (a previously second-best option may now be the best).
3. **Solved propagation**: if all children of an AND node on the marked path are SOLVED, the AND node becomes SOLVED. If the cheapest child of an OR node is SOLVED, the OR node becomes SOLVED.
4. Terminate when the root is SOLVED.

9.4.2 The AO* Algorithm

Algorithm 26 AO* (Algorithm)

```

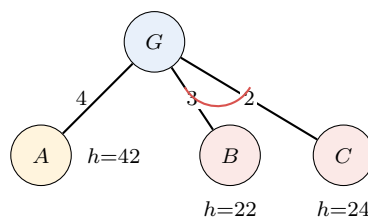
1: function AO*(root)
2:   Graph  $G \leftarrow \{\text{root}\}$ , marked best edges  $\emptyset$ ,  $h(\text{root})$  given
3:   while root is not SOLVED do
4:      $\triangleright$  Forward: traverse marked edges from root to an unsolved leaf  $n$ 
5:      $n \leftarrow$  leaf reached by following marked edges
6:     expand  $n$ : generate its children with their  $h$  values
7:     for each new child, attach it to  $n$  with its edge cost
8:      $\triangleright$  Backward: recompute  $n$ 's cost and propagate up
9:     recompute  $h(n)$  using the AND/OR rule above; mark best child
10:    for each ancestor  $a$  of  $n$ , bottom-up do
11:      recompute  $h(a)$  using its children's current costs
12:      if  $a$  is an OR node, re-mark the cheapest child
13:    end for
14:     $\triangleright$  SOLVED propagation
15:    if all children along marked path of  $n$  are SOLVED then mark  $n$  SOLVED
16:    end if
17:    propagate SOLVED upward as appropriate
18:  end while
19:  return marked subtree rooted at root
20: end function

```

9.5 Worked Example — Why Backed-Up Values Matter

Consider a root G with two OR alternatives:

- Left branch: single child A with $h(A) = 42$, edge cost 4 $\Rightarrow$ alternative cost = $4 + 42 = 46$.
- Right branch: AND of two children B and C with $h(B) = 22$, $h(C) = 24$, edge costs 3 and 2 $\Rightarrow$ alternative cost = $(3 + 22) + (2 + 24) = 51$.



The crucial subtlety. Node B has the *lowest individual* heuristic (22). But B is part of the more expensive option (51, not 46). **AO*** must expand based on **backed-up costs**, not raw h . It chooses the left branch and expands A .

Quiz Trap

Quiz trap: students often pick the node with the smallest h for expansion. Wrong! In an AND-OR graph, what matters is the cost of the *best partial solution*, which involves all of an AND node's children. Always compute backed-up costs first.

9.5.1 Continuing the Example

After expanding A , suppose A has two AND alternatives:

- $A \rightarrow (D, E)$ with $h(D) = 23, h(E) = 24$, edge costs 3, 4 $\Rightarrow (3 + 23) + (4 + 24) = 54$.
- $A \rightarrow (F, G')$ with $h(F) = 22, h(G') = 18$, edge costs 3, 2 $\Rightarrow (3 + 22) + (2 + 18) = 45$.

A 's new backed-up cost = $\min(54, 45) = 45$. So G 's left-branch cost becomes $4 + 45 = 49$.

Right branch (the $B + C$ AND) cost is still 51. Left branch wins ($49 < 51$). We continue down the marked path.

Remember This

Cost values can go up or down on expansion, depending on whether the heuristic was an underestimate (admissible) or overestimate. With admissible h , costs typically rise as the algorithm uncovers reality. We will revisit this when proving AO*'s admissibility.

9.6 Admissibility of AO*

Theorem (AO* admissibility)

If the heuristic function h **underestimates** the true cost of solving every node, then AO* returns the **least-cost** solution subtree.

Proof sketch. Same as A* in spirit:

- AO* always expands the cheapest-looking partial solution.
- If a partial solution's cost is underestimated, then as it's refined the cost can only rise to its true value.
- Therefore a more expensive option (whose cost is also underestimated) cannot become preferred unless it is genuinely cheaper.
- Once a fully refined (all leaves SOLVED) subtree's cost is below all other partial subtrees' costs, we know it is optimal.

9.6.1 Effect of Edge Cost on Search Depth

Edge cost 1 (sparse cost). Heuristic dominates; AO* quickly commits to one branch and plunges down.

Edge cost 10 (dense cost). Each expansion adds significant cost, raising the marked branch's value, allowing rivals to become competitive. AO* explores **more breadth** and switches branches more often.

Intuition

With heavier edge costs, even an underestimating heuristic produces a search that is more cautious — it keeps multiple options alive. This mirrors A*'s behaviour when ε (minimum edge cost) is large vs. small.

9.7 Means-Ends Analysis (Newell & Simon, 1957)

Goal trees naturally arise from **Means-Ends Analysis (MEA)**, a heuristic introduced by Newell, Shaw, and Simon for the **General Problem Solver (GPS)**:

MEA strategy

To achieve goal G from state S :

1. **Compare** S and G , list the *differences*.
2. **Evaluate** the differences (e.g. by importance / magnitude).
3. **Consult** an operator-difference table (which operator reduces which difference?).
4. **Reduce** the largest difference first.
5. Recursively: to apply operator O_i , ensure its preconditions are met (recurse on those).

This recursive structure unfolds into a goal tree:

- OR over which operator O_i to apply.
- AND over the sub-goals: achieve $\text{pre}(O_i)$ *and then* apply O_i *and then* achieve G from the new state S' .

Remember This

Means-Ends Analysis is the engine behind GPS and many early AI systems. It is essentially **Backward State Space Planning with depth-first decomposition**.

9.8 DENDRAL: The First Expert System (1965–1980)

DENDRAL (Stanford; Edward Feigenbaum, Bruce Buchanan, Joshua Lederberg) was one of the earliest AI successes — and the prototype of an **expert system**.

Goal: Given a mass spectrogram and the molecular formula of an unknown compound, determine its structural formula.

Approach: DENDRAL explored an AND-OR graph of candidate structures:

- **CONGEN** (CONstrained GENerator) generated candidate structures meeting the formula.
- Each candidate's synthetic spectrogram was predicted.
- Predicted spectra were compared to the observed spectrum.
- Knowledge of chemistry was used to prune branches *a priori*.

DENDRAL outperformed most human chemists. It introduced the **knowledge-intensive paradigm**: a small problem-solver plus a large body of expert rules. This paradigm dominated 1970s–80s AI.

9.9 Backward Chaining and Prolog: Goal Trees in Logic

A modern instance of goal-tree search appears in **Backward Chaining** for logical deduction — the engine inside Prolog.

9.9.1 Goal Trees from Rules

Given a rule IF $(P(?x) \wedge Q(?x))$ THEN $R(?x)$ and a goal $R(?z)$:

$$\text{Goal } R(?z) \implies \text{AND-subgoals } \{P(?z), Q(?z)\}.$$

A rule's antecedents form an AND node. Multiple rules with the same consequent form an OR node. Each known fact is a SOLVED leaf.

9.9.2 Worked Example

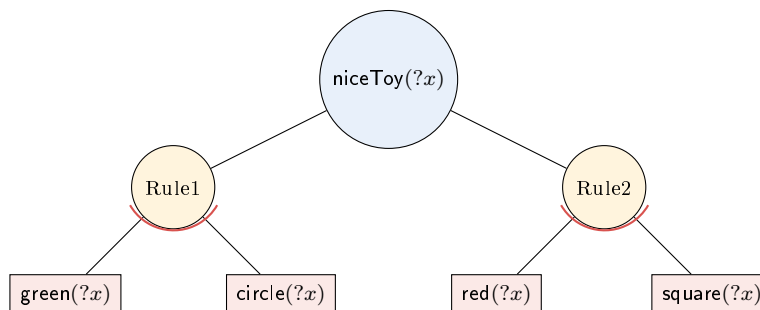
Rule1: IF (green(?x) AND circle(?x)) THEN niceToy(?x)

Rule2: IF (red(?x) AND square(?x)) THEN niceToy(?x)

Facts: green(A), green(B), circle(C), circle(E),
red(C), red(D), square(D)

Goal: niceToy(?z)

The resulting goal tree:



Depth-First Search trace:

1. Try Rule1: need green and circle of same x .
2. green(A): substitute $?x = A$. Need circle(A). FAIL.
3. green(B): $?x = B$. Need circle(B). FAIL.
4. Try Rule2: red(C), need square(C). FAIL.
5. red(D), need square(D). **SUCCESS**: $?x = D$.

This is exactly how Prolog executes.

9.9.3 Prolog Example: the Outing Plan

```
outingPlan(X,Y,Z) :- eveningPlan(X), moviePlan(Y), dinnerPlan(Z).
eveningPlan(X)    :- outing(X), likes(friend, X).
moviePlan(X)      :- movie(X), likes(friend, X).
dinnerPlan(X)     :- restaurant(X), likes(friend, X).
outing(mall).      outing(beach).
movie(theMatrix).  movie(bhuvanShome). ...
restaurant(pizzaHut). restaurant(saravanaBhavan).
likes(friend, beach). likes(friend, theMatrix).
likes(friend, bhuvanShome). likes(friend, saravanaBhavan).
```

Prolog's depth-first backtracking traces a path through this goal tree:

1. Try $X = \text{mall}$: likes friend mall? No. Backtrack.
2. Try $X = \text{beach}$: likes friend beach? Yes. Now move on to movie.
3. Try $Y = \text{theMatrix}$: likes? Yes. Move on to dinner.
4. Try $Z = \text{pizzaHut}$: likes? No. Backtrack.
5. Try $Z = \text{saravanaBhavan}$: likes? Yes. **SUCCESS**: $(X, Y, Z) = (\text{beach}, \text{theMatrix}, \text{saravanaBhavan})$.

Real-World Application

Prolog (Programmation en Logique, Colmerauer & Roussel, 1972) is built entirely around backward chaining on goal trees. Programs are knowledge bases of facts and rules; queries are goals; execution is depth-first search of the goal tree. This makes Prolog one of the most expressive languages for AI knowledge representation.

9.10 Comparison: A* vs. AO*

Aspect	A*	AO*
Search space	State space graph	AND-OR graph (goal tree)
Solution shape	Path (sequence)	Subtree
Node types	All OR (any successor)	OR <i>and</i> AND nodes
Cost combination	$g + h$ along path	Sum (AND) / Min (OR)
Best-first based on	$f(n) = g(n) + h(n)$	Backed-up cost of partial subtree
Admissibility	$h \leq h^*$	h underestimates every node
Termination	Goal popped from OPEN	Root marked SOLVED
Backtracking	Rare (with consistent h)	Frequent re-marking on backup

Memory Trick

Mnemonic: “A* finds a path; AO* finds a plan.” A* is for state-space search where you want a route. AO* is for problem decomposition where you want a strategy or recipe.

9.11 Common Mistakes

Common Mistake

1. Forgetting that AO* picks based on *backed-up cost*, not raw *h*. A node with low *h* in an expensive AND group is not the best choice.
2. Treating an AND node as if you only need one child — you need *all*.
3. Drawing an AND-OR tree without distinguishing AND from OR (use the arc!).
4. Computing AND-node cost using min instead of $\sum$, or OR-node cost using $\sum$ instead of min.
5. Forgetting to propagate SOLVED status upward.
6. Confusing AO* with A* on a tree of OR nodes — without AND nodes, the algorithms are the same.
7. In Prolog/backward chaining, expecting a goal tree's solution to be a path. It is a *subtree*.

9.12 Real-World Applications of AO*

Real-World Application

Expert systems: DENDRAL (chemistry), MYCIN (medical diagnosis), R1/XCON (configuring DEC computers) all used goal-tree search at their core.

Symbolic integration: Macsyma and modern computer algebra systems decompose an integral $\int f(x) dx$ via substitution, integration by parts, partial fractions — each a child of an OR node, with the resulting sub-integrals as AND children.

Hierarchical Task Network (HTN) planners: SHOP2, SIPE, O-Plan use AND-OR decomposition to plan at multiple levels of abstraction. Used by NASA in mission planning.

Theorem proving / Prolog: any logic program is essentially an AND-OR graph that the interpreter searches via backward chaining.

Diagnosis: model-based diagnosis (Week 11) decomposes “what failed in this system” into AND-OR trees over component faults.

Question answering: complex queries like “Who directed the highest-grossing film of 2020?” decompose into sub-queries via AND-OR graphs.

9.13 Quick Revision — Week 9

Quick Revision

- **Problem decomposition:** break a goal into independent sub-goals.
- **Goal tree / AND-OR tree:** OR nodes have alternatives; AND nodes (with arc) need all children.
- **Solution:** a *subtree* that picks one child at each OR node and all children at each AND node, with SOLVED leaves.
- **Cost combination:** AND = sum, OR = min.
- **AO* (Martelli-Montanari 1978; Nilsson 1980):** best-first over partial solution sub-

trees.

- **AO* loop:** forward (descend marked path, expand leaf), backward (recompute costs, re-mark), SOLVED propagation.
- **Best node to expand** = leaf of *cheapest partial solution*, not lowest raw h .
- **Admissibility:** AO* finds the optimal subtree when h underestimates the true cost.
- **Means-Ends Analysis (Newell-Simon):** compare state to goal, reduce largest difference; generates a goal tree.
- **DENDRAL:** first expert system (chemistry, 1965+); used CONGEN to explore AND-OR graphs.
- **Backward Chaining:** deductive retrieval traversing a goal tree built from rules and facts.
- **Prolog** programs are knowledge bases; execution = backward chaining on goal trees.

9.14 Most Important Quiz Points — Week 9

Exam Tip

1. AND-OR trees distinguish: OR = any child works; AND (arc) = all children needed.
2. Solution to a goal tree is a **subtree**, not a path.
3. AND-node cost = $\sum_i (e_i + \text{cost}(c_i))$; OR-node cost = $\min_i (e_i + \text{cost}(c_i))$.
4. AO* picks the leaf on the **currently-best partial solution** subtree.
5. AO* propagates costs *and* SOLVED markers upward after each expansion.
6. Admissibility of AO* requires h to underestimate the true cost (parallel to A*).
7. Means-Ends Analysis (GPS, Newell & Simon, 1957) generates AND-OR trees: choose an operator (OR), then satisfy its preconditions (AND).
8. DENDRAL (1965+) — first expert system, used AO*-style search for molecular structures.
9. Prolog implements depth-first backward chaining over an implicit goal tree.
10. AO* terminates when root is SOLVED, A* when goal is popped from OPEN.
11. AND-OR graphs subsume the case of plain OR-only graphs (then $\text{AO}^* = \text{A}^*$).

9.15 Practice Questions — Week 9

Conceptual.

1. Define an AND-OR tree. What is an AND node? What is an OR node? Draw a simple example.
2. Why is a goal-tree solution a subtree rather than a path?
3. State the cost-combination rules for AND nodes and OR nodes.
4. Describe AO*'s main loop in your own words (forward, backward, SOLVED propagation).
5. Why must AO* pick based on backed-up cost rather than raw h ?

6. State and explain AO*'s admissibility theorem.
7. How does edge cost affect AO*'s behaviour? Compare edge cost 1 and edge cost 10.
8. Describe Means-Ends Analysis and explain how it generates an AND-OR tree.
9. What was DENDRAL, why was it important, and how does it relate to AO*?
10. Explain how backward chaining in Prolog corresponds to a goal-tree search.
11. Compare A* and AO* on four axes: search space, solution shape, cost combination, termination.

Worked examples.

12. Construct an AND-OR tree for: "To make tea, choose either green tea (needs hot water and tea bag) or chai (needs hot milk, tea leaves, sugar, spices)." Identify all AND and OR nodes.
13. Given the partial AO* graph with root G , OR alternatives: left $\rightarrow A$ ($h = 10$, edge 1); right is AND (B, C) ($h = 3, h = 4$, edges 1, 1): which alternative is currently better? Show backed-up costs.
14. Expand the example tree above by giving A two AND children D, E ($h = 5, h = 4$, edges 1, 1). Recompute and identify the new best alternative.
15. Trace the AO* algorithm on a simple 6-node tree of your choice. Show forward, backward, and SOLVED propagation at each step.
16. Build the goal tree for the Prolog rules: $P(x) :- Q(x), R(x). \quad Q(a). \quad Q(b). \quad R(b). \quad R(c).$ What does the query $P(?z)$ return?

Algorithmic.

17. Write pseudocode for AO* in detail.
18. Modify AO* to allow cycles in the AND-OR graph (loops between nodes — common in expert systems).
19. Prove that AO* terminates if the graph is finite.
20. Sketch why AO* is admissible when h underestimates true cost.

MCQ-style.

21. In a goal tree, an AND node requires:
 - (a) any one child solved
 - (b) all children solved
 - (c) at least half the children solved
 - (d) a path from a parent.
 Answer: (b).
22. AO*'s cost at an OR node is:
 - (a) sum of children + edges
 - (b) max of children + edges
 - (c) min of children + edges
 - (d) average.
 Answer: (c).
23. AO*'s cost at an AND node is:
 - (a) sum of children + edges
 - (b) max
 - (c) min
 - (d) product.
 Answer: (a).
24. The solution returned by AO* is:
 - (a) a path
 - (b) a subtree
 - (c) a single node
 - (d) the entire graph.
 Answer: (b).

25. AO* is admissible when:

- (a) h is consistent (b) h underestimates true cost (c) the graph has no cycles (d) $h = 0$.

Answer: (b).

26. Means-Ends Analysis was introduced in:

- (a) STRIPS (1971) (b) Logic Theorist (1956) (c) GPS (1957) (d) DENDRAL (1965).

Answer: (c).

27. DENDRAL was an expert system for:

- (a) medical diagnosis (b) chemistry / molecular structures (c) chess (d) computer configuration.

Answer: (b).

28. Prolog's execution model is essentially:

- (a) BFS (b) forward chaining (c) backward chaining with depth-first goal-tree search
(d) A*.

Answer: (c).

End of Week 9.

10 Week 10: Pattern-Directed Inference Systems and the Rete Algorithm

“The programmer states the rules. The inference engine decides when to apply them. This is declarative programming.”

— The shift from imperative to declarative

Goal of Week 10. For nine weeks we hid all our domain knowledge inside a single function: `MoveGen`. This week we *dismantle* `MoveGen` into individual **rules**, each of the form “pattern $\Rightarrow$ action.” This gives us a **Pattern-Directed Inference System** — the technology behind classic **expert systems**, modern **Business Rule Management Systems (BRMS)**, and declarative AI. We will study the **OPS5** production-system language, conflict-resolution strategies, and the famous **Rete algorithm** (Charles Forgy, late 1970s) — one of the most important algorithms in symbolic AI, used to make rule-firing efficient even with thousands of rules and millions of facts.

10.1 From MoveGen to Rules

In every algorithm so far, `MoveGen(n)` produced the children of node n . What was inside it? A collection of **pattern–action** responses:

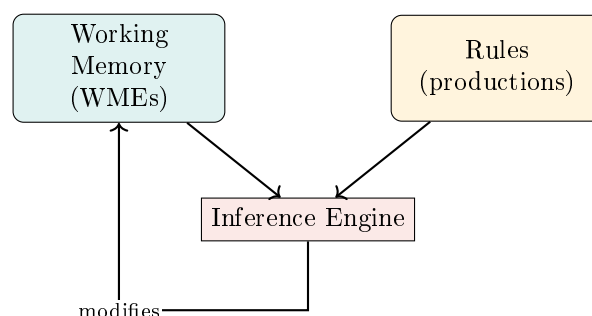
- “If the current state matches pattern P , then produce successor state S' via action A .”

Pattern-directed inference makes this explicit: `MoveGen` is replaced by a set of small, independent rules.

10.1.1 The Two-Memory Model

Rule-based systems are inspired by Newell & Simon’s model of human cognition (Week 1):

- **Working Memory (WM)** — short-term, currently active state. Contains **Working Memory Elements (WMEs)** — records or facts.
- **Long-Term Memory (LTM)** — stable rules of the domain.



10.1.2 Declarative Programming

Definition

A **rule** (or **production**) associates an action with a pattern in the state. The programmer states the rules; **the system decides when to apply them**. This is **declarative programming**, in contrast to imperative programming where the programmer specifies the order of operations.

Rule-based languages are Turing complete and remain enormously popular in:

- *Business rules*: lending decisions, insurance underwriting, regulatory compliance.
- *Diagnosis*: medicine (MYCIN), networks, manufacturing (CATS-1).
- *Configuration*: R1/XCON for DEC VAX systems.
- *Game AI*: Goal-Oriented Action Planning (GOAP) in F.E.A.R. and similar games.

10.2 Rule-Based Production Systems — Anatomy

A **production system** has three parts:

1. **Working Memory (WM)**: a set of WMEs representing the current state.
2. **Productions (rules)**: each has
 - A name,
 - One or more patterns on the **Left-Hand Side (LHS)**,
 - One or more actions on the **Right-Hand Side (RHS)**.
3. **Inference Engine (IE)**: the controller that executes the Match-Resolve-Execute cycle.

10.2.1 The Match–Resolve–Execute Cycle

The Inference Cycle

Repeat until termination:

1. **Match**. Find every (rule, WME-tuple) pair where the LHS patterns of the rule unify with WMEs in WM. This set is the **conflict set (CS)**.
2. **Resolve**. Pick one element from the CS — this is the **conflict resolution strategy**.
3. **Execute**. Run the RHS actions, possibly modifying WM (**make, remove, modify**).

Intuition

The conflict-resolution strategy is to rule-based systems what the heuristic function is to A* search: it encodes *control knowledge* about which choice to prefer. Different strategies lead to dramatically different behaviour.

10.3 OPS5 — The Reference Production System Language

OPS5 (Carnegie Mellon, late 1970s; Charles Forgy) — “**O**fficial **P**roduction **S**ystem 5” — was the first widely-used rule-based language. Form:

$$(\text{p ruleName LHS} \rightarrow \text{RHS})$$

10.3.1 WMEs in OPS5

A WME is a **structured record** with a class name and a set of attribute-value pairs:

```
(student ^name sudhir ^age 19 ^semester 3
      ^discipline math ^degree BSc)
```

Each WME has a hidden **time stamp** indicating when it entered WM (used for the *recency* conflict-resolution strategy).

10.3.2 Rule LHS — Patterns

The LHS is a conjunction of **patterns**, each one looking like a WME but with variables (<var>) and Boolean tests:

```
(p ranking
  (next ^rank <r>)
  (totalMarks ^student <s> ^marks <m> ^rank nil)
  -(totalMarks ^marks > <m> ^rank nil)          ; negative pattern
  ;-----
  (modify 1 ^rank (<r> + 1))
  (modify 2 ^rank <r>))
```

This rule reads: “Find the student with the highest **marks** who has not yet been ranked, and assign them the next rank.” The **negative pattern** (prefixed by **–**) is satisfied when *no* matching WME exists.

10.3.3 Tests in the LHS

Test	Meaning
constant	exactly this constant
<var>	any value (universal binding)
=	same type and value
<>	not equal
<=>	same type as
<, <=, >, >=	numeric comparisons
{t1 ... tN}	conjunction (all tests must pass)
<< v1 v2 ... >>	disjunction (any one)

10.3.4 Rule RHS — Actions

Available actions:

- **make**: add a new WME.
- **remove**: delete a WME.
- **modify** (= remove + make): change attribute values.
- **I/O**: read, write, load, halt.

Important: actions on the RHS happen **concurrently** (logically simultaneously), and they use the variable bindings from Match.

10.4 A Worked Example — Sorting with a Single Rule

```
(p swapSort
  (array ^index <i> ^value <X>)
  (array ^index {<j> > <i>} ^value {<Y> < <X>})
  ;-----
  (modify 1 ^value <Y>)
  (modify 2 ^value <X>))
```

Read: “If two array elements are out of order, swap them.” Apply repeatedly $\Rightarrow$ the array sorts itself. The control structure (*which* pair to swap next, when to stop) is handled by the inference engine — **the user wrote no loop, no comparison logic, no termination check.**

Remember This

This is the magic of declarative programming: tell the system *what* a valid step looks like; the engine figures out *when* to do it. The same rule that sorts a 4-element array also sorts a 4-million-element one — without any code change.

10.5 R1 / XCON — The Famous Expert System (1978)

R1 (internally called **XCON**, “eXpert CONfigurer”) was written in OPS5 by **John P. McDermott** at CMU in 1978. Its job: given a customer’s requirements, automatically configure Digital Equipment Corp’s VAX computer systems.

R1’s impact: by 1986, DEC was using R1 to configure every order. It saved DEC an estimated \$25 million/year and processed thousands of configurations. **R1 was the first expert system to be economically significant** and triggered the late-80s “expert systems boom.”

A sample rule:

```
ASSIGN-UB-MODULES-EXCEPT-THOSE-CONNECTING-TO-PANELS-4
IF: THE CURRENT CONTEXT IS ASSIGNING DEVICES TO UNIBUS MODULES
    AND THERE IS AN UNASSIGNED DUAL PORT DISK DRIVE
    AND THE TYPE OF CONTROLLER IT REQUIRES IS KNOWN
    AND THERE ARE TWO SUCH CONTROLLERS NEITHER OF WHICH ...
    AND THE NUMBER OF DEVICES THAT THESE CONTROLLERS ... IS KNOWN
THEN:
    ASSIGN THE DISK DRIVE TO EACH OF THE CONTROLLERS
    AND NOTE THAT THE TWO CONTROLLERS HAVE BEEN ASSOCIATED
    AND THAT EACH SUPPORTS ONE DEVICE
```

R1 grew to over 10,000 rules. **Hand-crafting that many rules is a famous knowledge engineering problem.**

10.6 Conflict Resolution Strategies

When the conflict set has multiple matching (rule, data) instances, **which one fires?** This is the central design decision of any production system.

10.6.1 1. Refractoriness

Definition

Refractoriness: a rule instance fires *at most once* per particular WME-binding. Even if the same rule still matches the same WMEs after firing, it will not be selected again.

Inspiration: neurons fire once, then go through a refractory (insensitive) period. Without refractoriness, a rule that doesn't change its triggering WMEs would loop forever.

10.6.2 2. Lexical Order

Choose the rule listed first in the program. If a rule has multiple instances, prefer earlier WMEs.

Prolog uses this strategy — but it weakens the declarative ideal (the programmer effectively orders the rules).

10.6.3 3. Specificity

Definition

Specificity: among matching rules, pick the one with the *most tests* (most specific patterns) — it likely captures a more refined situation.

Use case: default reasoning. A specific rule overrides a general one.

Example (bridge bidding):

- **pass-rule:** *default* — if it's your turn, pass.
- **1N-opening:** with 15–17 high-card points + balanced shape + no other bid, bid 1NT.

Both match. Specificity picks 1N-opening because it has more tests. The **pass-rule** acts as a default when nothing better fires.

10.6.4 4. Recency

Definition

Recency: pick the instance using the *most recently added* WME. The conflict set is maintained as a priority queue ordered by WME timestamp.

Effect: creates a **chain of thought** — actions cluster around recent additions to WM.

10.6.5 5. MEA (Means-Ends Analysis)

MEA strategy (CMU OPS5):

1. Partition rules by their *first pattern* (representing the current context / subgoal).
2. Apply **recency** on the first pattern to focus on one context.
3. Apply **specificity** for breaking ties within a context.

This mirrors Newell & Simon's problem-solving strategy from Week 9.

10.7 The Rete Algorithm — Making Matching Efficient

10.7.1 The Performance Problem

For 300 students each with 4 courses, there are 1200 **course** WMEs and 1200 instances of the grading rule in the conflict set. Each cycle:

- One instance fires, modifies one WME.
- *The other 1199 instances still match.*
- Naively recomputing the conflict set would re-test *everything*.

Brute-force match is hopeless — recomputing matches every cycle defeats the entire point of declarative rules.

10.7.2 The Rete Insight

Charles Forgy's insight (1979)

Don't recompute matches every cycle. Instead, build a *network* (in Latin: *rete* — a network of nerves or blood vessels) that:

- Stores partial matches between cycles (*inter-cycle savings*).
- Shares the test infrastructure across rules with common patterns (*intra-cycle savings*).
- Updates incrementally as WMEs are added or removed.

10.7.3 Architecture of a Rete Network

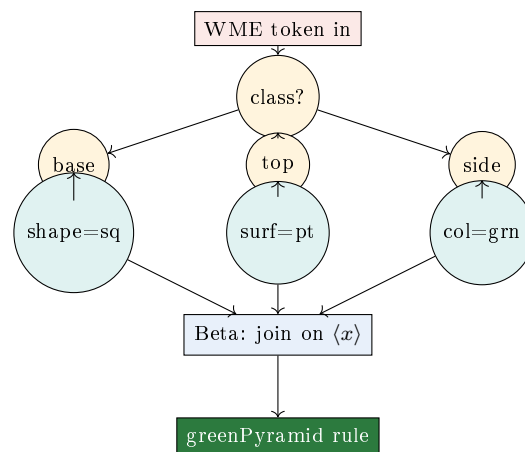
A Rete net has two halves:

Upper half — Alpha nodes (discrimination network):

- A discrimination tree of **single-WME tests**.
- Each test examines one attribute. WME-tokens travel from the root downward, taking branches based on test results.
- **Alpha memory nodes** at the leaves of this tree store WMEs that have passed all upstream tests — they act as “filtered WM.”

Lower half — Beta nodes (join network):

- Each beta node has *two or more parents* (other alpha or beta nodes).
- Performs a **join**: combine WMEs from each parent that share a common variable binding.
- Successful joins produce *token tuples* representing partial rule matches.
- A complete match (all patterns of a rule satisfied) is added to the conflict set as a rule instance.



10.7.4 Operation: Adding a WME

When a new WME enters WM:

1. A token $\langle +WME \rangle$ is inserted at the Rete root.
2. It descends the discrimination tree, taking branches based on its attributes.
3. Eventually it lands in zero or more alpha memory nodes.
4. Each alpha memory propagates the token down to its beta children, where joins are attempted.
5. Successful joins create new partial matches in beta memories.
6. If a rule's full pattern becomes matched, a new rule instance is added to the conflict set.

For **deletion**, the same process runs with $\langle -WME \rangle$, removing tokens from memories and rule instances from the CS.

10.7.5 Intra-Cycle Sharing

If rules **grade-AI-C** and **grade-ML-C** both have a pattern (**student** ...), the Rete network has *one* subtree handling that pattern, shared by both rules. Adding a student WME runs the test *once*, not twice.

10.7.6 Inter-Cycle Persistence

After one cycle, the beta memories still hold the partial matches that didn't fire. Next cycle, only the *difference* (newly added or deleted WMEs) needs processing. The 1199 surviving grade-assignments don't need to be recomputed.

The Rete payoff

Brute-force match time is $O(|R| \cdot |WM|^k)$ where k is the maximum patterns per rule. Rete reduces this by a factor proportional to the product of:

- rule similarity (shared patterns),
- WM stability (few changes per cycle),
- discriminative power of the discrimination tree.

Real systems see 100–10000x speedups. R1 with its 10,000 rules ran practically on 1980s hardware solely because of Rete.

10.7.7 Refractoriness Falls Out for Free

Once an instance is in the conflict set and fires, it is *removed* from the CS. It will not be re-added unless the matching WMEs change. **Refractoriness is automatic in Rete** — a beautiful design coincidence.

10.8 Modern Descendants of Rete

Algorithm / system	Notes
Rete (Forgy 1979)	The original. Powers OPS5, CLIPS, Jess.
Rete II / Rete III	Forgy's improvements; broke the "Rete wall" (degradation with large WM).
Rete-NT	Trade-secret algorithm from PST; very fast; used by major BRMS.
TREAT (Miranker 1987)	Trades inter-cycle persistence for simpler structure; sometimes faster.
LEAPS, Gator, HAL	Various optimisations and alternatives for specific patterns.
Drools, IBM ODM, FICO Blaze, Pegasystems	Modern commercial BRMS based on Rete variants.

10.9 Forward Chaining vs. Backward Chaining

Rule-based systems support two reasoning modes:

Aspect	Forward Chaining	Backward Chaining
Direction	Data-driven: facts $\rightarrow$ conclusions	Goal-driven: query $\rightarrow$ supporting facts
Engine	OPS5, CLIPS, Drools	Prolog, MYCIN
Use case	Monitoring, alerting, simulation	Diagnosis, theorem proving, deductive Q&A
Tree shape	Builds upward (facts grow)	Builds downward (goal $\rightarrow$ subgoals; Week 9)
Termination	When no rule fires	When goal proved (or all options exhausted)
Algorithm focus	Efficient Match (Rete)	Efficient unification + backtracking

Same rules can drive both directions. A rule IF (P, Q) THEN R: forward chaining derives *R* from *P, Q*; backward chaining tries to prove *R* by proving *P* and *Q*.

10.10 Common Mistakes

Common Mistake

1. Confusing **conflict set** (matched rule instances) with **conflict resolution** (the strategy for picking one).
2. Forgetting that OPS5 RHS actions happen *concurrently*, not sequentially — all use the

same Match bindings.

3. Treating a rule's LHS as a sequence; it is a **set** (order doesn't matter for matching, though the MEA strategy uses the first pattern's role specially).
4. Writing rules without refractoriness and getting infinite loops.
5. Designing a Rete network with the most discriminative test *last* — this defeats sharing. Most-discriminative test should be *first*.
6. Thinking the Rete net is the same as a decision tree — Rete combines a discrimination tree (upper half) with joins (lower half).
7. Confusing forward chaining (OPS5, Rete) with backward chaining (Prolog, MYCIN).

10.11 Real-World Applications

Real-World Application

R1/XCON (DEC): configured every VAX shipped 1980–1989. Saved \$25M+/year.

MYCIN: diagnosed bacterial infections from symptoms. Trained at Stanford. Achieved physician-level accuracy in narrow domains; pioneered *certainty factors* and *explanation facilities*.

Business Rule Management: every major bank uses BRMS (Drools, IBM ODM, FICO Blaze) for loan approval, fraud detection, regulatory compliance.

Telecom network monitoring: alarm correlation systems use Rete to process thousands of alarms per second, identifying root causes within seconds rather than hours.

Game AI: F.E.A.R. (2005), STALKER, and Halo use rule-based AI architectures (sometimes Rete-style) for enemy behaviour.

Insurance / underwriting: thousands of business rules for assessing policies — the canonical application of modern BRMS.

Compliance and tax: every modern tax-preparation system embeds thousands of conditional rules; Rete-style engines underpin many of them.

10.12 Quick Revision — Week 10

Quick Revision

- **Pattern-Directed Inference Systems** dismantle MoveGen into individual pattern-action rules.
- **Production system** = Working Memory + Productions + Inference Engine.
- **Match-Resolve-Execute** is the inference cycle.
- **OPS5**: first widely-used production language (Forgy, CMU, 1979). Rules: (p name LHS → RHS).
- **WMEs**: class name + attribute-value pairs + hidden timestamp.
- **Conflict Resolution Strategies**: refractoriness, lexical, specificity, recency, MEA.
- **R1/XCON (1978)**: first economically significant expert system; configured DEC VAXes.

- **Brute-force matching is hopeless:** $O(|R| \cdot |WM|^k)$.
- **Rete algorithm** (Forgy 1979): a discrimination tree (alpha nodes) + a join network (beta nodes) that stores partial matches across cycles.
- **Intra-cycle sharing:** common patterns shared across rules.
- **Inter-cycle persistence:** partial matches survive; only WM changes propagate.
- **Refractoriness** arises naturally in Rete.
- **Forward vs Backward chaining:** data-driven vs goal-driven. Same rules, different engines.

10.13 Most Important Quiz Points — Week 10

Exam Tip

1. A **production** = pattern (LHS) $\Rightarrow$ action (RHS).
2. The IE cycle is **Match–Resolve–Execute**.
3. **Conflict set** = matched rule instances; **Conflict resolution** = strategy to pick one.
4. Five common strategies: **refractoriness**, **lexical order**, **specificity**, **recency**, **MEA**.
5. **OPS5** originated at CMU; designer **Charles Forgy**.
6. **R1/XCON**: McDermott, 1978, configured VAXes, saved DEC \$25M/year.
7. **Rete** = Latin for “network” (of nerves / blood vessels).
8. Rete has **alpha nodes** (single-WME tests, discrimination tree) and **beta nodes** (joins on shared variables).
9. Rete gives **intra-cycle sharing** (between rules) and **inter-cycle persistence** (across cycles).
10. **Refractoriness** is automatic in Rete.
11. **Forward chaining** = data-driven (OPS5); **Backward chaining** = goal-driven (Prolog).
12. Major BRMS: Drools, IBM ODM, FICO Blaze, Pegasystems — all use Rete variants.

10.14 Practice Questions — Week 10

Conceptual.

1. What is a Pattern-Directed Inference System? How does it relate to **MoveGen**?
2. Describe the three components of a production system.
3. Explain the Match–Resolve–Execute cycle.
4. What is a WME? What are its parts in OPS5?
5. Distinguish a positive pattern from a negative pattern in OPS5.
6. List and explain five conflict-resolution strategies.
7. What is the MEA conflict-resolution strategy and how does it combine recency and specificity?

8. What problem does R1/XCON solve, and why was it historically important?
9. Why does brute-force matching scale poorly with the number of rules and WMEs?
10. What does “Rete” mean, and who invented the algorithm?
11. Describe the two halves of a Rete network.
12. How does Rete achieve intra-cycle sharing? Inter-cycle persistence?
13. Why does refractoriness arise naturally in Rete?
14. Compare forward chaining and backward chaining on at least four axes.

Worked examples.

15. Write an OPS5-style rule that says “If two students have the same total marks, mark them as tied.”
16. Write a single OPS5 rule that sorts an array (like `swapSort`). Trace its application on [3, 1, 4, 2].
17. Given the rules `grade-AI-C` and `grade-ML-C` (one for each subject), rewrite them as a single rule using a `cutoffs` WME.
18. Design a partial Rete net for two rules with one shared pattern (`student`) and two different second patterns. Show alpha and beta nodes.
19. Write OPS5-style rules to play tic-tac-toe with the heuristics “control centre,” “block opponent,” “complete a row.” Use specificity for ordering.

Algorithmic.

20. Sketch the Rete data structures: alpha memory, beta memory, join node.
21. Describe what happens when a new WME is added to WM in a Rete network.
22. Describe what happens when a WME is deleted from WM.
23. Outline pseudo-code for the Match algorithm using Rete.
24. Estimate the speedup of Rete over brute-force for 1000 rules each with 3 patterns and 10,000 WMEs.

MCQ-style.

25. OPS5 was designed by:
(a) John McCarthy (b) Charles Forgy (c) Edward Feigenbaum (d) Allen Newell.
Answer: (b).
26. The Inference Engine cycle is:
(a) Search–Plan–Execute (b) Match–Resolve–Execute (c) Pop–Expand–Push (d) Read–Eval–Print.
Answer: (b).
27. The Conflict Set contains:
(a) all rules (b) matched (rule, WME-tuple) instances (c) all WMEs (d) the goal.
Answer: (b).
28. Refractoriness means:
(a) a rule never fires (b) a rule instance fires at most once (c) the system halts (d) the WM is empty.
Answer: (b).

29. Specificity prefers:
(a) the rule with the most tests (b) the most recent WME (c) the first listed rule (d) random selection.
Answer: (a).
30. R1/XCON configured:
(a) IBM mainframes (b) DEC VAX systems (c) Apple computers (d) Cray supercomputers.
Answer: (b).
31. Alpha nodes in Rete do:
(a) joins (b) single-WME tests (c) rule execution (d) conflict resolution.
Answer: (b).
32. Beta nodes in Rete do:
(a) single-WME tests (b) joins on shared variables (c) WM modification (d) priority ordering.
Answer: (b).
33. Rete's main benefit is:
(a) deeper search (b) avoiding recomputation of matches across cycles (c) better heuristics
(d) parallelism.
Answer: (b).
34. Forward chaining is:
(a) data-driven (b) goal-driven (c) random (d) breadth-first.
Answer: (a).

End of Week 10.

11 Week 11: Constraint Satisfaction Problems

“Constraint Satisfaction Problems are a unifying formalism: large classes of problems can be expressed uniformly, and both search and reasoning can be used on them — even interleaved.”

— The unifying view

Goal of Week 11. For ten weeks we have alternated between two camps: *search* (state spaces, A^* , planning) and *reasoning* (rule-based systems). This week brings them together. **Constraint Satisfaction Problems (CSPs)** are a uniform way to pose problems so that an *off-the-shelf solver* can attack them with any combination of search and inference. We will (i) formalise CSPs and the constraint graph, (ii) study **Backtracking** with **dynamic variable ordering**, (iii) introduce **Arc Consistency** (AC-1, AC-3) and **Forward Checking**, (iv) meet the **Waltz algorithm** for interpreting line drawings, and (v) see how **model-based diagnosis** reduces to CSP.

11.1 Search vs. Reasoning, Unified

So far:

- **Search:** explore the state space / solution space by trial and error.
- **Reasoning:** represent knowledge in logic and draw inferences.

The Unifying Insight

CSPs subsume both. A user only needs to express the problem as a CSP, then use a solver that interleaves search (trying values) with reasoning (propagating constraints). This is one of the most influential ideas in modern AI — and the foundation of modern SMT/SAT solvers used in verification and combinatorial optimisation.

11.2 Mathematical Background: Relations

A **relation** on a set of variables is a subset of the Cartesian product of their domains. Two notations:

- **Extension form** — list the tuples explicitly: *LessThan* on $D = \{1, 4, 7, 9\}$ is $\{\langle 1, 4 \rangle, \langle 1, 7 \rangle, \langle 1, 9 \rangle, \langle 4, 7 \rangle, \langle 4, 9 \rangle, \langle 7, 9 \rangle\}$.
- **Intension form** — give a predicate: $\text{LessThan} = \{\langle x, y \rangle \mid x, y \in D, x < y\}$.

Relations of **arity** 1 (unary), 2 (binary), 3 (ternary), $\dots$, n (n -ary) generalise smoothly.

11.3 Constraint Satisfaction Problems — Formal Definition

Definition

A **CSP** (also called a **Constraint Network** $\mathcal{R}$) is a triple $\langle X, D, C \rangle$ where:

- $X = \{x_1, x_2, \dots, x_n\}$ — a set of **variable names**.
- $D = \{D_1, D_2, \dots, D_n\}$ — a set of **domains**, one per variable. We will restrict ourselves to *discrete*, *finite* domains.
- $C = \{c_1, c_2, \dots\}$ — a set of **constraints**. Each $c = (S, R)$ where $S \subseteq X$ is the **scope**

(variables involved) and R is a relation over S .

A **solution** is an assignment of a value $v_i \in D_i$ to every x_i such that *every* constraint is satisfied.

11.3.1 Binary Constraint Networks (BCNs)

When every constraint has scope of size 2 (**binary**), the CSP is a **BCN**. BCNs are visualised as a **constraint graph**:

- One node per variable, labelled with its domain.
- One edge per binary constraint, labelled with the relation (often inequality $\neq$, less-than $<$, etc.).

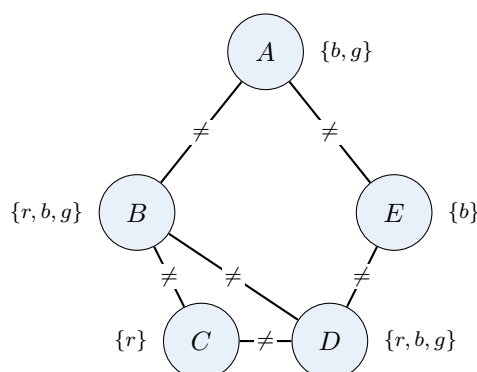
Universal relation by default: if two variables have no constraint between them, the implicit relation is *anything* — every value pair allowed.

11.4 Example: Map Colouring as a CSP

Problem: Colour each region so that no two adjacent regions share a colour.

CSP encoding:

- Variables = regions $\{A, B, C, D, E\}$.
- Domains = $\{r, g, b\}$ (or some subset per region).
- Constraints = $\neq$ on each adjacent pair.



A solution is an assignment (A, B, C, D, E) with all $\neq$ constraints satisfied.

11.5 Example: The n -Queens Problem

Problem: Place n queens on an $n \times n$ board such that no two queens attack each other (same row, column, or diagonal).

11.5.1 Encoding 1: Square-by-Square

- Variables: one per square (e.g. 36 variables for 6-Queens).
- Domains: $\{Q, \text{nil}\}$.
- Constraints: binary, between any two squares that attack each other.

This is wasteful — 36 variables for 6-Queens.

11.5.2 Encoding 2: Column-Based (much better)

- Variables: one per column $\{a, b, c, d, e, f\}$ (6 variables).
- Domains: $\{1, 2, 3, 4, 5, 6\}$ (the row where the queen goes).
- Constraints: R_{XY} = allowed row-pairs for queens in columns X and Y . E.g. $R_{ab} = \{(1, 3), (1, 4), (1, 5), (1, 6), (2, 1), \dots\}$ — pairs at non-attacking distance.

Intuition

The choice of encoding matters enormously. The column-based encoding builds in the constraint “one queen per column” *for free* (each column has one variable), eliminating millions of solutions before search even starts. **Good problem formulation is half the battle in CSP.**

11.6 Solutions and the Solution Relation

A CSP describes its solutions *implicitly*: through the constraints. The set of *all* solutions defines a relation ρ on the entire variable set:

$$\rho = R_{x_1 x_2 \dots x_n} = \{\text{tuples that satisfy every constraint}\}.$$

Solving a CSP = *extracting* an explicit solution from this implicit description. The CSP is like a *fog of possibilities*; solving clears the fog and reveals concrete assignments.

Remember This

Different solvers may extract *any* solution, *all* solutions, the *best* solution (optimisation), or just check *satisfiability* (is ρ non-empty?). The boolean version of CSP — does any satisfying assignment exist? — is the famous **SAT** problem when domains are $\{\text{true}, \text{false}\}$.

11.7 Algorithm Backtracking

The simplest CSP solver:

Algorithm 27 Backtracking

```

1: function BACKTRACKING( $X, D, C$ )
2:    $\mathcal{A} \leftarrow []$ ;  $i \leftarrow 1$ ;  $D'_i \leftarrow D_i$ 
3:   while  $1 \leq i \leq N$  do
4:      $a_i \leftarrow \text{SELECTVALUE}(D'_i, \mathcal{A}, C)$ 
5:     if  $a_i = \text{null}$  then
6:        $i \leftarrow i - 1$  ▷ backtrack
7:        $\mathcal{A} \leftarrow \text{tail } \mathcal{A}$ 
8:     else
9:        $\mathcal{A} \leftarrow a_i : \mathcal{A}$  ▷ augment
10:       $i \leftarrow i + 1$ 
11:      if  $i \leq N$  then  $D'_i \leftarrow D_i$ 
12:      end if
13:    end if
14:  end while
15:  return REVERSE( $\mathcal{A}$ )
16: end function

```

Algorithm 28 SelectValue (consistency check only)

```

1: function SELECTVALUE( $D'_i, \mathcal{A}, C$ )
2:   while  $D'_i$  not empty do
3:      $a_i \leftarrow \text{head } D'_i$ 
4:      $D'_i \leftarrow \text{tail } D'_i$ 
5:     if CONSISTENT( $a_i : \mathcal{A}$ ) then return  $a_i$ 
6:     end if
7:   end while return null
8: end function

```

This is **depth-first search with backtracking on conflict**. It is **complete** (will find a solution if one exists) and **sound** (returned assignments are valid).

The catch: in the worst case it explores $\prod_i |D_i|$ assignments — combinatorial explosion.

11.8 Dynamic Variable Ordering

Plain backtracking picks variables in fixed order. **Dynamic variable ordering** (DVO) picks at each step the variable that is *most constrained* now (smallest remaining domain) — a strategy called **minimum remaining values (MRV)** or **fail-first**.

Intuition

Fail fast. If a variable has only one possible value left, assigning it is forced — no real choice, no backtracking. By contrast, a variable with five remaining values is a real branching point. Postpone branching to as late as possible; tackle forced moves first.

11.8.1 Worked Example — Map Colouring with DVO and Propagation

Continuing the constraint graph above:

- Domain sizes: $A:2, B:3, C:1, D:3, E:1$. Pick C (smallest). $C = r$. *Propagate*: B, D each lose r .
- Now domains: $A:2, B:\{b, g\}, D:\{b, g\}, E:1$. Pick E (smallest, 1 value). $E = b$. *Propagate*: D loses b .
- Now: $A:\{b, g\}, B:\{b, g\}, D:\{g\}$. Pick D . $D = g$. *Propagate*: B loses g .
- Now: $A:\{b, g\}, B:\{b\}$. Pick B . $B = b$. A has $\{b, g\}$; either works.
- $A = g$. Done. No backtracking!

Combining DVO with constraint propagation eliminated all backtracking. On larger problems this can be the difference between hours and milliseconds.

11.9 Sudoku — Search vs. Reasoning

Sudoku is a 9×9 CSP:

- Variables: 81 cells.
- Domains: $\{1, 2, \dots, 9\}$ (or singleton for pre-filled cells).
- Constraints: *AllDifferent* on each row, column, and 3×3 box.

Human players reason: “5 is already in this row $\Rightarrow$ this empty cell cannot be 5.” This is *constraint propagation*. Brute-force search (try every digit in every empty cell) would take forever. Solvers combine both.

Remember This

Sudoku is the canonical pedagogical CSP. The harder Sudokus are designed so that pure logical reasoning gets stuck and forces some search; the easier ones are pure propagation.

11.10 Arc Consistency**11.10.1 Definition****Definition**

Let (X, Y) be an edge in the constraint graph. Variable X is **arc-consistent with respect to** Y if, for every value $a \in D_X$, there exists $b \in D_Y$ with $(a, b) \in R_{XY}$.

An **edge** (X, Y) is arc-consistent if *both* X is arc-consistent w.r.t. Y *and* Y is arc-consistent w.r.t. X .

A **network** is arc-consistent if every edge is arc-consistent.

11.10.2 The Revise Procedure

Algorithm 29 Revise — make X arc-consistent w.r.t. Y

```

1: function REVISE( $X, Y$ )
2:   for every  $a \in D_X$  do
3:     if no  $b \in D_Y$  with  $(a, b) \in R_{XY}$  then
4:       delete  $a$  from  $D_X$ 
5:     end if
6:   end for
7: end function

```

Complexity: $O(k^2)$ where $k = |D_X| \approx |D_Y|$.

11.10.3 Algorithm AC-1 — Brute Force

Algorithm 30 AC-1

```

1: function AC-1( $X, D, C$ )
2:   repeat
3:     for every edge  $(X, Y)$  in the constraint graph do
4:       REVISE( $X, Y$ )
5:       REVISE( $Y, X$ )
6:     end for
7:   until no domain changes in the cycle
8: end function

```

Why repeat? Because pruning a value from D_Y can break the arc-consistency of *another* edge (Z, Y) that was previously consistent. So we keep cycling until *no change* in a complete pass.

Complexity: $O(nek^3)$ where n = variables, e = edges, k = max domain size.

11.10.4 Algorithm AC-3 — Smarter Propagation

Plain AC-1 wastes effort by revising edges whose neighbours haven't changed. **AC-3** uses a worklist:

Algorithm 31 AC-3

```

1: function AC-3( $X, D, C$ )
2:    $Q \leftarrow$  all  $(P, T)$  such that  $(P, T)$  or  $(T, P)$  is an edge
3:   while  $Q$  non-empty do
4:      $(P, T) \leftarrow \text{head}(Q)$ ; remove from  $Q$ 
5:     REVISE( $P, T$ )
6:     if  $D_P$  changed then
7:       for each neighbour  $R \neq T$  of  $P$  do
8:         add  $(R, P)$  to  $Q$ 
9:       end for
10:    end if
11:  end while
12: end function

```

Complexity: $O(ek^3)$ — one factor of n better than AC-1.

Intuition

Constraint propagation = focus only on edges whose “support” has been threatened by a recent change. This is the same idea behind Rete (Week 10) — only do work when the input changes.

11.11 Higher-Order Consistency: i -Consistency**Definition**

A network is **i -consistent** if any consistent assignment to $(i - 1)$ variables can be extended to a consistent assignment to i variables (by assigning the i -th).

- **1-consistency** = *node consistency* (each domain satisfies its unary constraints).
- **2-consistency** = *arc consistency*.
- **3-consistency** = *path consistency* (any edge in the matching diagram extends to a triangle).
- Higher i = stronger pruning, more expensive.

The trade-off: higher consistency $\Rightarrow$ less backtracking later, but more preprocessing now. In practice, AC-3 (arc consistency) is the workhorse.

11.12 Lookahead Search: Forward Checking

Forward Checking (FC) combines backtracking search with on-the-fly arc consistency: every time a variable is assigned, prune all future variables’ domains to remove values inconsistent with the assignment.

Algorithm 32 Forward Checking

```

1: function FORWARDCHECKING( $X, D, C$ )
2:    $\mathcal{A} \leftarrow []$ 
3:   for  $k \leftarrow 1$  to  $N$  do  $D'_k \leftarrow D_k$ 
4:   end for
5:    $i \leftarrow 1$ 
6:   while  $1 \leq i \leq N$  do
7:      $a_i \leftarrow \text{SELECTVALUE-FC}(D'_i, \mathcal{A}, C)$ 
8:     if  $a_i = \text{null}$  then
9:       undo pruning from choosing  $a_{i-1}$ 
10:       $i \leftarrow i - 1$ 
11:       $\mathcal{A} \leftarrow \text{tail } \mathcal{A}$ 
12:    else
13:       $\mathcal{A} \leftarrow a_i : \mathcal{A}$ 
14:       $i \leftarrow i + 1$ 
15:    end if
16:  end while
17:  return REVERSE( $\mathcal{A}$ )
18: end function

```

Algorithm 33 SelectValue-FC (with forward pruning)

```

1: function SELECTVALUE-FC( $D'_i, \mathcal{A}, C$ )
2:   while  $D'_i$  not empty do
3:      $a_i \leftarrow \text{head } D'_i$ ;  $D'_i \leftarrow \text{tail } D'_i$ 
4:     for  $k \leftarrow i + 1$  to  $N$  do
5:       for each  $b \in D'_k$  do
6:         if not CONSISTENT( $b : a_i : \mathcal{A}$ ) then
7:           delete  $b$  from  $D'_k$ 
8:         end if
9:       end for
10:    end for
11:    if no  $D'_k$  became empty then return  $a_i$ 
12:    else
13:      undo deletes from this round
14:    end if
15:  end while
16:  return null
17: end function

```

11.12.1 FC on 6-Queens

Place Queen 1 in row 3. *Immediately prune*: in column 2, rows $\{2, 3, 4\}$ disallowed (own row + diagonals); in column 3, $\{1, 3, 5\}$; in column 4, $\{3, 6\}$; column 5, $\{3\}$; column 6, $\{3\}$. The remaining domains shrink dramatically before we even try Queen 2.

Effect: FC *detects dead-ends earlier* than plain backtracking, saving exploration.

Remember This

FC is incomplete in its propagation — it only ensures arc consistency between the *current* variable and future ones, not among all future variables. The stronger **Maintaining Arc**

Consistency (MAC) algorithm runs full AC-3 after each assignment.

11.13 The Waltz Algorithm — Interpreting Line Drawings

David Waltz (1972) used constraint propagation to interpret line drawings of 3-D objects — one of the earliest beautiful applications of CSPs.

11.13.1 Huffman's Labelling Scheme

David Huffman (1971) showed that for *tri-hedral* objects (where exactly three faces meet at each vertex), only four types of edge labels are possible:

- **Arrow** ($\rightarrow$): a convex edge of the object, with the object on the right of the arrow's direction.
- $+$: a convex edge (ridge).
- $-$: a concave edge (valley, e.g. where the object meets the table).
- **Ambiguous** (occlusion).

And only 18 types of vertices (junctions): L, T, Y/Fork, W/Arrow, etc. Each junction restricts which edge labels are possible.

11.13.2 The CSP Formulation

- Variables = vertices (junctions).
- Domains = the small set of valid label combinations for that junction type.
- Constraints = “edges must have the same label at both endpoints.”

11.13.3 Waltz's Algorithm

A propagation algorithm *between* AC-1 and AC-3 in sophistication: it propagates from vertex to vertex, removing impossible labels. In *most* natural drawings, propagation alone solves the problem with no search.

Waltz's extensions of Huffman's work:

- Allowed more than 3 edges per vertex.
- Handled *cracks*, *shadows*, and *light*.
- Labels grew from 4 to 50+, vertices to thousands.

The result is magical: a noisy line drawing is cleanly interpreted as a 3-D object, with shadow and crack lines automatically distinguished from object edges.

Real-World Application

Waltz's work was a milestone in computer vision and a vindication of the constraint-propagation paradigm. Patrick Winston dramatised it in his MIT lectures (“David Waltz — Constraint Propagation,” available on YouTube). The algorithm shows how a *small amount of geometric knowledge* (just the 18 junction types and 4 edge labels) plus propagation can do what would seem to need elaborate AI.

11.14 Model-Based Diagnosis

Model-based diagnosis (Reiter, de Kleer 1980s): given a model of a device, an observation of its inputs and outputs, and possibly some symptoms, find a *minimal set* of components that, if assumed broken, explains the discrepancy.

11.14.1 The Setup

A digital circuit is a CSP whose constraints come from the components:

- **Multiplier** M : $\text{ok}(M) \supset (\text{IN}_1 \times \text{IN}_2 = \text{OUT})$.
- **Adder** A : $\text{ok}(A) \supset (\text{IN}_1 + \text{IN}_2 = \text{OUT})$.
- **Connector**: $\text{OUT}_M = \text{IN}_A$ (same value).

If all components are OK, the constraints are consistent. If something is broken, the network becomes inconsistent.

11.14.2 Detecting and Localising Faults

Famous example: 3 multipliers + 2 adders. Inputs all $\{3, 2\}$. Expected outputs $F_1 = 12, F_2 = 12$. Observed: $F_1 = 10$.

This contradicts the predictions under the assumption $\{\text{ok}(M_1), \text{ok}(M_2), \text{ok}(A_1)\}$. We extract a **conflict set**: at least one of $\{M_1, M_2, A_1\}$ must be broken.

11.14.3 Candidate Space of Diagnoses

The **candidate space** is a lattice of subsets of components: singletons, pairs, triples, etc. The first observation eliminates the empty diagnosis. Further conflicts further constrain. **Minimal candidates** (smallest sets explaining all symptoms) are the most parsimonious diagnoses.

This is essentially **incremental SAT-solving on the network constraints**. Modern model-based diagnosis is used in:

- Spacecraft fault management (NASA's L2 lander).
- Automotive on-board diagnostics (OBD-II).
- Aerospace (engine health monitoring).
- Industrial automation.

11.15 CSPs as a Unifying Formalism

Many AI problems convert naturally to CSPs:

Problem	CSP encoding
SAT	domains = $\{T, F\}$; constraints = clauses
Map Colouring	vars = regions; constraints = $\neq$ adjacencies
Sudoku	81 cells; AllDifferent constraints on rows/cols/boxes
n-Queens	vars = columns; constraints = no-attack pairs
Scheduling	vars = tasks; domains = time slots; constraints = precedences, resources
Planning (SATPlan)	encode time-step variables and action effects
Diagnosis	vars = component ok flags; constraints = device equations
Cryptarithmic (e.g. SEND+MORE=MONEY)	vars = letters; domain $\{0..9\}$; AllDifferent + sum constraint

11.15.1 Modern CSP Solvers

State of the art: SMT solvers (Z3, CVC4), SAT solvers (MiniSAT, Glucose, CaDiCaL), CP solvers (Gecode, Choco, MiniZinc). These handle problems with millions of variables and clauses in seconds. They are at the heart of:

- Hardware and software verification.
- Optimising compiler back-ends.
- Cloud resource allocation.
- Bioinformatics (protein folding constraints).
- Recreational solvers (Sudoku phone apps).

11.16 Common Mistakes

Common Mistake

1. Confusing arc consistency (a property of the network) with backtracking (a search algorithm). They are complementary.
2. Thinking AC-1 is enough — one pass through all edges isn't, because pruning can re-break already-consistent edges.
3. Claiming arc-consistency $\Rightarrow$ solvable. Arc-consistency is necessary, not sufficient.
4. Treating AC-3's complexity as $O(ek^3)$ in all cases — it's the worst case; many real networks finish much faster.
5. Using fixed variable ordering when dynamic ordering would slash backtracking by orders of magnitude.
6. In Forward Checking, forgetting to *undo* the prunings on backtrack — leads to incorrect “empty domain” detection later.
7. Encoding a problem as a CSP without minimising the variables — e.g. using 81 squares for n -queens when 9 columns suffice.
8. Confusing the constraint graph (visual) with the matching diagram (concrete value pairings).

11.17 Real-World Applications

Real-World Application

Time-tabling: every university uses CSP-based systems to assign courses to slots, rooms, and faculty without clashes.

Vehicle routing and crew scheduling: airlines (American, Delta), logistics (UPS, FedEx, Amazon) use CSP/integer programming hybrids.

Hardware verification: Intel, AMD, ARM use SAT/SMT to formally verify CPU designs before fabrication — catching billion-dollar bugs.

Compilers: register allocation, instruction scheduling, code generation all use CSP techniques.

Bioinformatics: protein side-chain placement, DNA sequence design.

Computer vision: from Waltz to modern stereo matching and image segmentation (energy

minimisation = CSP-like).

Recreational: Sudoku, KenKen, crosswords — all CSP-friendly.

AI Planning: SATPlan, GraphPlan, and modern planners encode plans as CSPs and solve with SAT/SMT.

Diagnosis: spacecraft, automobiles, telecom equipment all use model-based diagnosis from CSP roots.

11.18 Quick Revision — Week 11

Quick Revision

- **CSP** = $\langle X, D, C \rangle$. Variables + domains + constraints. Solution = assignment satisfying all constraints.
- **Binary CSP** has only 2-arity constraints; visualised as a constraint graph.
- **n -Queens encoding:** column-based is much better than square-based.
- **Backtracking:** depth-first search with chronological backtracking on conflict.
- **Dynamic Variable Ordering (DVO / MRV / fail-first):** pick the variable with smallest current domain.
- **Arc Consistency:** X is AC w.r.t. Y if every value of X has at least one supporting value in Y .
- **Revise:** $O(k^2)$. **AC-1:** $O(nek^3)$, cycle until no change. **AC-3:** $O(ek^3)$, work-list driven.
- **i -consistency:** 1=node, 2=arc, 3=path. Higher = more pruning but more cost.
- **Forward Checking:** prune future domains after each assignment, undo on backtrack.
- **Waltz algorithm:** constraint propagation for interpreting line drawings (Huffman labels, junction types).
- **Model-Based Diagnosis:** faults represented as ok flags; inconsistency identifies conflict sets; minimal candidates are diagnoses.
- **CSPs unify** SAT, planning, scheduling, configuration, diagnosis under one solver framework.

11.19 Most Important Quiz Points — Week 11

Exam Tip

1. A **CSP** is a triple $\langle X, D, C \rangle$.
2. A constraint $c = (S, R)$ has a *scope* S and a relation R .
3. Arc consistency: $\forall a \in D_X, \exists b \in D_Y$ such that $(a, b) \in R_{XY}$.
4. One pass through all edges is *not* enough — domains can re-break arc-consistency elsewhere.
5. AC-1 complexity: $O(nek^3)$. AC-3: $O(ek^3)$.
6. DVO / Minimum Remaining Values / fail-first principle: pick most-constrained variable first.

7. Higher i -consistency $\Rightarrow$ less backtracking but more preprocessing.
8. Forward Checking deletes future-variable values inconsistent with the current assignment and *must undo* on backtrack.
9. Huffman labels: 4 edge types (Arrow $\rightarrow$, $+$, $-$, ambiguous); 18 junction types.
10. Waltz extended Huffman to handle cracks/shadows; uses constraint propagation vertex-by-vertex.
11. Model-based diagnosis represents faults via `ok(.)` flags; inconsistencies $\Rightarrow$ conflict sets $\Rightarrow$ minimal candidate diagnoses.
12. Many AI problems (SAT, planning, scheduling, configuration, diagnosis) reduce to CSPs.

11.20 Practice Questions — Week 11

Conceptual.

1. Define a CSP. What are its three components?
2. What is the difference between a binary CSP and a general CSP?
3. Define arc consistency mathematically.
4. Why is one pass of **Revise** not enough for arc consistency?
5. Distinguish AC-1 and AC-3. Give their complexities.
6. State the principle of dynamic variable ordering. Why does it usually help?
7. Define i -consistency. What is 1-, 2-, 3-consistency in plain terms?
8. Describe Forward Checking and explain why it must undo prunings on backtrack.
9. Describe the Huffman labelling scheme (edge types and junction types).
10. Explain how Waltz's algorithm interprets line drawings via constraint propagation.
11. Define model-based diagnosis. What is a conflict set? What is a minimal candidate?
12. List five AI problems that can be encoded as CSPs.

Worked / numerical.

13. Formalise the 4-Queens problem as a CSP with one variable per column. Write out R_{12} , R_{13} , and R_{14} .
14. Apply AC-3 to the map-colouring example with $D_A = \{r, g, b\}$, $D_B = \{r, g\}$, $D_C = \{g\}$, $D_D = \{r, g, b\}$ and constraints $\neq$ on $(A, B), (A, C), (A, D), (B, C), (C, D)$. Show the final domains.
15. Solve the cryptarithmic puzzle SEND+MORE=MONEY as a CSP. Identify variables, domains, and constraints.
16. Trace Backtracking + DVO on the 4-colour map-colouring problem of your choice.
17. Build the constraint network for a 2-multiplier + 1-adder circuit. Show how inconsistency localises the fault.

Algorithmic.

18. Write pseudocode for AC-3 in detail. Why does it use a work-list?

19. Modify Backtracking to use both DVO and Forward Checking.
20. Sketch the Waltz algorithm in pseudocode.
21. Prove that if a CSP is arc-consistent, it may still have no solution. Give a small counter-example.

MCQ-style.

22. A CSP is a triple:
(a) (S, A, γ) (b) (X, D, C) (c) (g, h, f) (d) (A, O, L, B) .
Answer: (b).
23. Arc consistency on edge (X, Y) ensures:
(a) every value of X has at least one supporting value in Y (b) X and Y have the same domain (c) no values are pruned (d) the network has a solution.
Answer: (a).
24. AC-3 has complexity:
(a) $O(n)$ (b) $O(ek^3)$ (c) $O(nek^3)$ (d) $O(2^n)$.
Answer: (b).
25. Dynamic Variable Ordering means:
(a) pick variables in alphabetical order (b) pick variables with the smallest remaining domain (c) pick variables with the largest domain (d) random order.
Answer: (b).
26. Forward Checking, after assigning a value, prunes:
(a) past variables' domains (b) future variables' domains (c) all domains (d) nothing.
Answer: (b).
27. Huffman's line-drawing scheme has:
(a) 4 edge types and 18 junctions (b) 50 edge types and 1000 junctions (c) 3 edge types and 7 junctions (d) infinite types.
Answer: (a).
28. The Waltz algorithm extends Huffman by:
(a) using deep learning (b) handling cracks, shadows, and more vertex types (c) using SAT solvers (d) using A*.
Answer: (b).
29. In model-based diagnosis, a *conflict set* is:
(a) the set of all components (b) a set of components that cannot all be OK simultaneously (c) the goal (d) the working memory.
Answer: (b).
30. 3-consistency is also called:
(a) node consistency (b) arc consistency (c) path consistency (d) global consistency.
Answer: (c).
31. Sudoku is solved by:
(a) pure brute force (b) a mix of constraint propagation and search (c) deep learning (d) random guessing.
Answer: (b).

End of Week 11.

12 Week 12: Course Revision and the Big Picture of AI

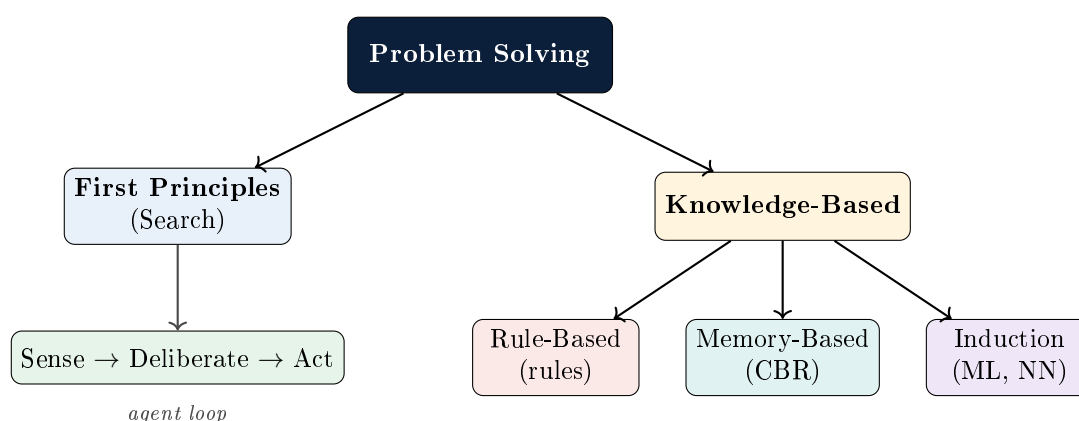
“You can solve problems by working from first principles (search) or by leveraging knowledge (rules, memory, induction). Real intelligence interleaves both.”

— The grand synthesis

Goal of Week 12. For eleven weeks we have built up a toolbox of algorithms for problem solving: search algorithms (DFS, BFS, A*, AO*, alpha-beta, SSS*), planning algorithms (FSSP, BSSP, GSP, POP), and reasoning algorithms (forward and backward chaining, Rete, AC-3, Forward Checking). This last week we step back and ask: *how does it all fit together?* We synthesize the course into one coherent picture, look at limits of finite-state methods, glance ahead to **probabilistic reasoning**, **causality**, **qualitative reasoning**, and **machine learning** as the frontier directions for AI, and consolidate the practical exam-preparation toolkit.

12.1 The Master Taxonomy of Problem-Solving

Khemani’s wrap-up diagram organises the whole field under one roof:



Two complementary strategies:

- **First Principles (search-based).** Don’t trust any prior knowledge; explore systematically. Used when (i) the problem is novel, (ii) good rules are unavailable, (iii) we have time to think.
- **Knowledge-Based.** Leverage what experts know — by encoding it as *rules*, by remembering *cases*, or by *learning* patterns from data.

Real systems mix both. Deep Blue beat Kasparov by combining a heuristic evaluator (knowledge) with massive alpha-beta search (first principles). AlphaGo combined learned evaluation + Monte Carlo search.

12.2 Summary of All Twelve Weeks

12.2.1 The Algorithm Family Tree

Week	Algorithm / Topic	Key contribution
1	Turing Test, Winograd Schema	The standards for “machine intelligence”
2	DFS, BFS, IDS	The uninformed-search backbone
3	Hill Climbing, Beam, Tabu, SA	Heuristic + local search
4	GA, ACO	Population-based search
5	A*	Optimal informed search
6	Consistency, BFHS, BSS, DCBSS	Space-saving A* variants
7	Minimax, α - β , SSS*	Adversarial search
8	STRIPS, FSSP, BSSP, GSP, POP	Classical planning
9	Goal trees, AO*	Problem decomposition
10	OPS5, Rete	Forward-chaining rule systems
11	CSP, AC-3, FC, Waltz	Constraints and unified solving
12	The big picture	Synthesis + future

12.2.2 Complexity Cheat-Sheet

Algorithm	Time	Space	Complete?	Optimal?
DFS	$O(b^m)$	$O(bm)$	N (cyclic graphs)	N
BFS	$O(b^d)$	$O(b^d)$	Y	Y (unit-cost)
DFID	$O(b^d)$	$O(bd)$	Y	Y
DBDFS_d	$O(b^d)$	$O(bd)$	Y (within depth)	N
Hill Climbing	varies	$O(b)$	N (local optima)	N
Simulated Annealing	varies	$O(1)$	N	probabilistic
A*	$O(b^d)$	$O(b^d)$	Y	Y (admissible h)
A* w/ consistent h	same	same	Y	Y
Beam Search	$O(wd)$	$O(wd)$	N	N
BSS / DCBSS	high	$O(1)$	Y	Y
Minimax	$O(b^d)$	$O(bd)$	Y (finite)	Y (zero-sum)
Alpha-Beta (best ord.)	$O(b^{d/2})$	$O(bd)$	Y	Y
GSP	exp	exp	sometimes N	N
POP	exp	exp	Y	Y
AO*	exp	exp	Y	Y (h under-est.)
Rete	poly per cycle	big network	—	—
AC-3	$O(ek^3)$	$O(ek)$	—	—
Backtracking + FC + DVO	exp worst	$O(n)$	Y	Y

12.3 Limits of What We've Built — Frontier Topics

The algorithms in this course are *symbolic, discrete, and largely deterministic*. The real world is none of these. Several big extensions take AI beyond these limits:

12.3.1 Probabilistic Reasoning and Bayesian Networks

When the world is uncertain, we want to compute *probabilities* not just truths. A **Bayesian Belief Network** is a directed graph where:

- Nodes are random variables.
- Edges encode causal / probabilistic dependencies.
- Each node has a conditional probability table $P(X \mid \text{parents}(X))$.

Inference: compute $P(\text{query} \mid \text{evidence})$. **Exact** inference is generally #P-hard; **approximate** inference uses sampling (Gibbs, importance sampling, MCMC).

Bayesian networks unify rule-based reasoning, diagnosis, vision, speech recognition, and machine learning under one probabilistic framework. **Judea Pearl** (2011 Turing Award) developed the foundational theory.

12.3.2 Causality

Pearl's later work (*The Book of Why*, 2018) argues that AI needs more than correlation — it needs **causality**. Three rungs on the *ladder of causation*:

- **Association** (level 1): “If I observe X , what's Y ?” — what ML does today.
- **Intervention** (level 2): “If I do X , what happens?” — requires causal model.
- **Counterfactuals** (level 3): “What would Y be *if X had been different*?” — needs structural causal models.

Example: jaundice $\rightarrow$ yellow eyes (cause $\rightarrow$ effect). The causal direction matters: treating the yellow eyes doesn't cure jaundice; treating jaundice does cure the yellow eyes.

12.3.3 Three Forms of Reasoning

Deduction, Induction, Abduction

- **Deduction** (*sound*): from a rule $P \Rightarrow Q$ and fact P , conclude Q . The basis of logic, Prolog, and theorem proving. Truth-preserving.
- **Induction** (*generalising*): from many (P_i, Q_i) examples, hypothesise the rule $P \Rightarrow Q$. The basis of **machine learning**. Not sound — induced rules may be wrong on new data.
- **Abduction** (*explaining*): from observation Q and rule $P \Rightarrow Q$, hypothesise P as a cause. The basis of **diagnosis**. Not sound — there may be other causes.

This course mostly covered *deduction* (Prolog, rule systems) and used *some* induction (GA, ACO learning weights). Modern AI heavily uses all three.

12.3.4 Machine Learning and Neural Networks

In the 2010s, **deep learning** revolutionised AI:

- Vision: ImageNet (2012), object detection, segmentation.

- Language: word embeddings, transformers, GPT family.
- Games: AlphaGo, AlphaZero (combined NN with MCTS).
- Robotics: end-to-end learning of policies.

Key shift: from hand-crafted rules to learned representations. But ML still benefits hugely from the search and reasoning frameworks of this course — modern systems are hybrids.

12.3.5 Qualitative Reasoning

When precise numerical models aren't available, **qualitative reasoning** (Kuipers, Forbus) reasons about increasing / decreasing / constant magnitudes.

Example (boiler with super-heater): water flows in at temperature T_{in} , is heated to 100°C , becomes steam in the super-heater, exits at T_{out} . Without knowing exact thermodynamic equations, we can reason: *if T_{in} rises, T_{out} rises (qualitatively)*. Useful in expert systems where exact models are infeasible.

12.3.6 Multi-Agent Systems and Game Theory

Going beyond two-player zero-sum: **multi-agent reinforcement learning, mechanism design** (auctions, voting), **coalition formation, cooperation and trust**. Foundations of automated negotiation, ad auctions, smart contracts.

12.3.7 The Voices of Skepticism

- **Gary Marcus & Ernest Davis** (*Rebooting AI*, 2019): argue that pure deep learning is brittle and lacks reasoning. We need hybrid systems combining the symbolic methods of this course with deep learning.
- **Erik Larson** (*The Myth of Artificial Intelligence*, 2021): argues that abductive inference — *the kind humans use to form hypotheses* — is fundamentally beyond statistical learning.

The synthesis lesson: symbolic AI (this course) and statistical AI (deep learning) are *complementary*, not competitors.

12.4 A Reasoning Puzzle to Test Understanding

Three friends A , B , C are wearing caps that are either black or white. They can see each other's caps but not their own. Each must announce whether they know their own cap colour. Assume they're perfect logicians.

If C sees both A and B wearing white, C can't be sure. If C sees a black and a white, C still can't be sure. *Q1: Does A know? Q2: Does B know? Q3: Does C now know?*

This puzzle illustrates **higher-order reasoning** — reasoning about *what others know*. Modal logic, epistemic logic, and game theory all live here. Whole subfields of AI.

12.5 Recurring Themes Across the Course

Several big ideas recurred:

The Recurring Themes

1. **Generate-and-Test**: enumerate candidates, test against goal. Every search algorithm is a refinement of this.
2. **Heuristics**: domain-specific shortcuts that bias the search. Trade optimality for efficiency.
3. **Trade-offs**: time vs space, completeness vs efficiency, optimality vs feasibility. No free lunch.
4. **Best-First**: when you can rank candidates, expand the best one first. A*, AO*, SSS*, beam search all share this idea.
5. **Forward / Backward**: most paradigms come in both flavours. FSSP/BSSP, forward/backward chaining, progression/regression.
6. **Least Commitment**: defer decisions until forced (POP, dynamic variable ordering).
7. **Propagation**: every change has consequences; propagating them eagerly saves later work (Rete, AC-3, Waltz).
8. **Refinement**: maintain partial solutions, refine them step by step (POP, AO*, SSS*).
9. **Symmetry-breaking**: avoid exploring equivalent states (CLOSED list, refractoriness, transposition tables).
10. **Caching**: memoise computations to avoid re-doing them (Rete's inter-cycle memory, transposition tables).

12.6 Course-Wide Comparison Tables

12.6.1 Search Method Selection

If ...	Use ...
no heuristic available, finite tree	BFS or DFID
no heuristic, very deep tree	DFS with cycle detection
have heuristic, want optimum	A* (admissible h)
have heuristic, tight memory	IDA*, RBFS, or BFHS
have heuristic, h rough, want quick answer	Weighted A* or Beam
local optimisation (TSP, scheduling)	SA, Tabu, GA, ACO
adversarial 2-player zero-sum	Alpha-Beta with move ordering
problem decomposes into independent sub-goals	AO* / goal trees
configuring or planning with operators	FSSP $\rightarrow$ BSSP $\rightarrow$ POP
constraint satisfaction (SAT, scheduling, etc.)	Backtracking + FC + AC-3
domain has many rules, large WM	Rule-based system with Rete

12.6.2 Algorithm Pairings That Reveal Trade-offs

Pair	Trade-off
DFS vs BFS	space vs completeness
BFS vs DFID	memory vs (slight) recomputation
A* vs IDA*	memory vs time
A* vs Weighted A*	optimality vs speed
Minimax vs Alpha-Beta	exhaustive vs pruned
GSP vs POP	linear plan vs least commitment
Forward vs Backward chaining	data-driven vs goal-driven
AC-1 vs AC-3	simple vs worklist-efficient
Backtracking vs Forward Checking	blind vs look-ahead

12.7 Definitions You Must Know Cold for the Exam

Remember This

- **Admissibility:** $h(n) \leq h^*(n)$ for all n .
- **Consistency (monotonicity):** $h(m) \leq k(m, n) + h(n)$ for every edge.
- **Completeness:** algorithm finds a solution if one exists.
- **Optimality:** algorithm finds the lowest-cost solution.
- **Branching factor b :** number of successors per node.
- **Depth d :** depth of the shallowest goal.
- **Heuristic function $h(n)$:** estimate of cost from n to goal.
- $f(n) = g(n) + h(n)$: A*'s evaluation function.
- **Strategy** (game theory): subtree picking one move at every Max node.
- **Production:** pattern $\Rightarrow$ action rule.
- **WME:** working memory element.
- **Conflict set:** matched (rule, WME-tuple) instances.
- **CSP:** $\langle X, D, C \rangle$.
- **Arc consistency:** $\forall a \in D_X, \exists b \in D_Y : (a, b) \in R_{XY}$.
- **Open goal / Threat:** the two POP flaws.
- **Forward / Backward chaining:** data-driven / goal-driven inference.

12.8 Names & Dates to Recognise

Person	Year(s)	Contribution
Alan Turing	1950	The Imitation Game (Turing Test)
Claude Shannon	1950	Programming a Computer for Playing Chess; chess eval formula
McCarthy, Minsky, et al.	1956	Dartmouth Conference; “AI” term coined
Newell & Simon	1957–72	GPS, Means-Ends Analysis, Logic Theorist, info-processing model
Edsger Dijkstra	1959	Dijkstra’s shortest-path algorithm
Hart, Nilsson, Raphael	1968	A* algorithm
Patrick Winston / D. Huffman	1971	Line-drawing labelling (precursor to Waltz)
Richard Fikes & Nils Nilsson	1971	STRIPS planner
D. Waltz	1972	Constraint propagation for line drawings
Alain Colmerauer	1972	Prolog
Gerald Sussman	1973	Sussman’s Anomaly
G. Stockman	1979	SSS* algorithm
Charles Forgy	1979	Rete algorithm, OPS5
J. McDermott	1978	R1/XCON expert system
A. Martelli, U. Montanari	1978	AO* algorithm
J. Pohl	1970	Weighted A*
N. Nilsson	1980	Definitive AO* treatment
J. Holland	1975	Genetic Algorithms
Goldberg	1989	Popularised GAs
Dorigo	1992	Ant Colony Optimization
Kirkpatrick et al.	1983	Simulated Annealing
Korf & Zhang	2000	Frontier Search
Zhou & Hansen	2003–05	SMGS, BFHS, BSS, DCBSS
Judea Pearl	1988+	Bayesian Networks, causality (Turing Award 2011)
H. Levesque	2011	Winograd Schema Challenge
Garry Kasparov vs. Deep Blue	1997	Computer beats world chess champion
Lee Sedol vs. AlphaGo	2016	DeepMind beats world Go champion
AlphaGo Zero	2017	Learning from scratch via self-play
Pluribus	2019	Multi-player poker (CMU)

12.9 One-Page Exam Survival Sheet

Quick Revision

Best-first ordering function: $f(n) = g(n) + w \cdot h(n)$.

$w = 0$: Branch & Bound. $w = 1$: A*. $w > 1$: Weighted A*. $w = \infty$: Best-First.

Heuristic conditions. Admissible: $h(n) \leq h^*(n)$. Consistent: $h(m) \leq k(m, n) + h(n)$. Consistent $\Rightarrow$ admissible (not vice versa).

A* with consistent h : no Case-3 reopening. f -values non-decreasing along any path.

Alpha-Beta with best ordering: $O(b^{d/2})$. Cutoff when $\alpha \geq \beta$.

STRIPS progression: $S' = (S \cup E^+) \setminus E^-$.

STRIPS regression: $G' = (G \setminus E^+) \cup \text{pre}(a)$.

Goal tree costs. AND: $\sum_i (e_i + c_i)$. OR: $\min_i (e_i + c_i)$.

POP flaws: open goals and threats.

Threat resolution: promotion, demotion, separation.

Rete: alpha nodes (single-WME tests) + beta nodes (joins on shared variables). Refractoriness automatic.

CSP arc consistency: $\forall a \in D_X, \exists b \in D_Y : (a, b) \in R_{XY}$.

AC-1: $O(nk^3)$. **AC-3:** $O(ek^3)$.

DVO/MRV = pick variable with smallest remaining domain. “Fail-first.”

Forward checking: prune future domains, undo on backtrack.

12.10 Final Synthesis Practice Questions

1. Compare and contrast *four* algorithms from this course in terms of (i) admissibility, (ii) space, (iii) time, (iv) suitability for a stated problem.
2. Given a problem (e.g. Sudoku, route-planning, configuration), justify which algorithm(s) from the course you would choose and why.
3. Explain the recurring theme of “best-first” in A*, AO*, Beam Search, and SSS*.
4. Compare “goal trees” (Week 9) with “game trees” (Week 7). What’s similar? What’s different?
5. Describe a hybrid that combines *search* (one of Weeks 2–7) with *reasoning* (one of Weeks 8–11).
6. For each of the three forms of inference — deduction, induction, abduction — give one algorithm in this course that exemplifies it.
7. Pick three real-world AI systems (Deep Blue, AlphaGo, R1/XCON, Prolog) and trace which course concepts they embody.
8. Identify *five* trade-offs that recur across the course, each captured by a pair of algorithms.
9. Explain why **least commitment** (POP, DVO) and **propagation** (Rete, AC-3) often go together in modern systems.
10. Where do the algorithms in this course *stop* working? What real-world phenomena (stochasticity, partial observability, multi-agent dynamics, learning) push us beyond?

12.11 Closing Thoughts

A Final Word

The algorithms in this course were invented between roughly 1956 and 2005, and many of them still drive production AI systems today. Modern deep-learning systems sit *on top of* these classical methods, not in place of them:

- AlphaGo = Monte Carlo Tree Search (game-tree variant) + neural networks.
- Modern planners = STRIPS + heuristic-extraction networks.
- Industrial-strength SAT solvers = Backtracking + learned clauses + propagation.
- LLM agents = forward chaining + heuristic prompts.

You now have the foundations. The next steps — probabilistic reasoning, machine learning, deep learning, reinforcement learning, causal inference — will rest on this bedrock.

Search, reason, learn. Combine.

End of Week 12 — End of the Course.